SYMBOLIC ANALYSIS OF LARGE ANALOG CIRCUITS WITH DETERMINANT DECISION DIAGRAMS

by

Xiang-Dong Tan

A thesis submitted in partial fulfillment
of the requirements for the Doctor of Philosophy
degree in Electrical and Computer Engineering in the
Graduate College of
The University of Iowa

May 1999

Thesis supervisor: Adjunct Assistant Professor C.-J. Richard Shi

Graduate College
The University of Iowa
Iowa City, Iowa

CERTIFICATE OF APPROVAL

PH.D. THESIS

This is to certify that the Ph.D. thesis of

Xiang-Dong Tan

has been approved by the Examining Committee
for the thesis requirement for the Doctor of
Philosophy degree in Electrical and Computer Engineering
at the May 1999 graduation.

Thesis committee: _____
Thesis supervisor

_____
Member

_____
Member

_____
Member

_____
Member

To my parents,
Lin-Sen Tan and Wan-Hua Mao

ACKNOWLEDGEMENTS

I am sincerely grateful to my graduate advisor, Prof. C.-J. Richard Shi, for his superior guidance, constant support and encouragement throughout my Ph.D study at the University of Iowa and the University of Washington. His many insightful ideas and suggestions during the course of this dissertation have improved both the results and their representations.

I am grateful to other members of my examining committee, Professors Sudhakar M. Reddy, Irith Pomeranz, Teodor Rus and Jon G. Kuhl, for their time and support on my proposals, dissertation and examinations. Many ideas in this dissertation have been inspired by what I learned in their excellent courses.

I am indebted to my colleagues Dragos Lungeanu, Youcef Bourai and Tao Pi in the VLSI Mixed-Signal Research Laboratory, UW. Dragos who was my roommate, brought much fun to my life while providing valuable critiques of my papers and this thesis. I thank Youcef for many helpful discussions on graph theory. Thanks are due to many of my friends, especially Rui-Feng Guo for his constant help.

I thank my parents for their unwavering affection and support over the past years. My wife, Yan Ye, deserves my deepest love and appreciation. This thesis may not have been completed without her emotional sustenance.

Finally, I would like to acknowledge the crucial financial assistance through research grants from U.S. Defense Advanced Research Projects Agency (DARPA), Wright Laboratory, Manufacturing Technology Directorate, and Rockwell Semiconductor Systems.

TABLE OF CONTENTS

Page

LIST OF TABLES vii

LIST OF FIGURES viii

CHAPTER

I INTRODUCTION 1

1.1 Motivations 1
1.2 Objectives and Results of This Thesis 2
1.3 Organization of This Thesis 4

II A REVIEW OF SYMBOLIC CIRCUIT ANALYSIS 5

2.1 Applications of Symbolic Analysis 5
2.1.1 Iterative Formula Evaluation 6
2.1.2 Insight into Circuit Behaviors 6
2.1.3 Other Applications 8
2.2 Review of Symbolic Analysis Techniques 8
2.2.1 Problem Formulation for Symbolic Analysis 8
2.2.2 Tree Enumeration Method 9
2.2.3 Signal Flow Graph Method 11
2.2.4 Numerical Interpolation Method 12
2.2.5 Parameter Extraction Method 13
2.2.6 Matrix-Determinant Method 14
2.2.7 The State of the Art in Symbolic Analysis 15
2.2.8 Other Research Topics 18
2.3 Summary 20

III EXACT SYMBOLIC ANALYSIS OF ANALOG CIRCUITS VIA DETERMINANT DECISION DIAGRAMS 21

3.1 Introduction 21
3.2 Preliminaries 22
3.2.1 Subset Systems and Zero-suppressed Binary Decision Diagrams 22
3.2.2 Matrix, Determinant and Cofactors 24
3.3 ZBDD Representation of Symbolic Matrix Determinant 26
3.4 An Effective Vertex-Ordering Heuristic 31
3.5 Manipulation and Construction of Determinant Decision Diagrams 39
3.5.1 Implementation of Basic Operations 39

3.6 Experimental Results 42
3.7 Summary 48

IV S-EXPANDED DETERMINANT DECISION DIAGRAMS 50

4.1 Introduction 50
4.2 s-Expanded Symbolic Representation 52
4.3 Vertex Ordering for s-Expanded DDDs 58
4.4 Construction of s-Expanded DDDs 60
4.4.1 The Construction Algorithm 60
4.4.2 Time and Space Complexity Analysis 63
4.5 Experimental Results 67
4.6 Summary 70

V HIERARCHICAL SYMBOLIC ANALYSIS OF ANALOG CIRCUITS 72

5.1 Introduction 72
5.2 Overview of Hierarchical Circuit Analysis 73
5.3 Hierarchical Analysis Using Determinants and Cofactors 76
5.4 An Illustration Example 78
5.5 Hierarchical Symbolic Analysis Procedure 86
5.6 Experimental Results 87
5.7 Summary 93

VI BALANCED PARTITIONING OF ANALOG INTEGRATED CIRCUITS FOR HIERARCHICAL SYMBOLIC ANALYSIS 95

6.1 Introduction 95
6.2 Problem Formulation 96
6.3 Multi-Way, Multi-Level Balanced Partitioning Algorithm 100
6.3.1 Initial Partition Construction 100
6.3.2 Iterative Refinement Algorithm 103
6.3.3 Balance Constraint Relaxation 107
6.3.4 Multi-Way Tree Partitioning 110
6.4 Experimental Results 110
6.4.1 $\mu A741$ Operational Amplifier 110
6.4.2 A Band-Pass Filter 113
6.4.3 Effectiveness of InitPart 115
6.4.4 Comparison with Contour Tableau Method 117
6.5 Summary 118

VII INTERPRETABLE SYMBOLIC SMALL-SIGNAL CHARACTERIZATION OF ANALOG INTEGRATED CIRCUITS 120

7.1 Introduction 120
7.2 A Framework for Interpretable Small-Signal Characterization 121
7.3 Discarding Insignificant Terms 123
7.3.1 Device Elimination 124

7.3.2 Node Contraction 124
7.4 Coefficient Suppression 126
7.5 Elimination of Canceling Terms 127
7.6 Generation of Dominant-Terms 131
7.7 Error Control Scheme 132
7.8 Experimental Results 133
7.8.1 TwoStage Operational Amplifier 133
7.8.2 Uncompensated CMOS Cascode Operational Amplifier . 135
7.8.3 $\mu A741$ Operational Amplifier 136
7.8.4 Discussions 137
7.9 Summary 141

VIII SYMBOLIC CIRCUIT-NOISE ANALYSIS AND MODELING 143

8.1 Introduction 143
8.2 Noise Models and Symbolic Noise Analysis 144
8.2.1 Noise Models 144
8.2.2 Symbolic Noise Analysis 146
8.2.3 New Circuit-Noise Modeling Method 147
8.2.4 Symbolic Transfer Function Computation 147
8.2.5 Generations of Noise Spectrum Density Functions 150
8.3 Experimental Results 151
8.4 Summary 155

IX CONCLUSIONS AND FUTURE WORK 156

9.1 Summary of Research Contributions 156
9.1.1 DDDs and s-Expanded DDDs 156
9.1.2 Hierarchical Decomposition and Approximations 158
9.1.3 Symbolic Circuit-Noise Analysis and Modeling 160
9.2 Future Research Topics 161

REFERENCES 164

LIST OF TABLES

Table		Page
1	Summary of basic operations	41
2	Comparison of DDD sizes without/with vertex-ordering and SCAPP	45
3	Comparison of the proposed algorithm against ISAAC and Maple-V	47
4	Comparison of frequency analysis by the proposed algorithm and by SPICE	49
5	Statistics on complex DDD and s-expanded DDD construction	68
6	Comparison against SPICE in numerical evaluation	70
7	Comparison against SPICE in numerical evaluation	90
8	Comparison against SCAPP	91
9	Comparison against SCAPP and SPICE in CPU time	92
10	Statistics for three-level, two-way partitioning on $\mu A741$	93
11	Statistics of two-level, multi-way partitioning on $\mu A741$	112
12	Statistics for three-level, two-way partitioning on $\mu A741$	114
13	Statistics of three-way, tree partitioning of the band-pass filter	115
14	Comparison between two initial partition construction algorithms	116
15	Partitioning results on band-pass filter by the contour tableau method	118
16	Zeros and poles for two-stage Opamp	134
17	Statistics for simplified symbolic expressions	142
18	Statistics of noise analysis for analog circuits	152

LIST OF FIGURES

Figure		Page
1	A small signal model for an emitter follower	7
2	Zero-suppressed BDD	23
3	A ZBDD representing $\{adgi, adhi, afej, cbgj, cbih\}$ under ordering $a > c > b > d > f > e > g > i > h > j$	27
4	A signed ZBDD for representing symbolic terms	29
5	A determinant decision diagram for matrix $\mathbf{M}$	31
6	The DDD representing $det(\mathbf{M})$ under ordering $a > c > d > f > g > h > b > e > i > j$	32
7	A DDD vertex-ordering heuristic	33
8	An illustration of DDD vertex-ordering heuristic	34
9	An illustration of DDD construction for band matrices	37
10	A comparison of DDD sizes versus numbers of product terms for band matrices	38
11	A three-section ladder network	38
12	Implementation of basic operations for symbolic analysis	40
13	Small-signal models for nonlinear devices	43
14	The circuit schematic of bipolar $\mu A741$	44
15	The circuit schematic of MOS Cascode Opamp	44
16	An example circuit	53
17	Complex DDD for a matrix determinant	54
18	An s-expanded DDD by the first labeling scheme	56
19	An s-expanded DDD by the second labeling scheme	58
20	The s-expanded DDD construction with the first labeling scheme	61

21 The basic s-expanded DDD construction algorithms 62
22 The s-expanded DDD construction with the second labeling scheme . . . 62
23 MULTIPLY($P[i], D.x$) operation 64
24 UNION($P1[i], P2[i]$) operation 65
25 Product term distribution of μA741 by the first labeling scheme 69
26 Product term distribution of μA741 by the second labeling scheme . . . 69
27 Model of a circuit hierarchy 74
28 A partitioned circuit 74
29 An active low-pass filter 78
30 An FDNR subcircuit. 79
31 A linear model of $\mu A741$ Opamp 79
32 The DDD for $det((\mathbf{T}^o)^{II})$ and $det((\mathbf{T}^o_{11})^{II})$ 81
33 The DDD for $det((\mathbf{T}^x)^{II})$ and $det((\mathbf{T}^x_{11})^{II})$ 83
34 The DDD for $det(\mathbf{T}_{11})$ and $det(\mathbf{T}_{16})$ 85
35 Miller-compensated two-stage Opamp 88
36 An active RC band-pass filter 89
37 The partitioned RC band-pass filter 89
38 The subcircuit in the band-pass filter 90
39 A three-level, two-way partition of $\mu A741$ 93
40 A three-level, two-way partitioned $\mu A741$ 94
41 Model of a circuit hierarchy 97
42 #DDD vertices and #product terms vs size of full matrices 99
43 Initial partition construction 101
44 An illustration of gain calculation in multi-way partitioning 107
45 Balance-relaxed multi-way partitioning 108
46 A two-way partitioned $\mu A741$ 111

47 A three-way partitioned $\mu A741$ 111

48 A four-way partitioned $\mu A741$ 112

49 A three-way partitioning of the band-pass filter 115

50 A five-way partition of band-pass filter by contour tableau method . . . 119

51 The matrix patterns causing term cancellation 127

52 Algorithm for symbolic canceling term elimination 129

53 A cancellation-free s-expanded DDD 130

54 A simplified two-stage CMOS Opamp 134

55 Voltage gains of two-stage Opamp in exact and simplified expressions by the proposed algorithm 135

56 Phases of two-stage Opamp in exact and simplified expressions by the proposed algorithm 136

57 Accuracy comparison in voltage gain for CMOS Cascode Opamp 137

58 Accuracy comparison in phase for CMOS Cascode Opamp 138

59 #terms distribution vs powers of s for Cascode 138

60 Accuracy comparison in voltage gain for $\mu A741$ 139

61 Accuracy comparison in phase for $\mu A741$ 140

62 #terms distribution vs powers of s for $\mu A741$ 140

63 Noise models for common IC devices 146

64 A network with a single noise source 148

65 A RC circuit example 149

66 A matrix determinant and its DDD 149

67 DDDs of representing noise transfer functions 150

68 State variable filter circuit 153

69 Noise model for operational amplifiers 153

70 Noise spectral densities by SPICE and DDD 155

CHAPTER I INTRODUCTION

1.1 Motivations

The rapidly growing very-large-scale-integration (VLSI) technology is moving towards the integration of complete systems containing billions of transistors on a single silicon chip. System-on-a-chip (SOC) design methodology will tremendously change the ways in which electrical systems are designed in the near future. A complete SOC system typically consists of digital and analog parts. Analog circuits, although a relative small portion of the entire system, are crucial building blocks used to interface the entire system with the outside physical world. Nevertheless, such a complicated mixed analog-digital system on a chip can't be designed without efficient and powerful computer-aided design (CAD) tools.

The automatic design of digital circuits is a mature field, and existing design automation tools based on them are efficient, powerful and widely used by designers. The design of analog circuits, however, is still mostly carried out manually. The lack of analog CAD tools results in lengthy design times and costly design errors. This explains the current large industrial demand for analog design automation tools.

Many research efforts have been carried out in this emerging field, and several methodologies for synthesis of analog modules have been proposed in the recent years [17, 31, 34, 45, 73]. One crucial task in the design automation of analog integrated circuits is to generate the required small-signal characteristics in an exact and approximate analytic form. These analytic expressions describing analog circuit behavior can be used to quickly evaluate the circuit performance or gain insight into

circuit behavior. The automatic generation of these small-signal characteristics or analytic models by symbolic simulators have been implemented in the programs like ADAM [18], ARIADNE [30], OPTIMAN [31] and OPASYM [45]. In this way, an open, flexible topology analog CAD system can be created, enabling designers to easily include new circuit topologies.

Symbolic analysis is a systematic approach to obtaining the knowledge of analog building blocks in an analytic form. It is an essential complement to numerical simulation. Its importance and increasing interest have been demonstrated by the success of modern symbolic analyzers such as ASAP [22], ISAAC [30], SCAPP [39], SYNAP [68] and RAINIER [99] for analog integrated circuits. The size of circuits that symbolic analysis can handle, however, is still much smaller than for numerical simulators. The root of the difficulty lies in the exponential growth of product terms in a symbolic network function with respect to the network size. This long-standing problem has been partially solved in modern symbolic analyzers [23, 24, 29, 39, 69, 70, 92, 99].

1.2 Objectives and Results of This Thesis

The main objective of this dissertation is to develop a new theory and methodology for efficient symbolic analysis of large integrated analog circuits. The key contribution of this research is the introduction and the exploration of a new graph representation, named Determinant Decision Diagrams (DDDs), for symbolic determinants and their use in symbolic circuit analysis. The major achievements accomplished in this dissertation are as follows:

- A new graph representation (DDDs) for symbolic determinant is proposed, which can naturally explore the sparsity of circuit matrices and the sharing of symbolic expressions in symbolic determinants. DDDs enable the exact symbolic analysis of analog circuits that are substantially larger than those previously reported.

- A vertex-ordering heuristic is developed, leading to DDD representations with the number of DDD vertices being orders-of-magnitude less than the number of product terms. The vertex ordering for DDD representations given by the heuristic is proved to be optimal for a class of circuit structures.
- A multi-rooted DDD structure is proposed to represent s-expanded polynomial expressions. Under the assumption that the maximum number of circuit parameters in an entry of a circuit matrix is bounded by a constant, we prove that both the time complexity of the construction algorithm and the size of the resulting s-expanded DDD are bounded by $m|DDD|$, where $|DDD|$ is the size of the original DDD, m is the highest power of s in the s-expanded expression.
- The DDD technology is also explored in hierarchical symbolic analysis. The problem of finding an optimal circuit partition and hierarchy is formulated; an efficient algorithm is developed for its solution. The resulting symbolic analyzer, for the first time, is capable of such analyzing analog integrated circuits with less structure regularities as operational amplifiers and outperforms the best partitioning-based symbolic analyzer SCAPP [39].
- A new symbolic approximation strategy based on efficient DDD-graph manipulations is developed. The new strategy provides a general framework for deriving interpretable small-signal characteristics not only for network transfer functions, but also for poles, zeros and sensitivities.
- Circuit-noise modeling by means of DDD-graph manipulations is investigated. A new circuit-noise modeling technique is developed for hierarchical system noise analysis.

All the results have been implemented in a computer program — SCAD3 (**S**ymbolic **C**ircuit **A**nalysis with **D**eterminant **D**ecision **D**iagrams). SCAD3 can be used as a stand-alone program or be integrated into other analog circuit synthesis

and optimization system. SCAD3 has been tested on a number of integrated analog circuits.

1.3 Organization of This Thesis

This dissertation is organized as follows: Chapter II offers the background of symbolic circuit analysis, including its applications, basic methods and the state-of-the-art in this field. Chapter III introduces the concept of determinant decision diagrams for the representations of symbolic determinants; it also describes an efficient vertex-ordering heuristic for DDDs. Chapter IV presents an extension of the DDD, named *s-expanded* DDD, for representing s-expanded symbolic expressions. S-expanded DDDs also pave the way for the new symbolic approximation and circuit-noise modeling technique. Chapter V presents a new hierarchical symbolic analysis method based on DDDs. The problem of finding the best circuit hierarchy for DDD-based hierarchical symbolic analysis is considered in Chapter VI. A new symbolic approximation strategy to generate interpretable symbolic expressions, based on DDD-graph manipulations, is described in Chapter VII. Chapter VIII is dedicated to circuit-noise analysis and noise model generations by means of DDDs. Chapter IX concludes this dissertation with a summary and a discussion of some future research topics.

CHAPTER II A REVIEW OF SYMBOLIC CIRCUIT ANALYSIS

Symbolic analysis is a technique for generating closed-form analytical expressions, in circuit parameters and frequency, of circuit characteristics. It is usually done in the frequency domain for linearized analog circuits [51].

The resulting analytical expression from symbolic analysis provides a means for understanding the qualitative influence of each circuit parameter over the circuit behavior. As an analytical model, a symbolic expression captures the behavior in large regions of the design space. From this perspective, symbolic analyzers are in contrast to numerical simulators, like SPICE [61], which show the behavior at an isolated point of this space. The goal of symbolic analysis, however, is not to replace numerical analysis, instead; it should be viewed as a complement to numerical analysis by providing other design tasks required by analog circuit design.

This chapter presents the general concepts and the application scenarios related to symbolic analysis. It then provides a brief review of the basic symbolic analysis methods developed over the past thirty years and the current status in this field.

2.1 Applications of Symbolic Analysis

The major applications of symbolic circuit analysis, in the design of analog circuits, can be basically grouped into two major areas [31]: (1) those requiring repetitive evaluations of the analytical models describing the behavior of a circuit; (2) those associated with the generation of the knowledge about the behavior of a circuit.

2.1.1 Iterative Formula Evaluation

Evaluation of expressions can yield a significant speedup over numerical circuit simulation. The symbolic expression describing circuit behavior can be in an exact or approximate analytical form. The approximate symbolic expression is often referred to as an *analytical* model used in the circuit device sizing and the circuit topology selection. The manual derivation of the analytic models is usually a time-consuming and error-prone process that has to be carried out for each circuit schematic. Symbolic analysis can fully automate the derivation of these models.

In the circuit topology selection, the influence of topology changes (adding or removing a component) can immediately be seen from the new resulting expressions. In this way, an open, non-fixed-topology analog synthesis and optimization can be created. The efficiency of this technique has been demonstrated in using the symbolic analyzer SCYMBAL [46, 47] for analysis of switch-capacitor circuits.

In the circuit device sizing (parameter optimization), circuit behavior are approximated by the symbolic analytical formulas coming from symbolic analyzers, and resulting formulas are evaluated repeatedly for multiple values of the circuits' parameters and/or frequencies. This methods have been adopted in analog optimization systems like OPTIMAN [31] and OPASYS [45].

Exact symbolic expressions can be utilized in such applications as statistical analysis (for instance, Monte Carlo simulation, where the same characteristics have to be evaluated many times in order to assess the influence of statistical device mismatches and process tolerance on the circuit performance), large-signal sensitivity analysis, yield estimation and design centering.

2.1.2 Insight into Circuit Behaviors

A numerical simulator can accurately verify circuit behavior, but it is specific for a particular set of parameter values. A symbolic simulator, on the other hand,

returns the analytic expressions that remain valid even when the numerical parameter value changes (as long as the circuit topology remains the same). Hence symbolic analysis gives a different perspective on a circuit than that provided by numerical simulators, which is most appropriate for practicing designers in order to obtain an intuitive insight into the behavior of a circuit. It is especially true if the generated expressions are simplified to retain the dominant contributions only.

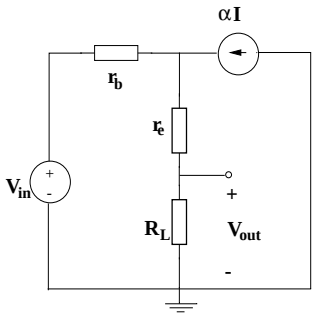


Figure 1: A small signal model for an emitter follower

For example, consider the simplified small signal circuit of an emitter-follower stage shown in Figure 1. The voltage gain of the circuit is found to be

$$\frac{V_{out}}{V_{in}} = \frac{R_L}{(1-\alpha)r_b + r_e + R_L}.$$

From this symbolic expression, it is immediately clear that the voltage gain is positive, less than 1, and very close to 1 (provided that $R_L \gg (1-\alpha)r_e + r_e$). Without the symbolic expression, these conclusion could only be reached after analyses of many numerical cases, and even then some degree of uncertainty still exists.

A symbolic analyzer is not only useful for novice designers, it is also a valuable tool for experienced designers to check and confirm their own intuitive knowledge and to obtain the analytic expressions for second-ordered characteristics, such as the harmonic distortion that is very difficult to be calculated by hand, especially at higher

frequencies.

2.1.3 Other Applications

Symbolic analysis is also employed in analog fault diagnosis and test generation in [94, 96], distortion analysis of weakly nonlinear circuits [91], tolerance analysis [71] and time-domain analysis [37].

2.2 Review of Symbolic Analysis Techniques

Research on symbolic analysis can be dated back to last century. Developments in this field gained real momentum in the 1950's when electric computers were introduced and used in circuit analysis. Methods developed from the 1950's to the 1980's can be basically categorized as: (1) *Tree Enumeration methods*, (2) *Signal Flow Graph methods*, (3) *Parameter Extraction methods*, (4) *Numerical Interpolation methods* and (5) *Matrix-Determinant methods*. The details of these method can be found in [31, 51]. In the late 1980's, symbolic analysis gains renewed interests as industrial demands for analog design automation increased. Various methods are proposed to solve the long-standing circuit-size problem. The strategies used in modern symbolic analyzers in general come in two categories: those based on hierarchical decompositions [39, 70] and those based on approximations [23, 24, 29, 69, 92, 99].

This section will briefly review these methods. We first begin with the formulation of the problem for general symbolic analysis.

2.2.1 Problem Formulation for Symbolic Analysis

Consider a lumped, linear(ized) time-invariant analog circuit in frequency domain. Its circuit equation can be formulated, for example, by the modified nodal analysis (MNA) approach in the following general form [86]:

$$\mathbf{Tx} = \mathbf{w}. \tag{2.1}$$

The *circuit unknown vector* $\mathbf{x}$ may be composed of n node voltages and branch currents, and the *circuit matrix* $\mathbf{T}$ is a $n \times n$ sparse symbolic matrix, w is a vector of external sources. Symbolic analysis of analog circuits can be stated as the problem of solving the symbolic equation (2.1), i.e., to find a symbolic expression of any circuit unknowns in terms of symbolic parameters in $\mathbf{T}$ and symbolic excitations expressed by $\mathbf{w}$. According to Cramer's rule, the kth component x_k of the unknown vector $\mathbf{x}$ is obtained as follows:

$$x_k = \frac{\sum_{i=1}^{n} \; w_i \; (-1)^{i+k} \; det(\mathbf{T}_{t_{i,k}})}{det(\mathbf{T})}. \tag{2.2}$$

Most symbolic simulators are targeted at finding various network functions, each being defined as the ratio of an output unknown from $\mathbf{x}$ to an input from $\mathbf{w}$. These are special cases of (2.2) or the ratios of the two expressions in the form of (2.2). Generally, the transfer function for a continuous-time, linear(ized) circuit in frequency domain can be obtained as a rational function in the complex frequency variable s:

$$H(s) = \frac{\sum f_i(p_1, p_2, \cdots, p_m) s^i}{\sum g_j(p_1, p_2, \cdots, p_m) s^j}, \tag{2.3}$$

where $f_i(p_1, p_2, \cdots, p_m)$ and $g_j(p_1, p_2, \cdots, p_m)$ are symbolic polynomial functions in circuit parameters $p_j, j = 1, ..., m$. These polynomials in turn can be in a nested form or an expanded sum-of-product form. If the polynomial coefficients, $f_i(\cdots)$ and $g_j(\cdots)$, only contain the symbols, it is named *fully* or *exact* symbolic analysis. If only some circuit parameters are represented as symbols, it is named *partial* or *mixed* symbolic analysis. In the extreme case, the transfer function, $H(s)$, may remains only one symbol – the complex frequency s.

2.2.2 Tree Enumeration Method

Tree enumeration method is one of the oldest, fully symbolic analysis methods proposed in the last century when Kirchoff and Maxwell first stated their results on RLC networks, which include only resistance, inductance and capacitance, in

their classical works. In this method, a passive RLC network is represented as an undirected weighted graph $G(N, E)$, where N is the set of nodes in the network and E is the set of edges with each edge representing a circuit element between the nodes the edge connects. The weight of each edge is the admittance of the corresponding circuit element. The basic idea in this method is that the determinant of the node admittance of an RLC network equals the sum of all *tree-admittance products* of the graph $G(N, E)$. A *tree-admittance product* is the product of all the edge weights in a spanning tree of the $G(N, E)$. So network transfer function calculation amounts to enumerating spanning trees of the undirected weighted graph. Some transfer functions, such as trans-impedance or voltage gain, may require finding a *2-tree*. *A 2-tree* is a pair of node-disjoint subgraphs that individually are connected, loopless, and together include all nodes. The attractive feature of this method is that all the product terms generated are irreducible or term-cancellation free.[1]

For an active RLC-g_m network (with voltage-controlled current source), however, the original tree enumeration method can't be directly applied. Two extension methods were proposed to solve this problem. The first is *2-graph* approach [55] and the second is *directed-tree* approach [12]. In the *2-graph approach*, each admittance of RLC circuit element is treated as a voltage-controlled current source (VCCS) with controlling and controlled nodes coinciding. In this way, VCCS type elements can be naturally accommodated in this method. To this end, the method constructs two graphs for a circuit network function. One graph, named the voltage graph G_V, only contains controlling voltage sources; the other, named the current graph G_I, only contains controlled current sources. The product terms in the determinant of the network function correspond one-to-one to the common spanning trees of G_V

[1]Term cancellation is due to the fact that two symbolic terms with the same symbol combination but opposite signs cancel each other.

and G_I. Nevertheless, the sign of each product terms has to be explicitly calculated in this method. In the *directed tree approach*, a directed graph G_d is constructed from the indefinite admittance matrix (IAM) – Y_{ind} of a circuit. The tree-admittance products are found by enumerating the directed spanning trees of the directed graph with any node treated as the root. The directed graph method avoids the complicated sign calculation for each product term, but the resulting product terms are not cancellation-free.

2.2.3 Signal Flow Graph Method

A signal-flow diagram [56] is a weighted, directed graph representing a set of linear equations, which can be expressed in the following general form:

$$X_{n\times 1} = A_{n\times n}X_{n\times 1} + B_{n\times m}U_{m\times 1}, \tag{2.4}$$

where X is the vector formed by the n dependent node variables, U is the vector formed by the m independent node (source node) variables, and A and B are transmittance matrices constructed from the coefficients of the linear equations.

Given a linear equation set in the form of (2.4), a signal flow graph can be constructed by the following rules: (1) Node weights represent variables (known or unknown). (2) Branch weights (transmittance) represent the coefficients in the relationships among node variables. (3) Each dependent node variable equals the sum of the products of the incoming branch weight and the node variable from which the branch originates. Any transfer function from an input variable x_i to an output variable x_j can be computed by Mason's rule [56] as follows:

$$\frac{x_i}{x_s} = \frac{\sum_{k=1} P_k \cdot \Delta_k}{\Delta}, \tag{2.5}$$

where

- Δ = 1 - (sum of all loop weights) + (sum of all second-order loop weights) - (sum of all third-order loop weights) + $\cdots$,

- P_k = weight of the kth path from the source nodes x_s to the dependent node x_i,
- Δ_k = sum of those terms in Δ without any constituent loops touching the path P_k.

The nth-order loop is defined as the loop set formed by n non-touching loops. The weight of the nth order loop is the sum (over all possible loop sets) of the products of the branch weights in these n non-touching loops.

Signal-flow graph method can be applied to various types (passive or active) of circuit networks, as the signal-flow graph is directly derived from linear equations and is independent of circuit formulations. This method, however, also suffers from the term-cancellation problem.

2.2.4 Numerical Interpolation Method

Numerical interpolation method is one of the partial symbolic analysis methods in which the network functions contain complex frequency variable s as the only symbol. For linear(ized) time-invariant networks in frequency domain, the network function can always be represented as a ratio of two polynomials in s with numerical coefficients. Consider a polynomial in s of degree n:

$$P(s) = \sum_{i=0}^{n} a_i s^i = a_0 + a_1 s + \cdots + a_n s^n, \tag{2.6}$$

with numerical coefficients $a_i, i = 1, ..., n$. Since one can evaluate $P(s)$, which is the determinant, by any standard numerical method at any desired value of s, then by obtaining the values of $(n+1)$ distinct points $s = s_0, s_1, ..., s_n$, one can determine the coefficients $a_0, a_1, ..., a_n$ by solving the following algebraic equations

$$\begin{bmatrix} 1 & s_0 & s_0^2 & ... & s_0^n \\ 1 & s_1 & s_1^2 & ... & s_1^n \\ . & . & . & ... & . \\ 1 & s_n & s_n^2 & ... & s_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ . \\ a_n \end{bmatrix} = \begin{bmatrix} P(s_0) \\ P(s_1) \\ . \\ P(s_n) \end{bmatrix}. \tag{2.7}$$

For RLC-g_m circuit, a $n \times n$ node admittance matrix Y_n describing the network of interest can be written in the following form:

$$Y_n = sC + G + L/s, \tag{2.8}$$

where C, G, L are $n \times n$ real matrices. This makes the numerator and the denominator of a network function to be rational functions instead of polynomials of s. One solution to this problem, suggested in [72], is as follows:

$$Y_n = sC + G + L/s = (s^2C + Gs + L]/s = Y^*/s. \tag{2.9}$$

Analysis is then carried out on Y^*, and the results are related back to Y_n. To avoid the numerical problem (such as overflow, stability, accuracy), the best interpolation points, $s_i, i = 1, ..., n+1$, are known to be those uniformly distributed on a unit circle on the complex plane [86]. Interpolation methods based the state variable formulation of networks were also reported and summarized in [51]. For very large network (n is large), the linear equations (2.7) are numerically ill-conditioned.

The numerical interpolation method can handle much larger circuits than fully symbolic methods at the cost of keeping limited symbolic information.

2.2.5 Parameter Extraction Method

Parameter extraction method is another partial symbolic analysis method in which only a small number of circuit parameters are kept as symbols in addition to the complex frequency variable. This is typically achieved by extracting symbolic element parameters one at a time from the determinants (numerator or denominator), until the resultant determinants have at most s as the only variable and then are calculated by a numerical method. Specifically, if the indefinite admittance matrix Y_{ind} is used for circuit equation formulation, each admittance variable α will appear exactly in four positions. Let α be a variable that appears in a $n \times n$ Y_{ind} in four

rectangular positions : α in position(i,k) and (j,m) and $-\alpha$ in position (i,m) and (j,k). According to the theorem in [3], the evaluation of any cofactor of this IAM containing the symbolic parameter α is now reduced to the evaluation of the cofactors of two IAMs not containing α. That is

$$det(Y_{ind}) = det(Y_{ind}|\alpha = 0) + (-1)^{j+m}\ \alpha\ det(Y_\alpha), \tag{2.10}$$

where Y_α is an $(n-1)\times(n-1)$ IAM not containing α, obtained by modifying Y_{ind} as follows:

1. add row j to row i,
2. add column m to column k,
3. delete row j and column m.

The theorem can be recursively repeated to extract other symbols remained in $det(Y_{ind}|\alpha = 0)$ and Y_α. If a matrix is obtained with only numerical entries and the s variable, its determinant can be evaluated by any standard numerical method.

2.2.6 Matrix-Determinant Method

A circuit transfer function can be calculated by directly applying Cramer's rule to the linear equations. In order to obtain the symbolic transfer functions, determinants or cofactors obtained from Cramer's rule have to be expanded. One way to expand a determinant is by its definition:

$$det(\mathbf{A}) = \begin{vmatrix} a_{1,1} & a_{1,2} & ... & a_{1,n} \\ a_{2,1} & a_{2,2} & ... & a_{2,n} \\ ... & ... & ... & ... \\ a_{n,1} & a_{n,2} & ... & a_{n,n} \end{vmatrix} = \sum_{j_1 \neq j_2 \neq ... \neq j_n} (-1)^p \cdot a_{1,j_1} \cdot a_{2,j_2} ..... a_{n,j_n}, \tag{2.11}$$

where $(j_1, j_2, ..., j_n)$ is a permutation of column indices of $\mathbf{A}$, and p is the number of permutations needed to make the sequence $(j_1, j_2, ..., j_n)$ monotonically increasing.

By the definition, a $n \times n$ full matrix will have $n!$ product terms in its determinant. Such an exponential space complexity is the crux of the problem facing all the

symbolic analysis methods. However, by using different expansion methods, one can save some CPU time when generating these product terms. For instance, a determinant can also be expanded by recursively applying Laplace's expansion with respect to a certain row or column (say row k) each time:

$$det(\mathbf{A}) = \sum_{j=1}^{n} (-1)^{k+j} a_{k,j} \cdot det(\mathbf{A}_{a_{k,j}}), \tag{2.12}$$

where $\mathbf{A}_{a_{k,j}}$ is the submatrix obtained by deleting the kth row and jth column.

For practical circuits, the circuit matrices usually are sparse and there exist many common subexpressions among expanded product terms. Symbolic analyzer ISAAC [30] exploits such sparsity by storing minors and dynamical selecting row/column during the recursive Laplace expansion. The stored minor can be retrieved when it is seen in the late course of the expansion procedure. The row/column dynamically selected for expanding is the one with the smallest number of nonzero elements. Such an expansion scheme can greatly enlarge the chance of recurrences of minors in the whole expansion process. Consequently, the combined scheme results in the smallest number of intermediate terms among all expansion methods and is the most efficient heuristic for matrix expansion [30].

The matrix-determinant method is also not cancellation-free. For exact symbolic analysis, however, it is shown in [30] that topological methods offer no advantage over the matrix-determinant method (and vice versa).

2.2.7 The State of the Art in Symbolic Analysis

As mentioned before, the primary difficulty in symbolic analysis is the exponential growth of product terms with the number of nodes and elements in a circuit. As a result, exact symbolic analysis is limited to only small circuits with number of nodes no more than 10 [30]. Two techniques were proposed in modern symbolic analyzers to overcome the long-standing circuit-size problem [31]. There are *hierarchical*

decompositions and *approximations*.

2.2.7.1 Hierarchical Decomposition

Hierarchical decompositions generate symbolic expressions in a nested form [39, 70]. There are two methods known as topological analysis [70] and network formulation [39]. Both methods are based on the *sequence-of-expressions* concept to obtain transfer functions.

- In the *topological analysis method* [70], circuit topology is represented as a directed graph; circuit parameters are represented as the weights of the edges in the graph. Hierarchical decompositions are carried on the directed graph. Analysis in subcircuits amounts to finding node-disjoint directed paths and node-disjointed directed loops. Results obtained in subcircuit analysis are combined upward until the root circuit is reached.
- In the *network formulation method* [39], hierarchical decompositions are performed directly on the circuit equations. The decomposition procedure is characterized by eliminating variables one at a time (called Reduced Modified Nodal Analysis) in each subcircuit analysis. The results of lower-level circuits are then combined according to some rules to form upper-level circuit equation sets. The process continues until a root circuit is reached, where a minimum set of equations, which only contain the external variables, is obtained and the transfer function is readily computed by solving the equation set. All the intermediate results are expressed as *sequence of expressions*.

The success of the hierarchical decomposition methods, however, crucially depends on how a circuit is partitioned. The existing methods can only be applied to analog circuits with highly structural regularity and loose coupling among circuit elements.

2.2.7.2 Approximation Method

Approximations discard those insignificant terms based on the relative numerical magnitude of symbolic parameters and the frequency defined at some nominal design points or over some ranges. It can be performed before [40, 97], during [25, 97] and after [24, 30, 69, 93] the generation of symbolic terms.

- *Approximations after generation* methods are the most reliable methods, but they require the expansion of product terms before approximations, and thus are limited to small analog circuits. Some improvements based on nested expressions have been proposed [23, 69]. Nested symbolic expressions, used in [69], are multiplied out in such a way that the product terms can be generated in a nonincreasing order without generating all the product terms. The method in [23] calculates the numerical contributions of each leaf-node expression in the nested expression with respect to both numerator and denominator. The resulting contributions are then used to guide the subsequent pruning of leaf nodes. Although these methods can handle larger analog circuits than the methods using non-nested form, they generally suffer symbolic term-cancellation and align-term problems [2].
- *Approximations during generation* methods extract only significant product terms [25, 90, 91, 97, 100]. They are based on the fact that product terms can be generated in a nonincreasing order by finding the smallest weight spanning trees [25, 97], or by using matroid intersection algorithm [91, 100], or by the inherently cancellation-free algorithm [90]. They are very fast, but work only for transfer functions. Other small-signal characteristics, such as sensitivities and symbolic poles and zeros, can't be extracted in general.

[2] Canceling terms left in the final expressions.

- *Approximations before generation* method removes circuit elements whose contributions to the transfer function containing them are negligible before product generation processes, and therefore it reduces the complexity of the final expanded expressions. Such an approximation can be performed on graphs [97, 99], or directly on the circuit equations [40, 41].

A good summary of the various approximation schemes in modern symbolic simulators can be found in [26, 31]. If an analog circuit is too large, the simplified symbolic expressions, which satisfy certain error margins, may be still too lengthy to be interpretable. This is the limitation of approximation methods. Even worse, if more accurate expressions are needed, the time complexity of the existing approximation methods become exponential.

2.2.8 Other Research Topics

2.2.8.1 Symbolic Pole and Zero Extraction

This method is to derive the symbolic expressions for the dominant poles and zeros in the circuit transfer functions. Theoretically, no general analytical forms exist for exact poles and zeros for an arbitrary network function. But under the assumption that all the poles or zeros are far away from each other, pole-splitting method can be used to obtain approximate poles or zeros [27, 32]. Consider how to find the root for a polynomial $f(s)$, which may be the numerator or the denominator of the transfer function of interest:

$$f(s) = a_n s^n + ... + a_1 s + a_0 = 0. \tag{2.13}$$

At low frequencies, this equation may be approximated with

$$a_1 s + a_0 = 0. \tag{2.14}$$

If the first root is far away from the rest, that is:

$$|p_1| << |p_2| \leq |p_3| \leq ... \leq |p_n|. \tag{2.15}$$

Then the first root, p_1, can be given by

$$p_1 = -\frac{a_0}{a_1}. \tag{2.16}$$

If all the roots are far away from each other, it is easy to obtain

$$p_i = -\frac{a_{i-1}}{a_i}. \tag{2.17}$$

Symbolic pole and zero extraction methods were reported in symbolic analyzers: ASAP [22] and AnalogSifter [41, 42]. The method in [41] can isolate groups of poles and zeros from the other poles and zeros under the assumption that the others are much larger or smaller in their complex frequency magnitude.

2.2.8.2 Symbolic Circuit-Noise Analysis

Symbolic circuit noise analysis is to compute the noise spectral densities at certain outputs of an analog circuit block. Noise in analog integrated circuits comes from the inherent noise sources of circuit elements. Those noise sources are caused by different physical phenomena, and hence are statistically uncorrelated. So the contribution from each noise source has to be computed separately, and the entire noise analysis essentially amounts to computing a number of transfer functions from different input ports to the same output port. A symbolic analysis technique was proposed in [30] in which all terms in the required symbolic functions are first generated and then approximated. As the consequence of this, the sizes of the circuits handled are still very small.

2.2.8.3 Symbolic Sensitivity Analysis

Symbolic sensitivity analysis gives the sensitivities (derivatives) of a network function with respect to any circuit parameters. A method based on the concept of sequence of expressions [39] was proposed in [52]. In this method, a separate sequence of expressions has to be generated for each sensitivity with respect to a

circuit element. Therefore, a huge number of sequences will be generated if a large amount of sensitivity information is required.

2.2.8.4 Symbolic Analysis of Weakly Nonlinear Circuits

In this method, no hard nonlinearity is assumed; thus all the nonlinear elements in the circuit can be well approximated by their third-order power series expansion [89, 91]. That is, all the circuit element can be represented by

$$k_0 + k_1 s + k_2 s^2, \tag{2.18}$$

where k_0, k_1 and k_2 are all constants represented by symbols. k_0 corresponds to the linear behavior of the element, and k_1 and k_0 to the weakly nonlinear behavior. All the higher-order coefficients are truncated with their contributions calculated as corrections in the lower-order response. The generated symbolic expressions contain the same small-signal parameters as the ones used in linear analysis, but also they contain parameters, $k_i, i = 1, 2, 3$, that describe the nonlinear behavior. Symbolic analysis of harmonic distortion in weakly nonlinear circuits is also reported by the same author in [91].

2.3 Summary

This chapter provided an overview of symbolic analysis applications and most important symbolic analysis methods, pointed out the pluses and minuses associated with each method. Although symbolic analysis has aroused much interest in the past few years, many research results are still in their preliminary stages. The long-standing circuit-size problem is only partially solved by the various approaches proposed thus far. In this dissertation, we focus on the fundamental issue facing all the symbolic analysis methods and propose a new theory and methodology to address this problem. Our innovative graph-based approach will be covered in the following chapters.

CHAPTER III EXACT SYMBOLIC ANALYSIS OF ANALOG CIRCUITS VIA DETERMINANT DECISION DIAGRAMS

3.1 Introduction

One of the main difficulties with exact symbolic analysis is the exponential growth of product terms with the size of circuits.

The starting point of this research is the introduction of an efficient graph representation for a symbolic determinant. Our approach is based on the two observations on symbolic analysis of a large analog circuit: (a) the circuit matrix is sparse and (b) a symbolic expression often shares many subexpressions. Under the assumption that all the matrix elements are distinct, each product term can be viewed as a subset of all the symbolic parameters. We adopt a data structure — Zero-suppressed Binary Decision Diagrams (ZBDDs) — introduced originally for representing sparse subset systems [59]. A ZBDD is a variant of a Binary Decision Diagram (BDD) introduced by Akers [1] and popularized by Bryant [5]. BDDs have brought a great success to the formal verification and testing for combinational and sequential digital circuits [6, 43].

Our work has been inspired by such success, resulting in a new graph representation of a symbolic determinant, called Determinant Decision Diagram (DDD). This representation has several advantages over both the expanded and arbitrarily nested forms of a symbolic expression. First, similar to the nested form, our representation is compact for a large class of analog circuits. A ladder-structured network can be represented by a diagram where the number of vertices in the diagram (called its *size*) is equal to the number of symbolic parameters. As indicated by our experiments, the typical size of DDD is dramatically smaller than that of product terms. For instance,

5.71×10^{20} terms can be represented by a diagram with 398 vertices. Second, similar to the expanded form, our representation is canonical, i.e., every determinant has a *unique* representation, and is amenable to symbolic manipulations. The canonical property facilitates symbolic analysis and may provide a potential way to formally verify analog circuits. Finally, evaluations and manipulations of symbolic determinants (such as sensitivity calculations) have time complexities proportional to the DDD sizes.

This chapter is organized as follows: Section 3.2 introduces some notations for the rest of the dissertation. Section 3.3 presents the concept of determinant decision diagrams as an application of ZBDDs to represent symbolic determinants. Section 3.4 describes an effective heuristic for ordering DDD vertices so that the resulting DDD has a smallest or near-smallest size. DDD-based algorithms for symbolic analysis are described in Section 3.5. Some experimental results are presented in Section 3.6. This chapter is summarized in Section 3.7.

3.2 Preliminaries

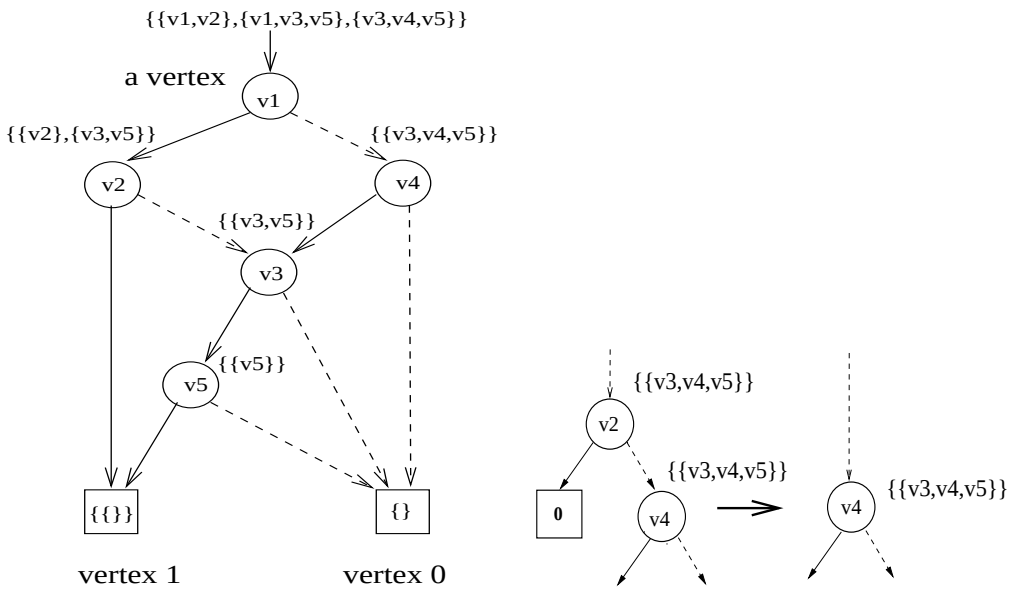
In this section, we introduce some basic mathematical notations and concepts that will be used throughout the dissertation. Since these basics come from several different research areas, an attempt has been made to choose a self-consistent set of notations.

3.2.1 Subset Systems and Zero-suppressed Binary Decision Diagrams

Let V be a *set* of elements. The number of elements in V is called the *cardinality* of V, denoted by $|V|$. The set of all *subsets* of V is called the *power set* of V, denoted by 2^V. A subset X of the power set, written as $X \subseteq 2^V$, is called a *subset system* of V.

A subset system X of V can be decomposed with respect to an element v

in V into two unique subset systems, X_v and $X_{\overline{v}}$, where X_v is the set of subsets of V belonging to X that contain v, from which v has been removed, and $X_{\overline{v}}$ is the set of subsets of V belonging to X that do not contain v. For instance, let $X = \{\{v_1, v_2\}, \{v_1, v_3, v_5\}, \{v_3, v_4, v_5\}\}$. Then we have $X_{v_1} = \{\{v_2\}, \{v_3, v_5\}\}$, and $X_{\overline{v}_1} = \{\{v_3, v_4, v_5\}\}$. This decomposition can be represented graphically by a decision *vertex*. It is labeled by v_1, and represents the subset system X. As illustrated in Figure 2(a), the vertex has two outgoing edges: one points to X_{v_1} (called *1-edge*), and the other to $X_{\overline{v}_1}$ (called *0-edge*). We say that the edges are *originated* from the vertex. If X is recursively decomposed with respect to all the elements of V, one obtains a binary decision tree whose leaves are $\{\{\}\}$ and $\{\}$, respectively. For convenience, we denote leaf $\{\{\}\}$ by the 1-terminal, and $\{\}$ by the 0-terminal.



(a) A ZBDD example

(b) An illustration of the zero-suppression rule

Figure 2: Zero-suppressed BDD

When a subset system X is decomposed with respect to an element that does not appear in X, then its 1-edge points to the 0-terminal. This decomposition is illustrated in Figure 2(b) for decomposing $X = \{\{v_3, v_4, v_5\}\}$ with respect to v_2. To make the diagram compact, Minato suggested the following *zero-suppression* rule for representing sets of *sparse* subsets: eliminate all the vertices whose 1-edges point to the 0-terminal vertex and use the subgraphs of the 0-edges, as shown in Figure 2(b) [59]. A Zero-suppressed Binary Decision Diagram (ZBDD) is such a zero-suppressed graph with the following two rules due to Bryant [5]: *ordered*—all elements of V, if one appears, will appear in a fixed order in all the paths of the graph; and *shared*—all equivalent subgraphs are shared. ZBDD is a canonical representation of a subset system, i.e., every subset system has a unique ZBDD representation under a given vertex ordering. For example, Figure 2(a) is a unique ZBDD representation for the subset system $\{\{v_1, v_2\}, \{v_1, v_3, v_5\}, \{v_3, v_4, v_5\}\}$ with respect to ordering v_1, v_2, v_4, v_3 and v_5. For convenience, every nonterminal vertex is indexed by an integer number greater than those of its descendant vertices [59]. How all the nonterminal vertices are indexed is called *vertex ordering*.

A path from a nonterminal vertex to the 1-terminal is called *1-path*. It defines a subset of V. The subset consists of all the elements of V from which the 1-edges in the 1-path originate. The number of vertices in a ZBDD is called its *size*.

3.2.2 Matrix, Determinant and Cofactors

Let $e = \{1, ..., n\}$ be a set of integers. Let A denote a set of m elements, called *symbolic parameters* or simply *symbols*, $\{a_1, ..., a_m\}$, where $1 \leq m \leq n^2$ and each symbol is labeled by a unique pair (r, c), where $r \in e$ and $c \in e$. Often, we write A as an $n \times n$ (square) *matrix*, denoted by $\mathbf{A}$, and use $a_{r,c}$ to denote the element of matrix $\mathbf{A}$ at row r and column c. We sometimes use $r(a)$ and $c(a)$ to denote, respectively,

the row and column indices of element a.

$$\mathbf{A} = \begin{pmatrix} a_{1,1} & a_{1,2} & ... & a_{1,n} \\ a_{2,1} & a_{2,2} & ... & a_{2,n} \\ ... & ... & ... & ... \\ a_{n,1} & a_{n,2} & ... & a_{n,n} \end{pmatrix}.$$

If $m = n^2$ the matrix is said to be *full*. If $m << n^2$ the matrix is said to be *sparse*. The *determinant* of $\mathbf{A}$, denoted by $det(\mathbf{A})$, is defined by

$$\begin{vmatrix} a_{1,1} & a_{1,2} & ... & a_{1,n} \\ a_{2,1} & a_{2,2} & ... & a_{2,n} \\ ... & ... & ... & ... \\ a_{n,1} & a_{n,2} & ... & a_{n,n} \end{vmatrix} = \sum_{j_1 \neq j_2 \neq ... \neq j_n} (-1)^p \cdot a_{1,j_1} \cdot a_{2,j_2} ..... a_{n,j_n}. \tag{3.1}$$

Here $(j_1, j_2, ..., j_n)$ is a permutation of e, and p is the number of permutations needed to make the sequence $(j_1, j_2, ..., j_n)$ monotonically increasing. The right-hand side of (3.1) is a symbolic expression of $det(\mathbf{A})$ in the *expanded* form, more precisely, the *sum-of-product* form, where each *term* is an algebraic product of n symbolic parameters. We note that each symbol can be assigned a real or complex value for analog circuit simulation.

Let p, $p \subseteq e$, and q, $q \subseteq e$ such that $|p| = |q|$. The square matrix obtained from matrix $\mathbf{A}$ by deleting those rows not in p and columns not in q forms a *submatrix* of $\mathbf{A}$, and is represented by $\mathbf{A}(p, q)$. It has the dimension $|p|$ by $|q|$.

Let $a_{r,c}$ be the element of $\mathbf{A}$ at row r and column c. Let $\mathbf{A}_{a_{r,c}}$ be the $(n-1) \times (n-1)$-matrix obtained from matrix $\mathbf{A}$ by deleting row r and column c, and let $\mathbf{A}_{\overline{a}_{r,c}}$ be the $n \times n$-matrix obtained from $\mathbf{A}$ by setting $a_{r,c} = 0$. Then, the determinant of matrix $\mathbf{A}$ can be *expanded* as below in a way similar to Shannon expansion for Boolean functions:

$$det(\mathbf{A}) = a_{r,c}(-1)^{r+c} det(\mathbf{A}_{a_{r,c}}) + det(\mathbf{A}_{\overline{a}_{r,c}}), \tag{3.2}$$

where $(-1)^{r+c}det(\mathbf{A}_{a_{r,c}})$ is referred to as the *cofactor* of $det(\mathbf{A})$ with respect to $a_{r,c}$, and $det(\mathbf{A}_{\overline{a}_{r,c}})$ as the *remainder* of $det(\mathbf{A})$ with respect to $a_{r,c}$. The determinant $det(\mathbf{A}_{a_{r,c}})$ is called the *minor* of $det(\mathbf{A})$ with respect to $a_{r,c}$. We note that the following two special cases of the expansion above are well known as *Laplace expansions* along row r and column c, respectively:

$$det(\mathbf{A}) = \sum_{r=1}^{n} a_{r,c}(-1)^{r+c} det(\mathbf{A}_{a_{r,c}}), \tag{3.3}$$

$$det(\mathbf{A}) = \sum_{c=1}^{n} a_{r,c}(-1)^{r+c} det(\mathbf{A}_{a_{r,c}}). \tag{3.4}$$

According to the definition of symbolic analysis problem introduced in Chapter II, most symbolic simulators are targeted at finding various network functions, each being defined as the ratio of an output unknown from $\mathbf{x}$ to an input from $\mathbf{w}$. These are special cases of (2.2) or the ratios of the two expressions in the form of (2.2).

Note that $(-1)^{i+k}det(\mathbf{T}_{t_{i,k}})$ in (2.2) is the cofactor of $det(\mathbf{T})$ with respect to element $t_{i,k}$ of matrix $\mathbf{T}$ at row i and column k. Therefore, the central issue in determinant-based symbolic analysis is how to find symbolic expressions of $det(\mathbf{T})$ and the cofactors of $det(\mathbf{T})$. The rest of the chapter focuses on how to represent a symbolic determinant (Sections 3.3 and 3.4), and how to compute, manipulate, and evaluate a symbolic determinant (Section 3.5). In our approach, we assume that all the entries in $\mathbf{T}$ are distinct.

3.3 ZBDD Representation of Symbolic Matrix Determinant

In this section, we apply the notation of Zero-suppressed Binary Decision Diagram (ZBDD) to represent a symbolic matrix determinant. This leads to a new interpreted graph called *determinant decision diagram* (DDD). DDD is a canonical representation for matrix determinants, similar to BDD for representing *binary functions* and ZBDD for representing *subset systems*.

A key observation is that the circuit matrix is sparse and that many times, a symbolic expression may share many subexpressions. For example, consider the following determinant

$$det(\mathbf{M}) = \begin{vmatrix} a & b & 0 & 0 \\ c & d & e & 0 \\ 0 & f & g & h \\ 0 & 0 & i & j \end{vmatrix} = adgj - adhi - aefj - bcgj + cbih. \tag{3.5}$$

We note that subterms ad, gj, and hi appear in several product terms, and each product term involves a subset (four) out of ten symbolic parameters. Therefore, we view each symbolic product term as a subset, and use a ZBDD to represent the subset system composed of all the subsets each corresponding to a product term. Figure 3 illustrates the corresponding ZBDD representing all the subsets involved in $det(\mathbf{M})$ under ordering $a > c > b > d > f > e > g > i > h > j$. It can be seen that subterms ad, gj, and ih have been shared in the ZBDD representation.

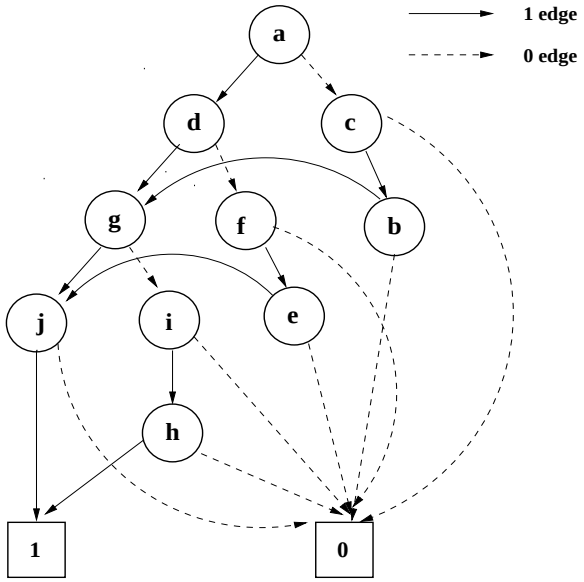


Figure 3: A ZBDD representing $\{adgi, adhi, afej, cbgj, cbih\}$ under ordering $a > c > b > d > f > e > g > i > h > j$

Following directly from the properties of ZBDDs, we have the following observations. First, given a fixed order of symbolic parameters, all the subsets in a symbolic determinant can be represented uniquely by a ZBDD. Second, every 1-path in the ZBDD corresponds to a product term, and the number of 1-edges in any 1-path is n. The total number of 1-paths is equal to the number of product terms in a symbolic determinant.

We can view the resulting ZBDD as a graphical representation of the recursive application of the determinant expansion formula (3.2) with the expansion order $a, c, b, d,\ f, e, g,\ i, h, j$. Each vertex is labeled with the matrix entry with respect to which the determinant is expanded, and it represents all the subsets contained in the corresponding submatrix determinant. The 1-edge points to the vertex representing all the subsets contained in the cofactor of the current expansion, and 0-edge points to the vertex representing all the subsets contained in the remainder.

To embed the signs of the product terms of a symbolic determinant into its corresponding ZBDD, we associate each vertex v with a sign, $s(v)$, defined as follows:

1. Let $P(v)$ be the set of ZBDD vertices that originate the 1-edges in any 1-path rooted at v. Then

$$s(v) = \prod_{x \in P(v)} sign(r(x) - r(v))\ sign(c(x) - c(v)), \tag{3.6}$$

   where $r(x)$ and $c(x)$ refer to the absolute row and column indices of vertex x in the original matrix, and u is an integer so that

$$sign(u) = \begin{cases} 1 & \text{if } u > 0, \\ -1 & \text{if } u < 0. \end{cases}$$

2. If v has an edge pointing to the 1-terminal vertex, then $s(v) = +1$.

This is called the *sign rule*. For example, in Figure 4, shown beside each vertex are the row and column indices of that vertex in the original matrix, as well as the sign of

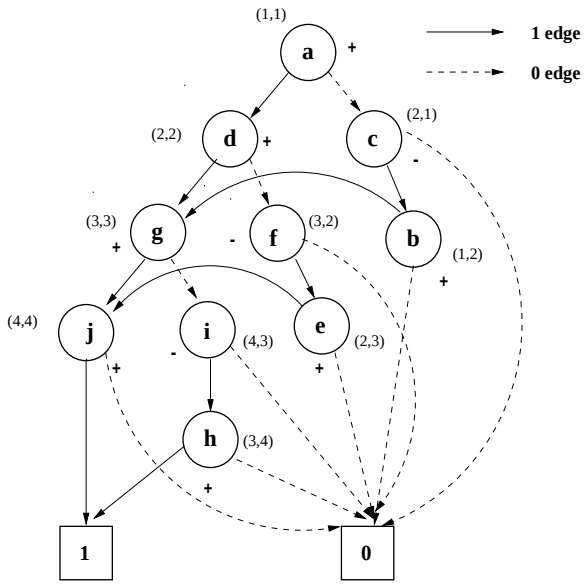


Figure 4: A signed ZBDD for representing symbolic terms

that vertex obtained by using the sign rule above. For the sign rule, we have following result:

Theorem 1 *The sign of a DDD vertex v, $s(v)$, is uniquely determined by (3.6), and the product of all the signs in a path is exactly the sign of the corresponding product term.*

Proof. We first consider one step of matrix expansion with respect to $a_{r,c}$ as defined by (3.2). The sign in the cofactor with respect to $a_{r,c}$ is $(-1)^{r+c}$. Note that r and c are, respectively, the row and column indices of the element $a_{r,c}$ in the submatrix before this step of expansion, say $\mathbf{A}^{'}$. Let the *absolute* row and column indices of the element $a_{r,c}$ in the original matrix $\mathbf{A}$ before any expansion be $r(a_{r,c})$ and $c(a_{r,c})$, respectively. Then, we observe that $r + c$ is equal to the number of rows in $\mathbf{A}^{'}$ with absolute indices less than $r(a_{r,c})$ plus the number of columns in $\mathbf{A}^{'}$ with absolute indices less than $c(a_{r,c})$. We also note that all the rows and columns in $A^{'}$ except that of $a_{r,c}$ are covered exactly once by any 1-path in the subgraph rooted at the

vertex pointed to by the 1-edge of vertex $a_{r,c}$. This is because the subgraph, rooted at the DDD vertex corresponding to $a_{r,c}$, essentially represents $det(\mathbf{A}')$, i.e. all the product terms in $\mathbf{A}'$. Hence, $(-1)^{r+c}$ is equal to the $s(a_{r,c})$ defined in equation (3.6) by selecting any 1-path in the subgraph rooted at $a_{r,c}$. As a result, the product of all the signs in a path is all the signs we will meet when we expand $\mathbf{A}$ by (3.2) to obtain the product term that the path corresponds to. □

For example, consider the 1-path *acbgih* in Figure 4. The vertices that originate all the 1-edges are c, b, i, h, their corresponding signs are $-$, $+$, $-$ and $+$, respectively. Their product is $+$. This is the sign of the symbolic product term *cbih*.

With ZBDD and the sign rule as two foundations, we are now ready to introduce formally our representation of a symbolic determinant. Let $\mathbf{A}$ be an $n \times n$ sparse matrix with a set of distinct m symbolic parameters $\{a_1, ..., a_m\}$, where $1 \leq m \leq n^2$. Each symbolic parameter a_i is associated with a unique pair $r(a_i)$ and $c(a_i)$, which denote, respectively, the row index and column index of a_i. A *determinant decision diagram* is a signed, rooted, directed acyclic graph with two terminal vertices, namely the 0-terminal vertex and the 1-terminal vertex. Each nonterminal vertex a_i is associated with a sign, $s(a_i)$, determined by the sign rule defined by (3.6). It has two outgoing edges, called 1-edge and 0-edge, pointing, respectively, to D_{a_i} and $D_{\overline{a}_i}$. A determinant decision graph having root vertex a_i denotes a matrix determinant D defined recursively as

1. If a_i is the 1-terminal vertex, then $D = 1$.
2. If a_i is the 0-terminal vertex, then $D = 0$.
3. If a_i is a nonterminal vertex, then $D = a_i\ s(a_i)\ D_{a_i} + D_{\overline{a}_i}$.

Here $s(a_i)D_{a_i}$ is the *cofactor* of D with respect to a_i, D_{a_i} is the *minor* of D with respect to a_i, $D_{\overline{a}_i}$ is the *remainder* of D with respect to a_i, and operations are algebraic multiplications and additions. For example, Figure 5 shows the DDD representation

of $det(\mathbf{M})$ under ordering $a > c > b > d > f > e > g > i > h > j$.

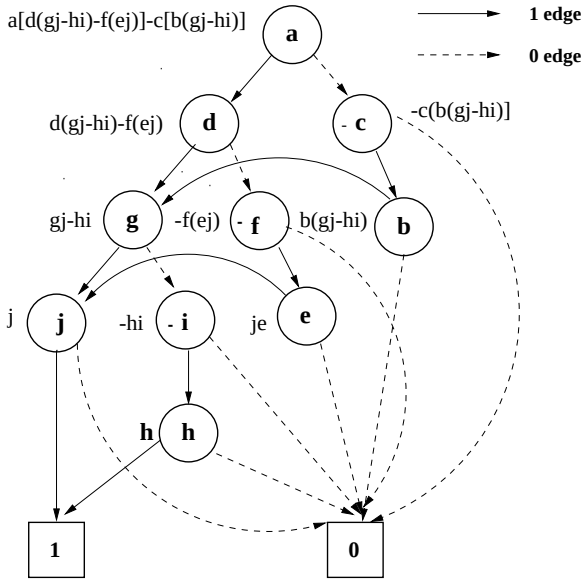


Figure 5: A determinant decision diagram for matrix $\mathbf{M}$

To enforce the uniqueness and compactness of the DDD representation, the three rules of ZBDDs, namely, zero-suppression, ordered, and shared, described in Section 3.2.1 are adopted. This leads to DDDs having the following properties:

- Every 1-path from the root corresponds to a product term in the fully expanded symbolic expression. It contains exactly n 1-edges. The number of 1-paths is equal to the number of product terms.
- For any determinant D, there is a unique DDD representation under a given vertex ordering.

We use $|DDD|$ to denote the *size of* a DDD, i.e., the number of vertices in the DDD.

3.4 An Effective Vertex-Ordering Heuristic

A key problem in many decision diagram applications is how to select a vertex ordering, since the size of the resulting decision diagram strongly depends on the

chosen ordering. For example, if we choose vertex order $a > c > d > f > g > h > b > e > i > j$ for $det(\mathbf{M})$ in Section 3.3, then the resulting DDD is shown in Figure 6. It has 13 vertices, in comparison to 10 in Figure 5, although they represent the same determinant. In this section, we describe an efficient heuristic for selecting a good vertex ordering, and show that it is optimal for a class of circuit matrices.

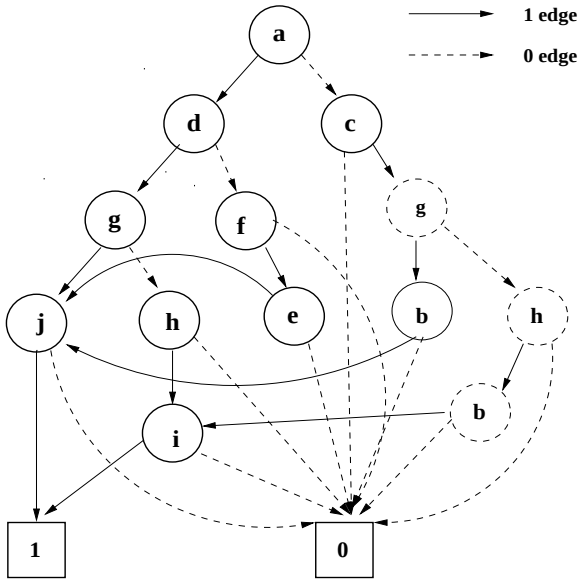


Figure 6: The DDD representing $det(\mathbf{M})$ under ordering $a > c > d > f > g > h > b > e > i > j$

We propose to select the vertex ordering for a DDD by examining the structure of the original matrix. Suppose that $\mathbf{A}$ is an $n \times n$ matrix with m nonzero elements (entries, or symbols). The vertex-ordering problem is how to *label* all the nonzero elements in $\mathbf{A}$ using integers 1 to m so that the resulting DDD constructed with the chosen order has a small size. As stated in Section 3.2.1, those elements labeled by smaller integers will appear close to the leaves, or the bottom, of the DDD, and the root is labeled by m.

We propose a vertex-ordering heuristic, based on the refinement of a well-known

strategy for Laplace expansion of a sparse matrix [30]. The basic idea is to label those columns or rows with fewer nonzero elements with larger indices. Elements in those dense rows and columns will be labeled using small indices. Intuitively, this strategy increases the possibility of DDD subgraph sharing. Figure 7 describes the proposed heuristic MATRIX_GREEDY_LABELING($\mathbf{A}(p,q)$) for labeling all the elements in matrix $A(p,q)$. In the algorithm, $A(p-\{i\},q-\{j\})$ denotes the matrix obtained from $A(p,q)$ by removing row i and column j. We keep a global counter k. Initially k is set to 1, and $p=q=e$.

```
MATRIX_GREEDY_LABELING(A(p,q))
1    select column j (row i) that has least nonzero elements
2    s ← all the rows i (columns j) so that a_{i,j} ≠ 0
3    for all row i (column j) in s from the one with least nonzero elements
4         if there exist unlabeled elements in A(p−{i},q−{j})
5              MATRIX_GREEDY_LABELING(A(p−{i},q−{j})
6    for all row i (column j) in s from the one with most nonzero elements
7         if a_{i,j} has not been labeled
8              label a_{i,j} by k and then increment k
```

Figure 7: A DDD vertex-ordering heuristic

As an example, consider how to apply MATRIX_GREEDY_LABELING to label the matrix $\mathbf{M}$ defined in Section 3.3. The complete process is illustrated in Figure 8. First, columns 1 and 4 of $\mathbf{M}$, as well as rows 1 and 4, have the least number of nonzero elements (2). We arbitrarily select column 1. Then the set of rows that have a nonzero element at column 1 are 1 and 2, i.e., $s=\{1,2\}$. Since row 1 has one nonzero element, and row 2 has two nonzero elements, lines 3-5 invoke first MATRIX_GREEDY_LABELING on matrix $\mathbf{M}$ after removing row 1 and column 1, then

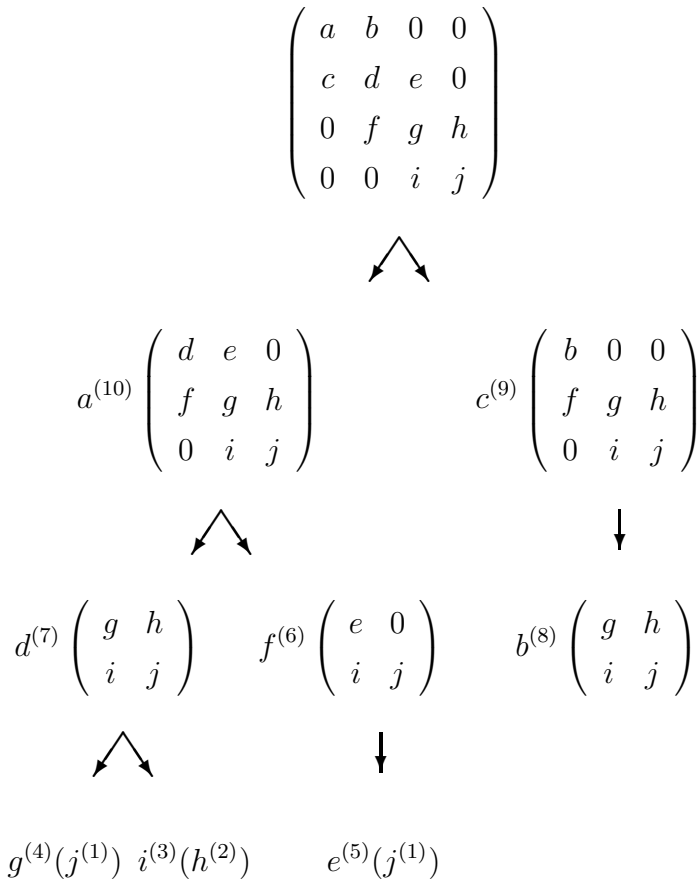


Figure 8: An illustration of DDD vertex-ordering heuristic

on matrix **M** after removing row 2 and column 1. The process is applied recursively on the resulting submatrices, and is illustrated in Figure 8 from the top to the bottom. Then, elements are labeled in the Figure 8 from the bottom to the top in the reverse order of expansion. These labels are marked in Figure 8 at the top right corner of each element. If we summarize all the labels assigned to the matrix elements using the original matrix structure, we have

$$\begin{pmatrix} a & b & 0 & 0 \\ c & d & e & 0 \\ 0 & f & g & h \\ 0 & 0 & i & j \end{pmatrix} \quad \rightarrow \quad \begin{pmatrix} 10 & 8 & 0 & 0 \\ 9 & 7 & 5 & 0 \\ 0 & 6 & 4 & 2 \\ 0 & 0 & 3 & 1 \end{pmatrix}.$$

The heuristic leads to compact DDDs for a large class of circuit matrices, as observed in our experiments described in Section 3.6.

In fact, algorithm MATRIX_GREEDY_LABELING yields the optimal vertex ordering for a class of circuit matrices, called *band matrices*. Band matrices are matrices that have only nonzero elements at positions (i, i), $(i-1, i)$ and $(i+1, i)$. They have the following structure:

$$\mathbf{A}_{n+1,n+1} = \begin{pmatrix} a_{n+1,n+1} & a_{n+1,n} & 0 & ... & .. & ... & 0 \\ a_{n,n+1} & a_{n,n} & a_{n,n-1} & 0 & ... & ... & 0 \\ 0 & a_{n-1,n} & a_{n-1,n-1} & a_{n-1,n-2} & 0 & ... & 0 \\ 0 & 0 & ... & ... & ... & ... & ... \\ 0 & ... & 0 & a_{3,4} & a_{3,3} & a_{3,2} & 0 \\ 0 & ... & ... & 0 & a_{2,3} & a_{2,2} & a_{2,1} \\ 0 & ... & ... & ... & 0 & a_{1,2} & a_{1,1} \end{pmatrix}.$$

The number of nonzero elements in $\mathbf{A}_{n+1}$ is $3n + 1$. Then, we have the following result:

Theorem 2 *The DDD representation under the vertex ordering given by* MATRIX_GREEDY_LABELING *for a band matrix is optimal.*

Proof. We note that the lower bound on the number of DDD vertices is equal to the

number of matrix entries, i.e., each vertex appears once in the final DDD. We will show that, for band matrices, the ordering given by MATRIX_GREEDY_LABELING results in a DDD with the number of vertices equal to the number of nonzero matrix elements. This is proved by induction. It is easy to see that the result is true for 1×1 and 2×2 matrices. Now we assume that it is true for $\mathbf{A}_{n,n}$, i.e., the number of DDD vertices is $3n-2$. We prove that it is also true for $\mathbf{A}_{n+1,n+1}$: the number of DDD vertices is $3n+1$. Let vertex $D_{n-1,n-1}$ represent the DDD of $det(\mathbf{A}_{n-1,n-1})$ and $D_{n,n}$ represent the DDD of $det(\mathbf{A}_{n,n})$. Matrix $\mathbf{A}_{n+1,n+1}$ has an extra row and column with three nonzero elements $a_{n+1,n+1}$, $a_{n,n+1}$, and $a_{n+1,n}$. Algorithm MATRIX_GREEDY_LABELING assigns integer labels $3n+1$, $3n$ and $3n-1$ to elements $a_{n+1,n+1}$, $a_{n,n+1}$, and $a_{n+1,n}$, respectively. This gives ordering $a_{n+1,n+1} > a_{n,n+1} > a_{n+1,n} > a_{n,n}$. We thus first create a DDD vertex labeled by $a_{n+1,n+1}$. Its 1-edge points to vertex $D_{n,n}$, and its 0-edge points to next vertex, which corresponds to $a_{n,n+1}$. The 0-edge of vertex $a_{n,n+1}$ points to the 0-terminal. Its 1-edge points to the vertex that represents the determinant of matrix $\mathbf{A}_{n+1,n+1}$ after removing the first column and the second row. Since the first row in the resulting matrix contains only one nonzero element $a_{n+1,n}$, we can create a DDD vertex for $a_{n+1,n}$ with its 1-edge pointing to $D_{n-1,n-1}$, and its 0-edge pointing to the 0-terminal. This is illustrated in Figure 9. Only three vertices are added for representing $\mathbf{A}_{n+1,n+1}$. The total number of DDD vertices for $\mathbf{A}_{n+1,n+1}$ is thus $3n+1$. □

We have thus proved that for band matrix $\mathbf{A}_{n,n}$, the number of DDD vertices is $3n-2$. As a comparison, the number of expanded product terms in $det(\mathbf{A}_{n,n})$ is $F(n+1)$, where $F(i)$ is the ith Fibonacci number defined by

$$\begin{aligned} F(n) &= F(n-1) + F(n-2), && n > 2, \\ F(1) &= F(2) = 1. \end{aligned}$$

Each product term involves n symbols. To store all the product terms without con-

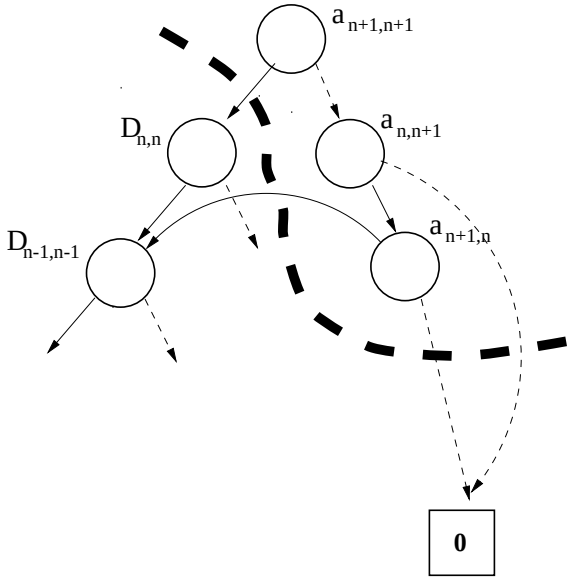


Figure 9: An illustration of DDD construction for band matrices

sidering sharing, the memory requirement is proportional to $nF(n+1)$. Further, any symbolic manipulation using the expanded form would have time complexity at best proportional to $nF(n+1)$. In Figure 10, the number of DDD vertices is plotted against the number of product terms.

The practical relevance of band matrices is that they correspond to the circuit matrices for ladder networks—an important class of structures in analog design. A three-section ladder network is shown in Figure 11.

The system of equations can be formulated as follows:

$$\begin{pmatrix} \frac{1}{R_1} & -\frac{1}{R_2} & 0 & 0 \\ -\frac{1}{R_2} & \frac{1}{R_1}+\frac{1}{R_2}+\frac{1}{R_3} & -\frac{1}{R_3} & 0 \\ 0 & -\frac{1}{R_3} & \frac{1}{R_3}+\frac{1}{R_4}+\frac{1}{R_5} & -\frac{1}{R_5} \\ 0 & 0 & -\frac{1}{R_5} & \frac{1}{R_5}+\frac{1}{R_6} \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{pmatrix} = \begin{pmatrix} I_{in} \\ 0 \\ 0 \\ 0 \end{pmatrix}. \tag{3.7}$$

If we view each entry as a distinct symbolic parameter, the resulting circuit matrix is a band matrix. We note that many practical circuits have ladder-like structures.

We emphasize that as with BDD representation for Boolean functions, in the

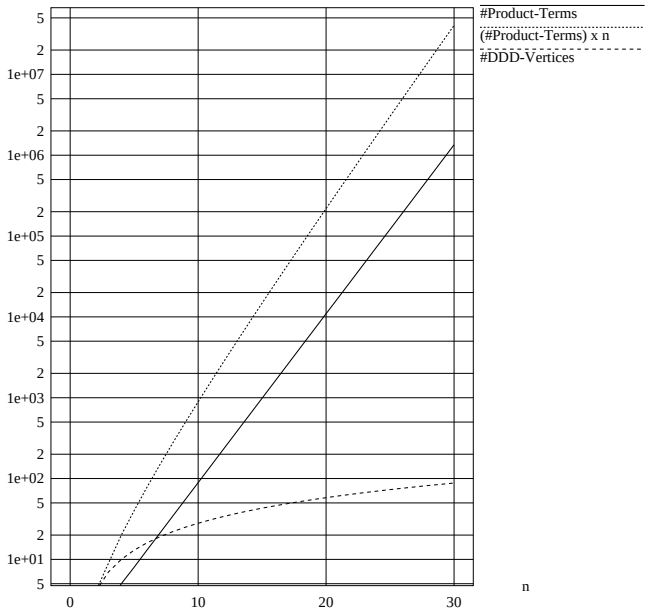


Figure 10: A comparison of DDD sizes versus numbers of product terms for band matrices

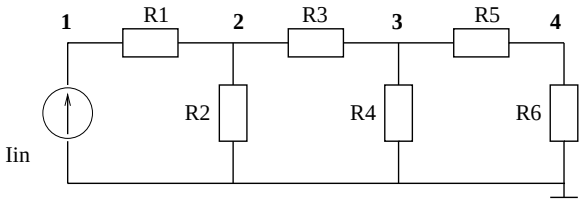


Figure 11: A three-section ladder network

worst cases the number of DDD vertices can grow exponentially with the size of a circuit. Nevertheless, as we have observed that with the proposed vertex ordering, the numbers of DDD vertices are manageable for practical analog circuits.

3.5 Manipulation and Construction of Determinant Decision Diagrams

In this section, we show that, using determinant decision diagrams, algorithms needed for symbolic analysis and its applications can be performed with the time complexity proportional to the size of the diagrams being manipulated, *not the number of 1-paths in the diagrams, i.e., product terms in the symbolic expressions.* Hence, as long as the determinants of interest can be represented by reasonably small graphs, our algorithms are quite efficient.

A basic set of operations on matrix determinants is summarized in Table 1. Most operations are simple extensions of subset operations introduced by Minato on ZBDDs [59]. These few basic operations can be used directly and/or combined to perform a wide variety of operations needed for symbolic analysis. In this section, we first describe these operations, and then use an example to illustrate the main ideas of these operations and how they can be applied to compute network function sensitivities—a key operation needed in optimization and testability analysis. We also show that the generation of significant product terms can be cast as the k-shortest path problem in a DDD and solved elegantly in time $O(k \cdot |DDD|)$.

3.5.1 Implementation of Basic Operations

We summarize the implementation of these operations in Figure 12. For the clarity of the description, the steps for computing the signs associated with DDD vertices, using the sign rule defined in Section 3.3, are not shown.

As the basis of implementation, we employ two techniques originally developed by Brace, Rudell and Bryant for efficiently implementing decision diagrams [7]. First,

```
Cofactor(D, s)
1   if (D.top < s) return VertexZero()
2   if (D.top = s) return D_1
3   if (D.top > s) return GetVertex(D.top,
    Cofactor(D_0, s), Cofactor(D_1, s))

Remainder(D, s)
1   if (D.top < s) return D
2   if (D.top = s) return D_0
3   if (D.top > s) return GetVertex(D.top,
    Remainder(D_0, s), Remainder(D_1, s))

Multiply(D, s)
1   if (D.top < s) return GetVertex(s, 0, D)
2   if (D.top = s) return GetVertex(s, D_1, D_0)
3   if (D.top > s) return GetVertex(D.top,
    Multiply(D_0, s), Multiply(D_1, s))

Subtract(D, P)
1   if (D = 0) return VertexZero()
2   if (P = 0) return D
3   if (D = P) return VertexZero()
4   if (D.top > P.top) return GetVertex(D.top, Subtract(D_0, P), D_1)
5   if (D.top < P.top) return Subtract(D, P_0)
6   if (D.top = P.top)
        return GetVertex(D.top, Subtract(D_0, P_0), Subtract(D_1, P_1))

Union(D, P)
1   if (D = 0) return P
2   if (P = 0) return D
3   if (D = P) return P
4   if (D.top > P.top) return GetVertex(D.top, Union(D_0, P), D_1)
5   if (D.top < P.top) return GetVertex(P.top, Union(D, P_0), P_1)
6   if (D.top = P.top)
        return GetVertex(D.top,Union(D_0, P_0), Union(D_1, P_1))

DDD_of_Matrix(A(p, q))
1   if (A = 0) return VertexZero()
2   if (A = 1) return VertexOne()
3   let s be a nonzero element at row i and column j of A
4       return GetVertex(s,
        DDD_of_Matrix(A(p - {i}, q - {j})), DDD_of_Matrix(A|_{s=0}))

Evaluate(D)
1   if (D = 0) return 0
2   if (D = 1) return 1
3   return Evaluate(D_0) + s(D) * D.top * Evaluate(D_1)
```

Figure 12: Implementation of basic operations for symbolic analysis

Table 1: Summary of basic operations

Determinant operation	Result	Subset operation
VertexOne()	return 1	Base()
VertexZero()	return 0	Empty()
Cofactor(D, s)	return the cofactor of D wrt s	Subset1(D, s)
Remainder(D, s)	return the remainder of D wrt s	Subset0(D, s)
Multiply(D, s)	return $s \times D$	Change(D, s)
Subtract(D, P)	return $D - P$	Diff(D, P)
Union(D, P)	return $D + P$	Union(D, P)
ddd_of_Matrix($\mathbf{A}$)	construct the DDD for matrix $\mathbf{A}$	–
Evaluate(D)	return the numerical value of D	–

a basic procedure GetVertex(top, D_1, D_0) is to generate (or copy) a vertex for a symbol *top* and two subgraphs D_1 and D_0. In the procedure, a hash table is used to keep each vertex unique, and vertex elimination and sharing are managed mainly by GetVertex. With GetVertex, all the operations for DDDs we need are described in Figure 12.

Second, similar to conventional BDDs, we use a cache to remember the results of recent operations, and refer to the cache for every recursive call. In this way, we can avoid duplicate executions for equivalent subgraphs. This enables us to execute these operations in a time linearly proportional to the size of a graph.

Evaluation: Given a determinant decision diagram pointed to by D and a set of numerical values for all the symbolic parameters, Evaluate(D) computes the numerical value of the corresponding matrix determinant. Evaluate(D) naturally exploits subexpression sharing in a symbolic expression, and has time complexity *linear* in the size of the diagram.

Construction: Let $\mathbf{A}(e, e)$ be an n-by-n symbolic matrix, where e is a set of

integers from 1 to n. DDD_OF_MATRIX($\mathbf{A}(p, q)$) constructs a determinant decision diagram of a submatrix of $\mathbf{A}$ with row set $p \subseteq e$ and column set $q \subseteq e$ such that $|p| = |q|$ for a given ordering of symbolic parameters. It can be viewed as a generalized Laplace expansion procedure of matrix determinants. In line 3 of the procedure DDD_OF_MATRIX, a nonzero element s is selected, and the determinant is expanded. Due to the canonical property of DDD, s can be any nonzero matrix element, and the resulting DDD is always the same. However, the best expansion order is the DDD vertex ordering. That is to use the element with the largest integer label in line 3 of the procedure DDD_OF_MATRIX.

Cofactor and Derivative: COFACTOR(D, s) is to compute the cofactor of a symbolic determinant D represented by a DDD with respect to symbolic parameter s. It is exactly the derivative of D with respect to s. COFACTOR is perhaps the most important operation in symbolic analysis of analog circuits. For example, the network functions can be obtained by first computing some cofactors, and then combining these cofactors according to some rules (Cramer's rule).

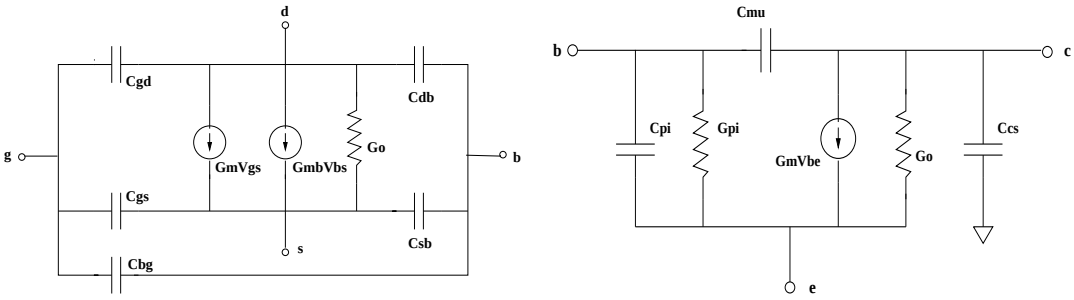
3.6 Experimental Results

We have implemented in C++ the proposed symbolic analysis algorithms, and tested our program on a set of circuits varying from RLC filters to bipolar and MOS integrated circuits. The set of test circuits includes:

- *miller*, a two-stage miller compensated MOS Opamp from [30]
- $\mu A741$, a bipolar Opamp containing 26 transistors and 11 resistors, with the schematic in Figure 14
- *Cascode*, a CMOS Cascode Opamp containing 22 transistors with the schematic in Figure 15

- *ladder7, ladder21, ladder100*, 7-, 21- and 100-section cascade resistive ladder networks
- *rctree1, rctree2*, two RC tree networks
- some RLC filters, named *butter*, *rlctest*, *vcstest*, *ccstest* and *bigtst*

For nonlinear integrated circuits, DC analysis is first performed using SPICE, and the resulting small-signal models from the output of SPICE are used in symbolic analysis. The small-signal models used for bipolar and MOS transistors are described in Figure. 13(a) and 13(b), respectively. The modified nodal analysis (MNA) approach as in SPICE is used to formulate the circuit equations. Exact symbolic expressions for network transfer functions (voltage gains) are computed using Cramer's rule and shared DDD representations of symbolic determinants and cofactors. The transfer function is in the form of the ratio of two DDDs, which are represented compactly using a *single shared* DDD with two roots; this is referred to as the DDD representation of the transfer function.



(a) MOSFET small-signal model

(b) Bipolar transistor model

Figure 13: Small-signal models for nonlinear devices

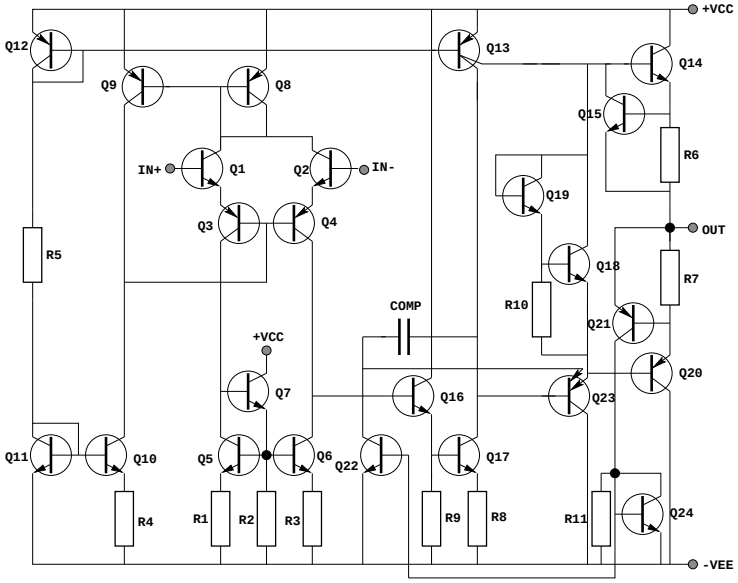


Figure 14: The circuit schematic of bipolar $\mu A741$

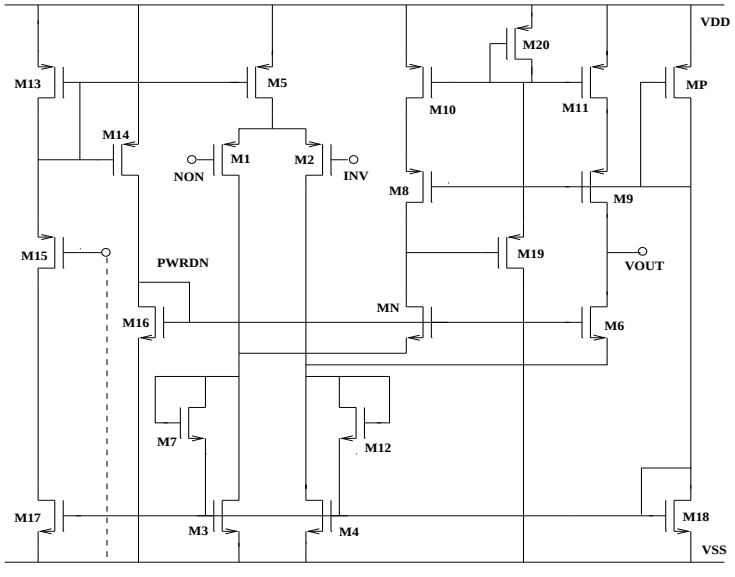


Figure 15: The circuit schematic of MOS Cascode Opamp

Table 2 [1] describes the statistics of all the test circuits, the resulting DDD sizes, and SCAPP results. Columns 2 to 6 describe, respectively, the number of nodes in each circuit, the number of nonzero elements in each circuit matrix, the number of product terms in the transfer function and the number of vertices in the DDD representation of the transfer function without and with the use of the vertex-ordering heuristic described in Figure 7. For each circuit, the number of DDD vertices without vertex-ordering is the average of that of ten randomly generated orderings. SCAPP is used to generate the sequence of expressions (in the C program) for each transfer function. The number of multiplications, the number of additions and the number of intermediate expressions used for computing each transfer function are reported in Columns 7 to 9. The sequence of expressions can be generated from the DDD representation of the transfer function. In this case, the number of multiplications and the number of additions are bounded by the number of DDD vertices.

Table 2: Comparison of DDD sizes without/with vertex-ordering and SCAPP

ckt name	#nodes	#symbols (matrix)	#terms	$\|DDD\|$ w/o ordering	$\|DDD\|$ w/ ordering	SCAPP		
						#mul	#add	#expr
miller	7	21	29	95.2	51	36	60	44
butter	8	19	14	57.1	22	30	21	39
ladder7	9	22	22	85.6	26	36	25	46
rlctest	10	37	200	634.2	152	208	161	219
ccstest	10	35	176	510.8	97	208	162	219
vcstest	11	46	120	435.8	70	238	169	252
ladder21	23	64	17712	53434.4	84	116	79	140
Cascode	15	77	119395	150868.1	1994	444	596	460
$\mu A741$	24	89	119011	$-*$	6654	198	365	233
bigtst	33	112	1.6×10^7	–	951	996	715	1030
ladder100	102	301	5.7×10^{20}	–	398	594	398	697
rctree1	41	119	7.1×10^7	–	208	508	311	550
rctree2	54	158	3.0×10^{10}	–	299	660	407	715

-*: out of memory

[1] In [79], only the statistics of the determinants of the circuit matrices are reported. Here the statistics are collected for the transfer functions in order to compare with SCAPP, which calculates the transfer functions.

From Table 2, we can make several observations:

- Ordering leads to DDDs significantly smaller than that of nonordering. For ladder networks *ladder7*, *ladder10*, *ladder21*, the numbers of DDD vertices are exactly the numbers of nonzero matrix elements.
- For a small circuit, the number of DDD vertices may be greater than the number of product terms. This is not surprising, since the number of DDD vertices without exploiting sharing would be the number of product terms times the number of symbolic parameters in each product term.
- For large circuits, the numbers of DDD vertices can be several orders of magnitude smaller than the numbers of product terms. Further, the difference becomes more dramatic with the increase of the circuit size.
- The DDD-based approach uses about two thirds of multiplications of SCAPP for ladder-structured circuits (*ladder7, ladder21, ladder100*), and uses less than half the amount of multiplications of SCAPP for tree-structured circuits (*rctree1* and *rctree2*). It generates the sequences of expressions with many more operations than SCAPP for those circuits that do not have ladder- or tree-like structures (*cascodeOpamp* and $\mu A741$). [2]

We then compare our program with *ISAAC* [30] and *Maple-V* [11] for generating the complete sum-of-product expressions of the transfer functions. *ISAAC* is a well-known special-purpose symbolic analyzer designed for analog integrated circuits. *Maple-V* is a general-purpose mathematic package capable of solving linear equations symbolically. To use *Maple-V*, we use our program to set up the circuit equations and then feed the circuit matrices to *Maple*. The results are described in Table 3. *CPU time* and *Memory* are the time and memory usage of driving symbolic expres-

[2] We will show, in Chapter V, that by exploiting design hierarchy and automated partitioning, the DDD-based approach can generate even more compact representations than that by SCAPP (see also [83]).

sions by different methods. All data are obtained using a Sun-SPARC 20 with 32M memory. We can observe that our program runs significantly faster than both *ISAAC* and *Maple-V*, and uses much less memory. For slightly larger circuits, both *ISAAC* and *Maple-V* ran out of memory. The symbolic expressions generated by ISAAC, Maple-V and the DDD-based approach are the same for each circuit.

Table 3: Comparison of the proposed algorithm against ISAAC and Maple-V

circuit	ISAAC		Maple-V		Proposed Algorithm	
	CPU time (seconds)	Memory (bytes)	CPU time (seconds)	Memory (bytes)	CPU time (seconds)	Memory (bytes)
miller	0.49	640k	0.6	250k	0.066	24.5k
butter	0.61	493k	0.03	10.9k	0.033	8.1k
ladder7	1.05	980k	0.63	250k	0.033	8.1k
rlctest	–*	–	17.3	15.2M	0.10	49.1k
vcstest	–	–	104.3	23.1M	0.15	112.1k
ccstest	–	–	23.4	17.3M	0.06	128.3k
ladder21	–	–	–	–	0.066	32.7k
Cascode	–	–	–	–	7.62	4.9M
$\mu A741$	–	–	–	–	9.13	5.2M
bigtst	–	–	–	–	2.81	2.6M
ladder100	–	–	–	–	1.43	1.4M
rctree1	–	–	–	–	0.36	0.5M
rctree2	–	–	–	–	137.3	16.1M

–*: out of memory.

To verify the numerical correctness of our symbolic analysis algorithms, we have implemented a frequency-domain circuit simulator based on DDD evaluation. For all the test circuits, our simulator produced the exactly same results as SPICE did. We note that the complexity of such a *repetitive numerical evaluation* is linearly proportional to the number of DDD vertices. Since the number of DDD vertices may

grow exponentially with the size of the circuit, the method of DDD-based numerical simulation is not desirable for large analog circuits. Table 4 shows the comparison of our symbolic analysis with SPICE in terms of CPU time in repetitive numerical evaluation. For each circuit, 1000 frequency points were simulated. The proposed algorithm is actually faster than SPICE for some small circuits, but is slower for large circuits. In general, unless other properties of the symbolic expressions are exploited, DDD-based numerical evaluation do not have advantages over SPICE-like simulators for large circuits. This is because LU decomposition used in SPICE has the complexity $O(n^3)$, and it is bounded by $O(n^{1.1-1.4})$ in practice [61]. However, the advantage of symbolic analysis is not in numerical simulation, but in those applications where analytic expressions are more desirable than numerical values such as analog synthesis [30] and testability analysis [9].

3.7 Summary

In this chapter, a new graph representation, called Determinant Decision Diagrams (DDDs), for symbolic matrix determinants is proposed and exact symbolic analysis algorithms for analog circuits are presented. Unlike previous approaches based on either the expanded form or the nested form representations of symbolic expressions, DDD-based symbolic analysis utilizes the sparsity and sharing in a canonical manner. We described an efficient vertex-ordering heuristic. We proved theoretically that the heuristic yields an optimum ordering for ladder-structured circuits, and observed experimentally that for practical circuits the numbers of DDD vertices are quite small—usually several orders-of-magnitude smaller than the number of product terms in the expanded form. Generating the complete sum-of-product symbolic expressions from the DDD representation offers, for large analog circuits, orders-of-magnitude improvement in both CPU time and memory usages over the symbolic analyzers ISAAC and Maple-V. It enables the exact analysis of such large analog

Table 4: Comparison of frequency analysis by the proposed algorithm and by SPICE

circuit	SPICE (seconds)	DDD-based numerical evaluation (seconds)
miller	0.36	0.28
butter	0.96	0.26
ladder7	0.30	0.24
rcltest	0.71	0.65
ccstest	0.94	0.70
vcstest	0.82	0.62
ladder21	0.96	0.75
Cascode	3.16	11.81
$\mu A741$	3.2	39.22
bigtst	3.68	2.78
ladder100	4.52	3.97
rctree1	1.05	1.80
rctree2	1.41	2.52

circuits as $\mu A741$ for the first time.

We also compared the DDD-based approach with the best-known hierarchical symbolic analyzer SCAPP in generating the sequences of expressions for network transfer functions. For ladder-structured circuits, the DDD-based approach uses only two thirds of multiplications as required by SCAPP. For large circuits that do not have ladder-like structures such as $\mu A741$ Opamp, the sequences of expressions generated by the DDD-based approach are generally manageable by modern computers, but can be substantially larger than those from SCAPP. But DDD representation is more amenable to efficient symbolic manipulations than the sequences of expressions generated by SCAPP.

CHAPTER IV S-EXPANDED DETERMINANT DECISION DIAGRAMS

4.1 Introduction

The transfer function of a linear(ized) circuit can be represented as a ratio of two polynomials in frequency domain, in the following general form:

$$H(s) = \frac{N(p_1, p_2, \cdots, p_m, s)}{D(p_1, p_2, \cdots, p_m, s)}, \tag{4.1}$$

where the numerator, $N(p_1, p_2, \cdots, p_m,$ s), and denominator, the $D(p_1, p_2, \cdots, p_m, s)$, are polynomials in the circuit parameters $p_1, p_2, \cdots, p_m$ and complex frequency s. The numerator and the denominator are typically composed of the determinants of the circuit matrix of the circuit or its cofactors. In Chapter III, we introduced determinant decision diagrams to represent all those determinants and cofactors.

For many symbolic analysis applications, however, such a representation is still inadequate. These applications commonly require symbolic expressions to be represented in the so-called *fully expanded form in s* or in the *s-expanded form*. For an $n \times n$ circuit matrix $A(s)$ with its entries being the linear function in the complex frequency s, its determinant, $det(A(s))$, can be written into an s-expanded polynomial of degree n:

$$det(A(s)) = a_n s^n + a_{n-1} s^{n-1} +, ..., +a_0. \tag{4.2}$$

As a result, the same linear(ized) circuit transfer function $H(s)$ can be written in the following s-expanded form:

$$H(s) = \frac{\sum f_i(p_1, p_2, \cdots, p_m) s^i}{\sum g_j(p_1, p_2, \cdots, p_m) s^j}, \tag{4.3}$$

where $f_i(p_1, p_2, \cdots, p_m)$ and $g_j(p_1, p_2, \cdots, p_m)$ are symbolic polynomials that do not contain the complex variable s. Despite the usefulness of s-expanded symbolic expressions, no efficient derivation method exists. The difficulty is still rooted in the huge number of s-expanded product terms that are far beyond the capabilities of symbolic analyzers using traditional methods. Although the numerical interpolation method reviewed in Chapter II can generate s-expanded expressions, only complex frequency s is kept as a symbol. This method also suffers the numerical problem due to the ill-conditioned equations for solving for numerical coefficients, and thus has limited applications.

Various simplification schemes can be used to find approximate symbolic expressions [26, 31]; yet it is well known that approximate expressions may not be adequate for such circuit characterizations as symbolic pole-zero derivations [31]. Arbitrarily nested hierarchical expressions or sequences of expressions can be useful for circuit simulation [39], but no method exists to derive the s-expanded expressions from these representations.

It turns out that s-expanded polynomial expressions can be easily computed using determinant decision diagrams and the related graph-based operations. The key idea still lies in the exploration of the DDD structure. The result is a slightly different DDD, called an s-expanded determinant decision diagram (s-expanded DDD), where each root represents a symbolic coefficient of a particular power of s. s-Expanded DDDs share all the properties of DDDs, but greatly extend the capabilities of DDDs in representing symbolic expressions.

We present an efficient algorithm of constructing an s-expanded DDD from an original DDD. If the maximum number of admittance parameters in an entry of a circuit matrix is bounded (true for most practical analog circuits), we prove that both the size of the resulting s-expanded DDD and the time complexity of the construction

algorithm is $O(m|D|)$, where m is the highest power of s in the s-expanded polynomial of the determinant of the circuit matrix and $|D|$ is the size of the original DDD D representing the determinant. Experimental results indicate that the number of DDD vertices used can be many orders-of-magnitudes less than that of product terms represented by the DDDs.

With s-expanded expressions, approximation on symbolic transfer functions can be performed very efficiently (see Chapter VII). In addition, symbolic poles and zeros can also be approximated using the pole-splitting method [27] (also see Chapter VII). With some or all circuit parameters substituted with numerical values, frequency response calculation using s-expanded representation can be performed much faster than SPICE-like methods [51]. Such an s-expanded form also paves the way for our circuit-level noise analysis and modeling method (see Chapter VIII).

The rest of this chapter is organized as follows: Section 4.2 introduces the concept of s-expanded determinant decision diagrams through a circuit example. Section 4.3 presents an efficient vertex-ordering scheme for s-expanded DDDs. An efficient procedure for deriving s-expanded DDDs from original DDDs is presented in Section 4.4. Section 4.5 reports some experimental results on the circuit behavioral modeling using the proposed method and the comparison with SPICE on circuit simulation. Section 4.6 concludes this chapter with a summary.

4.2 s-Expanded Symbolic Representation

In this section, we introduce the concept of s-expanded determinant decision diagrams. Instead of presenting the concept in a formal way, we illustrate it through a circuit example.

Consider a simple circuit given in Figure 16. By using the nodal formulation,

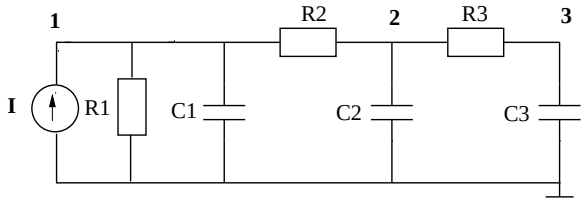


Figure 16: An example circuit

its circuit matrix can be written as

$$\begin{array}{c} v_1 \\ v_2 \\ v_3 \end{array} \begin{bmatrix} \frac{1}{R_1} + sC_1 + \frac{1}{R_2} & -\frac{1}{R_2} & 0 \\ -\frac{1}{R_2} & \frac{1}{R_2} + sC_2 + \frac{1}{R_3} & -\frac{1}{R_3} \\ 0 & -\frac{1}{R_3} & \frac{1}{R_3} + sC_3 \end{bmatrix}.$$

In modified nodal analysis formulation, the admittance of each circuit or lumped circuit parameter, p_i, arrives in the circuit matrix in one of three forms—$g_i, c_i s$ and $1/(l_1 s)$—for the admittance of resistances and capacitances and inductances, respectively. To construct DDDs, we need to associate a label with each entry of a circuit matrix. We call this procedure *labeling scheme*.

Instead of labeling one symbol for each matrix entry, we label each admittance parameter in the entries of the circuit matrix when deriving the s-expanded DDDs. Depending on how the circuit parameters are labeled, an s-expanded DDD comes in two forms: (1) In the first labeling scheme, all the circuit parameters in an entry of circuit matrix are first lumped together according to their admittance type, and each lumped admittance parameter is then represented by a unique symbol. (2) In the second labeling scheme, we label each admittance of circuit parameters by a unique symbol. Obviously the second labeling scheme will generate more product terms than the first. The selection of labeling schemes depends on the applications of symbolic analysis. In this chapter, we present both labeling schemes along with their implementations.

By the first labeling scheme, we can rewrite the circuit matrix of the example

circuit as follows:

$$\begin{bmatrix} a+bs & c & 0 \\ d & e+fs & g \\ 0 & h & i+js \end{bmatrix},$$

where $a = \frac{1}{R_1} + \frac{1}{R_2}$, $b = C_1$, $c = d = -\frac{1}{R_2}$, $e = \frac{1}{R_2} + \frac{1}{R_3}$, $f = C_2$, $g = h = -\frac{1}{R_3}$, $i = \frac{1}{R_3}$ and $j = C_3$. By using the second labeling scheme, the circuit matrix can be rewritten as follows:

$$\begin{bmatrix} a+b+cs & d & 0 \\ e & f+g+hs & i \\ 0 & j & k+ls \end{bmatrix},$$

where $a = \frac{1}{R_1}$, $b = f = \frac{1}{R_2}$, $d = e = -\frac{1}{R_2}$ $g = k = \frac{1}{R_3}$, $i = j = -\frac{1}{R_3}$, $c = C_1, h = C_2, l = C_3$.

We first consider the original DDD representation shown in Figure 17 of the circuit matrix. Each DDD vertex is labeled using the first labeling scheme.

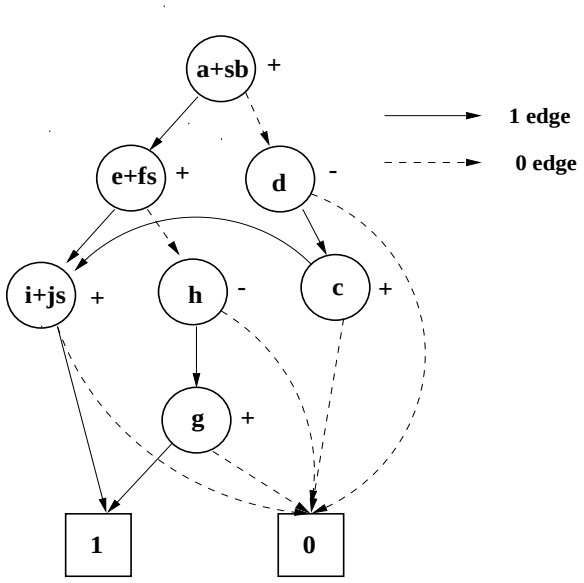


Figure 17: Complex DDD for a matrix determinant

By the definition of DDDs, each 1-path in a DDD corresponds to a product

term in the determinant that the DDD represents. In this example, there are three 1-paths, and thus three product terms:

$$(a+sb)(e+fs)(i+js),$$
$$(a+sb)(-h)(g),$$
$$(-d)(c)(i+js).$$

We now consider how to expand a symbolic expression into an s-expanded one and represent the expanded product terms by a new DDD structure. Expanding the three product terms, we have

$$(a+sb)(e+fs)(i+js) \rightarrow \begin{cases} +aeis^0 & +aejs^1 \\ +afis^1 & +beis^1 \\ +afjs^2 & +bejs^2 \\ +bfis^2 & +bfjs^3 \end{cases},$$
$$(a+sb)(-h)(g) \rightarrow \begin{cases} -ahgs^0 \\ -bhgs^1 \end{cases},$$
$$(-d)(c)(i+js) \rightarrow \begin{cases} -dcis^0 \\ -dcjs^1 \end{cases}.$$

We can easily represent these product terms using a multi-rooted DDD structure as shown in Figure 18. The new DDD has four roots and each DDD root represents a symbolic expression of a coefficient of a particular power of s. Each DDD seen from a root is called a *coefficient* DDD, and the resulting multi-rooted DDD is called an *s-expanded* DDD. The original DDD is referred to as the *complex* DDD as complex frequency variable s appears in some vertices throughout the rest of the dissertation. Such a representation exploits the sharing among different coefficients in a polynomial in addition to that explored by complex DDDs. In Figure 18, 18 nonterminal vertices are used. In comparison, without exploiting the sharing and the sparsity, 108 ($= 12 \times 9$, #product-terms $\times$ #symbols) vertices would be used.

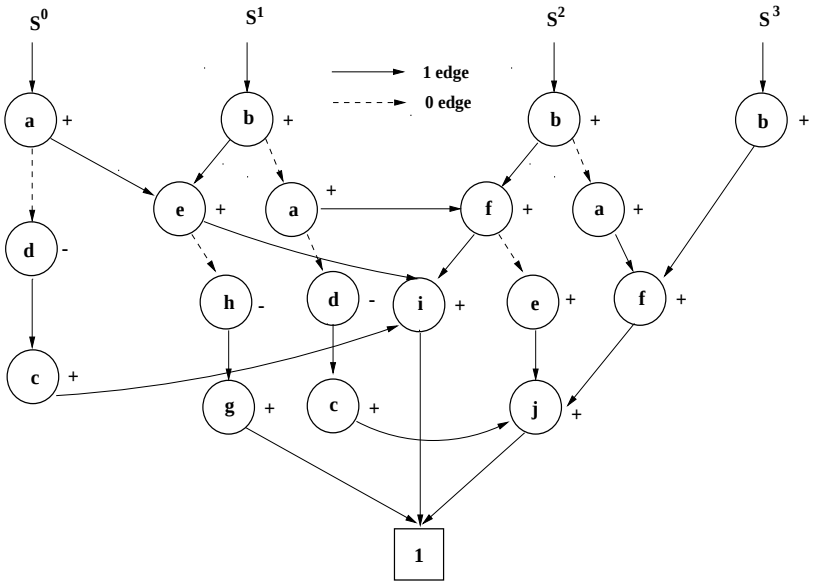


Figure 18: An s-expanded DDD by the first labeling scheme

Note that each vertex a in a complex DDD may be mapped into several vertices, $a_i, i = 1, ..., m$, in the resulting s-expanded DDD. We say that *a contains* a_i and denote this relationship by $a_i \subset a$. As a result, a product term, p, in a complex DDD will generate a number of product terms, $p_i, i = 1, ..., l$, in the resulting s-expanded DDD. Similarly, we say *p contains* p_i and denotes this relationship by $p_i \subset p$. If we further define the row and the column indices of a vertex a_i in a coefficient DDD as that of $a, a_i \subset a$, respectively, we have the following result:

Proposition 1 *A coefficient DDD represents the sum of all the s-expanded product terms of particular power of s in the s-expanded polynomial of a determinant.*

Proof. We note that for a vertex a_i in a coefficient DDD, there is one and only one vertex a in the complex DDD such that $a_i \subset a$. Because vertex a_i has the same row and column indices as a, so a_i has the same sign as a using the sign rule defined in Chapter III. Hence the sign of each product term p_i in a coefficient DDD will be identical to that of product term p where $p_i \subset p$. Hence all the product terms

$p_i,\ p_i \subset p,\ i = 1, ..., m$ have the same sign as p. This agrees with the arithmetic operation of transforming an expression in a product-of-sum form into the one in a sum-of-product form. □

Lemma 1 implies that an s-expanded DDD shares the same properties as a complex DDD, although it does not represent a determinant, instead only those terms that have the same powers of s in a determinant. All the manipulations of complex DDDs mentioned in the Chapter III therefore can be applied to s-expanded DDDs.

Under a fixed vertex ordering of all vertices representing admittance parameter in a circuit matrix, the representation of the circuit-matrix determinant by an s-expanded DDD is also canonical. The canonical property in an s-expanded DDD ensures that the maximum sharing among all its coefficients is attained, and the size of the resulting s-expanded DDD is a minimum under a vertex ordering.

If we adopt the second labeling scheme, the same three product terms in the complex DDD of the example circuit will be expanded into 23 product terms in different powers of s:

$$(a+b+cs)(f+g+hs)(k+ls) \rightarrow \left\{ \begin{array}{llll} +afks^0 & +bgls^1 & +agks^0 & +ahks^1 \\ +bfks^0 & +bhks^1 & +bgks^0 & +ahls^2 \\ +cfks^1 & +bhls^2 & +cgks^1 & +chks^2 \\ +afls^1 & +cfls^2 & +agls^1 & +cgls^2 \\ +bfls^1 & +chls^3 & & \end{array} \right. ,$$

$$(a+b+cs)(-j)(i) \rightarrow \left\{ \begin{array}{l} -ajis^0 \\ -bjis^0 \\ -cjis^1 \end{array} \right. ,$$

$$(-e)(d)(k+ls) \rightarrow \left\{ \begin{array}{l} -edks^0 \\ -edls^1 \end{array} \right. .$$

The resulting s-expanded DDD is depicted in Figure 19. It is easy to see that the

second labeling scheme results in more vertices than the first one. The resulting s-expanded DDD has the same properties as the previous one (using the first labeling scheme), but it will be more suited for the DDD-based approximation to be presented in Chapter VII.

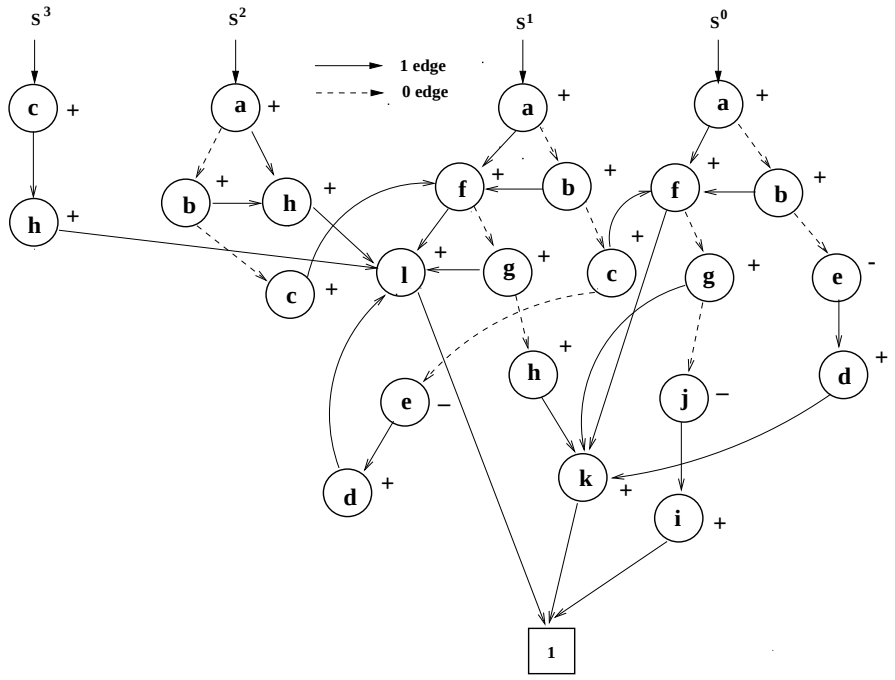


Figure 19: An s-expanded DDD by the second labeling scheme

4.3 Vertex Ordering for s-Expanded DDDs

Vertex ordering in determinant decision diagrams crucially determines the size of the resulting DDDs. We therefore need to find a good vertex ordering for an s-expanded DDD as to minimize its size.

Because an s-expanded DDD is derived from an existing complex DDD, we can take the advantage of the vertex ordering, found by the heuristic described in Chapter III, in the complex DDD. We will show in Section 4.4 that the new vertex-ordering scheme is also vital to the efficient s-expanded-DDD construction algorithm.

Let a_i and a_j are two vertices in an s-expanded DDD; a and b are two vertices in the corresponding complex DDD such that $a_i \subset a$ and $b_j \subset b$. Let $L(x)$ denote the label of vertex x in a complex DDD and $L_s(x)$ denote the label of vertex x in an s-expanded DDD. The basic idea is to order the vertices such that if $L(a) > L(b)$ then $L_s(a_i) > L_s(b_j)$. Such an ordering scheme will ensure that the sharing in a complex DDD is still explored in the resulting s-expanded DDD. The relative order of the vertices contained by one complex DDD vertex can be arbitrarily selected since it makes no impact on the size of the resulting s-expanded DDDs as shown in Section 4.4.

In particular, we use a simple mapping function to compute the label of a vertex in an s-expanded DDD from the label of its containing vertex. We consider an admittance parameter to be represented by a vertex, a_i, in an s-expanded DDD and its containing vertex a. The mapping function for the first labeling scheme is defined as

$$L_s(a_i) = f_{map1}(a, a_i) = 3 * L(a) + f_{type}(a_i), \tag{4.4}$$

where

$$f_{type}(x) = \begin{cases} 0, & x \text{ is an admittance of resistance} \\ 1, & x \text{ is an admittance of capacitance} \\ 2, & x \text{ is an admittance of inductance} \end{cases} . \tag{4.5}$$

So we always have $f_{map1}(a, a_i) > f_{map1}(b, b_i)$, if $a > b$. On the other hand, once we know the label of a vertex in an s-expanded DDD, $L_s(a_i)$, the label of its containing vertex can be easily computed by $L(a_i)/3$ and its admittance type can be obtained by $L_s(a_i)\%3$, where % is the modulus operation.

The mapping function for the second labeling scheme is defined as

$$L_s(a_i) = f_{map2}(a, a_i) = k * L(a) + f_{index}(a_i), \tag{4.6}$$

where k is the maximum number of circuit admittance parameters in a complex

DDD vertex. Function $f_{index}(x)$ gives the unique index of a_i in a, and $f_{index}(x) \leq k$. Similarly, from the label of a vertex in an s-expanded DDD, one can easily compute the label of its containing vertex and its unique index in its containing vertex.

The s-expanded DDDs in Figure 18 and Figure 19 are constructed using f_{map1} and f_{map2}, respectively. In the following section, we show that with the new vertex-ordering scheme the size of the resulting s-expanded DDD constructed from a complex DDD is only proportional to the size of the complex DDD, the highest power of s in the s-expanded expression and k.

4.4 Construction of s-Expanded DDDs

4.4.1 The Construction Algorithm

An s-expanded DDD can be constructed from a complex DDD by one depth-first search of the complex DDD. The procedure is very efficient with the time complexity linear in the number of the resulting s-expanded DDD.

For convenience, we first present the construction algorithm using the first labeling scheme. Let D be a complex DDD vertex, with its 1-edge pointing to D_1 and its 0-edge pointing to D_0. Let $D.g$, $D.c$ and $D.l$ denote, respectively, the admittance of the conductance, capacitance and inductance in the circuit. An s-expanded DDD, P, is list of coefficient DDDs with $P[i]$ denoting the coefficient DDD of power s^i and $i \subset [-n, n]$. Then, we introduce the following four basic operations:

- CoeffUnion(P_1, P_2) computes the union of two s-expanded DDDs, P_1 and P_2.
- CoeffMulplty$(P, D.x)$ computes the product of s-expanded DDD P and coefficient DDD vertex $D.x$.
- $P * s$ increments the power of s in s-expanded DDD P.
- P/s decrements the powers of s in s-expanded DDD P.

Algorithm CoeffConst described in Figure 20 takes a complex DDD vertex and creates its corresponding coefficient DDDs. The implementations of CoeffU-

NION and COEFFMULPLTY are also shown in Figure 21 in terms of the basic DDD operations MULTIPLY and UNION, whose implementations can be found in the Figure 12 in Chapter III.

As in all other DDD operations [79], we cache the result of COEFFCONST(D), and in case D is encountered again, and its result will be used directly.

```
COEFFCONST(D)
1   if ( D = 0 or D = 1)
2       return NULL
3   L_0 = COEFFCONST(D_0)
4   L_1 = COEFFCONST(D_1)
5   if (D.g ≠ 0)
6       P_g = COEFFMULPLTY(L_1, D.g)
7   if (D.c ≠ 0)
8       P_c = COEFFMULPLTY(L_1 * s, D.c)
9       P_result = COEFFUNION(P_c, P_g)
10  if (D.l ≠ 0)
11      P_l = COEFFMULPLTY(L1/s, D.l)
12      P_result = COEFFUNION(P_l, P_result)
13  return COEFFUNION(P_result, L_0)
```

Figure 20: The s-expanded DDD construction with the first labeling scheme

In the second labeling scheme, we use $D.x_i$ to represent the ith admittance parameter in a complex DDD vertex D. $D.x_i$ can be a resistive admittance, a capacitive admittance or an inductive admittance. The function $type(D.x_i)$ will return res, cap and ind for the three admittance types, respectively. The COEFFCONST using the second labeling scheme is expressed in Figure 22.

```
CoeffUnion(P_1, P_2)
1   for i = −n to n do
2       P[i] = Union(P_1[i], P_2[i])
3   return P

CoeffMulplty(P, D.x)
1   for i = −n to n do
2       P[i] = Multiply(P[i], D.x)
3   return P
```

Figure 21: The basic s-expanded DDD construction algorithms

```
CoeffConst(D)
1   if ( D = 0 or D = 1)
2       return NULL
3   L_0 = CoeffConst(D_0)
4   L_1 = CoeffConst(D_1)
5   P_result = NULL
6   for i = 1 to k do
7       if (type(D.x_i) = res)
8           P_g = CoeffMulplty(L_1,D.x_i)
9           P_result = CoeffUnion(P_g, P_result)
10      if (type(D.x_i) = cap)
11          P_c = CoeffMulplty(L_1 * s,D.x_i)
12          P_result = CoeffUnion(P_c, P_result)
13      if (type(D.x_i) = ind)
14          P_l = CoeffMulplty(L_1/s,D.x_i)
15          P_result = CoeffUnion(P_l, P_result)
16  return CoeffUnion(P_result, L_0)
```

Figure 22: The s-expanded DDD construction with the second labeling scheme

4.4.2 Time and Space Complexity Analysis

Consider a $n \times n$ circuit matrix. A complex DDD D_r with its size denoted by $|D_r|$ is used to represent the determinant of the circuit matrix. Let n be the size of the determinant D_r represents. The maximum number of the circuit admittance parameters in an entry of a circuit matrix is k. Then, we have the following result for the s-expanded DDD derived from D_r by CoeffConst for both labeling schemes:

Theorem 3 *The time complexity of* CoeffConst(D_r) *and the number of vertices (size) of the resulting s-expanded DDD are* $O(kn|D_r|)$.

Proof. Function CoeffConst(D_r) performs a depth-first search on D_r, so it will visit each DDD vertex just once, and CoeffConst will be called just $|D_r|$ times.

At each vertex D in the complex DDD, we visit all the admittance parameters according to their unique indices (from the smallest one to the largest one). For each admittance parameter $D.x$, CoeffConst calls CoeffMulplty no more than once and CoeffUnion just once.

In function CoeffMulplty, we claim that the DDD operation Multiply($P[i]$, $D.x$) can complete in a constant time, where $D.x \subset D$. To see this, we note that $L_s(D.x)$ is always larger than the labels of any coefficient DDD vertices in L_1, as the labels of all the coefficient DDD vertices in L_1 are mapped from the complex DDDs whose labels are less than $L(D)$ according to the new vertex-ordering scheme. So the resulting coefficient DDD from Multiply($P[i], D.x$) can be readily constructed in a constant time as shown in Figure 23. Hence function CoeffMulplty takes $O(n)$ to finish for no more than $2n$ coefficient DDDs.

In the function CoeffUnion, we first consider its uses in steps 9 and 12 in Figure 20 and in steps 9, 12 and 15 in Figure 22. DDD operation Union($P1[i], P2[i]$) in CoeffUnion will also take a constant time to complete, where $P1[i]$ and $P2[i]$ are two coefficient DDDs. To see this, we take a close look at how $P1[i]$ and $P2[i]$

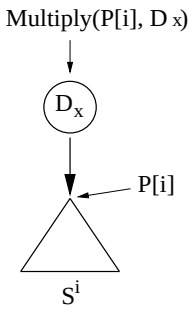


Figure 23: MULTIPLY($P[i], D.x$) operation

are constructed. We note that $P1[i]$ is the result of a MULTIPLY($L1[t]$, $D.x$) operation where $D.x$ is the root vertex in $P1[i]$, where $t = i$ if $D.x$ is an admittance of resistance; $t = i - 1$ if $D.x$ is an admittance of capacitance and $t = i + 1$ if $D.x$ is an admittance of inductance. We also observe that $P2[i]$ is the result of the previous UNION($P1[i], P2[i]$) operation. Because $L(D.x)$ is always larger than $L(D.x')$, where $D.x'$ is the admittance visited before $D.x$ in D. So $L(D.x)$ will be larger than the labels of all the vertices in $P2[i]$. As a result, the resulting coefficient DDD from UNION($P1[i], P2[i]$) can be readily constructed in a constant time as shown in Figure 24 for all admittance types. So function COEFFUNION also takes $O(n)$ to finish for no more than $2n$ coefficient DDDs.

Both COEFFMULPLTY and COEFFUNION will be called at most k times for each admittance parameter. So it will take $O(kn)$ to finish all the tasks before the last COEFFUNION call.

Finally we consider the calls of COEFFUNION(P_{result}, L_0) at line 13 in Figure 20 and at line 16 in Figure 22. We claim that this operation also takes $O(kn)$ to complete. This is can be verified by observing that the coefficient DDD of s^i from the result of the last COEFFUNION operation consists of four types of coefficient DDDs:

- The coefficient DDD from the multiplication of an admittance of resistance—

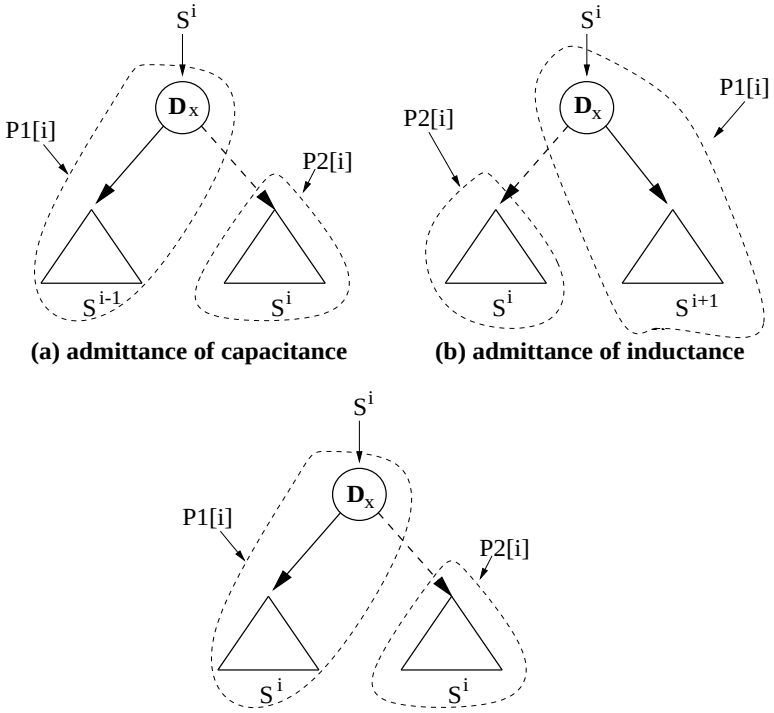


Figure 24: UNION($P1[i], P2[i]$) operation

$D.x$, and $L_1[i]$

- The coefficient DDD from the multiplication of an admittance of capacitance—$D.x$, and $L_1[i-1]$
- The coefficient DDD from the multiplication of an admittance of inductance—$D.x$, and $L_1[i+1]$
- $L_0[i]$

The final coefficient of s^i from the last CoeffUnion operation is the Union of all the coefficient DDDs of the four types. Note that the Union of the coefficient DDDs of the first three types is actually $L_{result}[i]$ in the Union($L_{result}[i]$, $L_0[i]$) operation. As a result, the 0-edge of the vertex, $D.x_s$, in $L_{result}[i]$ which has the smallest label among all the admittance parameters in D, must point to 0-terminal because $D.x_s$ is the first admittance parameter visited in D, and $P2[i]$ in fact is 0-terminal when operation Union($P1[i], P2[i]$) is carried out for $D.x_s$. With this, the Union operations in the last CoeffUnion operation can be readily performed by redirecting the 0-edge of $D.x_s$ to $L_0[i]$. Such an operation also takes a constant time. □

In summary, the total time complexity and also the final size of the s-expanded DDD is $O(kn|D_r|)$.

To conclude this section, we make the following observations:

1. From the previous analysis, one can easily see that a tighter bound of time complexity of function CoeffConst is the final size of the resulting s-expanded DDD. In addition, in the function CoeffMulplty and CoeffUnion, we only need to visit all the nonzero coefficient DDDs. So a more precise size bound of the resulting s-expanded DDD should be $O(km|D_r|)$, where m is the highest power of s in the s-expanded polynomial of the determinant which D_r represents.
2. By using the first label scheme, the time complexity of CoeffConst(D_r) and the size of the resulting s-expanded DDD is actually $O(n|D_r|)$, since $k = 3$ in

this case.

3. For a practical analog circuits, k actually is bounded by a constant and is independent of the size of the circuit.

Considering all the observations, we have the following conclusion:

Corollary 1 *Let m be the highest power of s in the s-expanded polynomial of the determinant of a circuit matrix. If the maximum number of the admittance parameters in each entry of the matrix is bounded by a constant, the time complexity of* CoeffConst(D_r) *and the size of the resulting s-expanded DDD is $O(m|D_r|)$.*

4.5 Experimental Results

We have implemented the proposed method in SCAD3and tested SCAD3 on a set of benchmark circuits. For each circuit, the MNA formulation is used, and the construction algorithm DDD_OF_Matrix described in Table 1 in Chapter III is applied to construct the complex DDD of the MNA matrices of the tested circuits. The results using the first labeling scheme are summarized in Table 5.

The statistics about the analog circuits used in this table can be found in Table 2. For each *circuit* in this table, *#numP* is the total number of product terms, i.e., 1-paths, in the numerator of the transfer function. *#demP* is the total numbers of product terms, i.e., 1-paths, in the denominator of the transfer function. $|DDD|$ is the size (number of vertices) of the DDD representing both the numerator and the denominator of the transfer function. The *CPU* time, which is given in seconds on an Ultra-SPARC-I workstation, is the time of constructing complex DDDs from circuit matrices or the time of constructing s-DDDs from complex DDDs.

From Table 5, we can see that s-expanded DDDs are able to represent a huge number of product terms with a relatively small number of vertices. Such a compact representation is achieved by exploring the sharing among different coefficients of polynomials for both the numerator and the denominator of the transfer function.

Table 5: Statistics on complex DDD and s-expanded DDD construction

circuit	Complex DDD				s-expanded DDD			
	#numP	#denP	$\|DDD\|$	CPU	#numP	#denP	$\|DDD\|$	CPU
miller	9	13	53	0.02	92	252	461	< 0.01
butter	1	13	24	0.01	1	239	122	< 0.01
ladder7	1	21	26	0.01	1	21	52	< 0.01
rlctest	80	120	152	0.04	140	444	220	0.01
ccstest	56	120	97	0.03	100	234	270	0.02
vcstest	55	64	70	0.04	80	372	122	< 0.01
ladder21	1	17711	82	0.05	1	123009	250	0.01
Cascode	42154	79643	2058	3.17	9.28×10^6	1.52×10^7	21384	0.88
$\mu A741$	10970	108032	6656	2.96	7.77×10^{10}	9.31×10^{10}	99846	5.08
bigtst	164168	1.6×10^7	951	0.89	1.91×10^8	6.67×10^{10}	14382	0.58
ladder100	1	5.7×10^{20}	398	1.14	1	5.7×10^{20}	796	0.18
rctree1	3560	7.1×10^7	208	0.21	3560	7.1×10^7	416	0.05
rctree2	1.4×10^5	3.0×10^{10}	299	92.5	1.4×10^5	3.0×10^{10}	598	0.09

Figures 25 and 26 show the distribution of the number of product terms over powers of s in the denominator of the transfer function of the μA741 Opamp by the first labeling scheme and by the second labeling scheme, respectively. The construction of DDDs takes only a few CPU seconds on a Ultra-SPARC-I workstation for all the examples of both cases. It is seen that the second labeling scheme will generate a far greater number of product terms (more than 9 order of magnitudes) than the first labeling scheme. In contrast, the number of DDD vertices used in both methods are 99846 and 297117 (more results using second labeling scheme can be found in the Chapter VII), for both numerators and denominators, respectively. The difference in DDD size is only one order of magnitude. This clearly shows the power of DDDs in the symbolic representation.

We implemented the results in our symbolic analyzer SCAD3. Table 6 describes

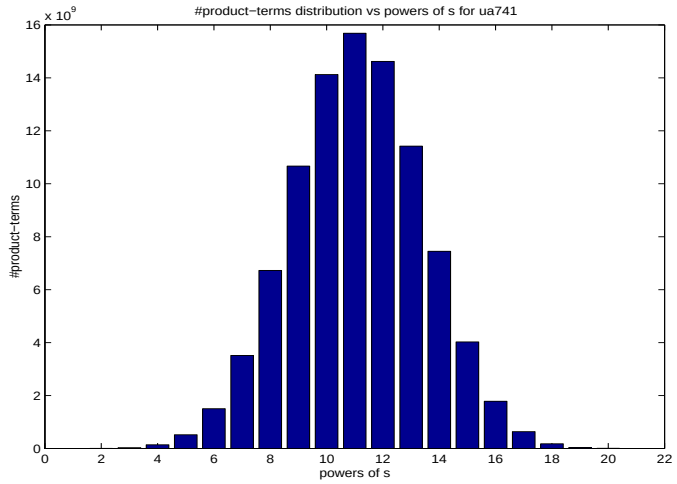


Figure 25: Product term distribution of μA741 by the first labeling scheme

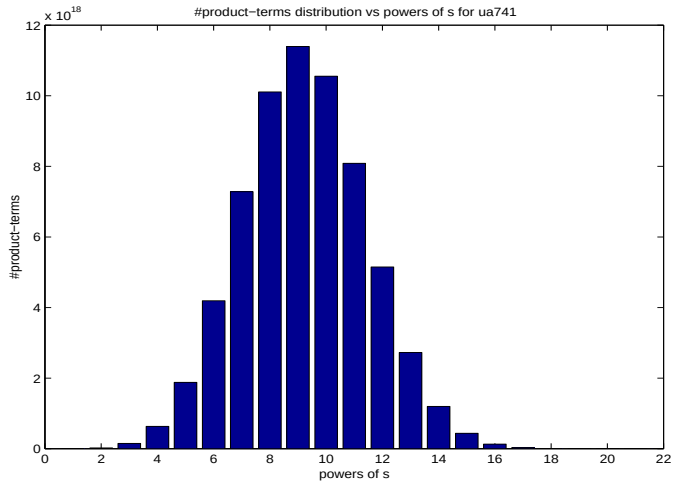


Figure 26: Product term distribution of μA741 by the second labeling scheme

the CPU time used, where *s-expanded_Eval* is the time used for calculating the numerical values of a coefficient DDD in the resulting s-expanded DDDs, *s-expanded DDD* is the CPU time used for frequency-domain simulation (numerical evaluation) using the calculated s-expanded expressions over 1000 frequency points. We observe that an order-of-magnitude speedup has been achieved, and further that the speedup approximately increases with the size of the circuit for practical analog circuits.

Table 6: Comparison against SPICE in numerical evaluation

circuit	s-expanded_Eval	s-expanded DDD	Spice	speedup
miller	< 0.01	0.02	0.32	16.0
butter	< 0.01	0.03	0.54	18.0
ladder7	0.01	0.01	0.21	21.0
rlctest	0.01	0.03	0.44	14.7
ccstest	0.01	0.02	0.62	31.0
vcstest	< 0.01	0.02	0.52	26.0
ladder21	0.01	0.03	0.51	16.7
Cascode	0.52	0.03	1.16	38.6
$\mu A741$	3.32	0.05	2.40	48.0
bigtst	0.58	0.04	1.94	48.5
ladder100	0.17	0.11	3.10	28.2
rctree1	0.05	0.05	0.72	14.4
rctree2	0.08	0.07	0.94	13.4

4.6 Summary

In this chapter, we presented a new approach to derive the s-expanded symbolic expressions using the DDD-graph manipulations. The key idea is the introduction of a multi-rooted DDD structure, called s-expanded DDD, to represent the expanded symbolic expressions with each rooted DDD representing a coefficient of a particular

power of s.

Experimental results indicate that the new method can produce the exact s-expanded symbolic transfer function for analog circuits as large as μA741 Opamps with 26 transistors in several CPU seconds. The effect of s-expanded DDDs is so remarkable that, for the first time, 10^{19} product terms in a real analog circuit transfer function have been successfully represented with less than 3×10^5 vertices.

We also showed that the use of the generated symbolic expression achieves an order-of-magnitude speedup over SPICE3f5 in frequency-domain small-signal AC simulation. The proposed method, therefore, has substantially extended the capabilities of the exact symbolic analysis. The introduction of s-expanded DDDs has also paved the way for other important symbolic analysis techniques, such as approximation and circuit-level noise modeling, which will be described in the following chapters.

CHAPTER V HIERARCHICAL SYMBOLIC ANALYSIS OF ANALOG CIRCUITS

5.1 Introduction

Determinant decision diagrams and s-expanded DDDs have substantially increased the generatability of exact symbolic analysis for large analog circuits. In previous chapters, we have proved that DDD-based symbolic analysis yields optimal representations for ladder circuits. Our experimental results have shown that integrated analog circuits, with 25-30 BJT or MOS transistors and 25-30 nodes, can be exactly analyzed by our method in a reasonable amount of CPU time. The complexities of many integrated analog building blocks such as operational amplifiers and mixers fall into this size category.

In this chapter, we investigate how the circuit hierarchy and structural regularity can be explored by *hierarchical decomposition* to handle much larger analog circuits. Existing hierarchical decomposition methods include two types: topological analysis [70] and network formulation [39]. The resultant expressions, however, are not compact enough, and the number of expressions can grow so rapidly that even the compilation of the generated expressions could take enormous time. Manipulations (other than evaluation) of the resulting sequences of expressions are known to be complicated and they often require dedicated efforts, e.g., sensitivity calculation in [52] and lazy approximations in [69].

This chapter presents a new approach to exact hierarchical symbolic analysis. The new method takes advantage of both hierarchical decomposition and determinant decision diagrams introduced in previous chapters (see also [79]).

The rest of the chapter is organized as follows: Section 5.2 gives an algorithmic overview of the hierarchical circuit analysis procedure. Section 5.3 presents the basic idea of our DDD-based hierarchical decomposition scheme. Section 5.4 illustrates the proposed method through a practical analog circuit. Section 5.5 summarizes the complete algorithm of DDD-based hierarchical symbolic analysis. Section 5.6 presents some experimental results along with the comparison with symbolic analyzer SCAPP and numerical simulator SPICE on practical analog circuits. A brief summary is given in Section 5.7.

5.2 Overview of Hierarchical Circuit Analysis

The circuit equations of a linear(ized), time-invariant analog circuit can be formulated by the modified nodal analysis (MNA) approach, in the following general form [86]:

$$\mathbf{Ax} = \mathbf{b}, \tag{5.1}$$

where $\mathbf{x}$ is the vector of node-voltage and branch-current variables, $\mathbf{A}$ is the modified nodal admittance matrix or simply the *circuit matrix*, and $\mathbf{b}$ represents the external sources.

The circuit hierarchy can be viewed as a rooted tree shown in Figure 27. A circuit may have one or more subcircuits at each hierarchical level. A subcircuit at a leaf in the circuit hierarchy tree is called a *leaf* circuit; otherwise it is called a *middle* circuit. In this chapter, we assume the presence of the predefined *subcircuits* in the circuit hierarchy.

Consider a subcircuit with some internal structures and terminals, as illustrated in Figure 28. The circuit unknowns—the node-voltage variables and branch-current variables—can be partitioned into three disjoint groups $\mathbf{x}^I$, $\mathbf{x}^B$, and $\mathbf{x}^R$, where the superscripts *I, B, R* stand for, respectively, *internal* variables, *boundary* variables and the *rest* of variables. *Internal* variables are those local to the subcircuit, *boundary*

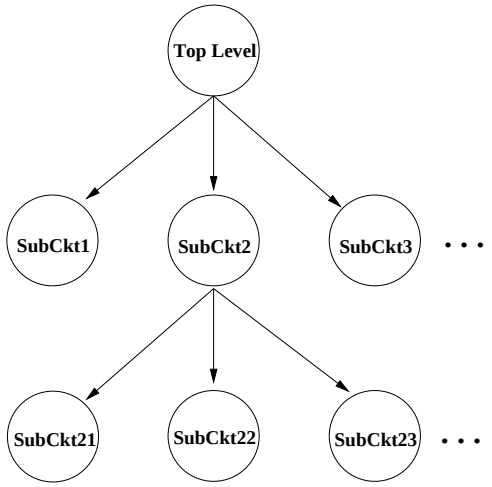


Figure 27: Model of a circuit hierarchy

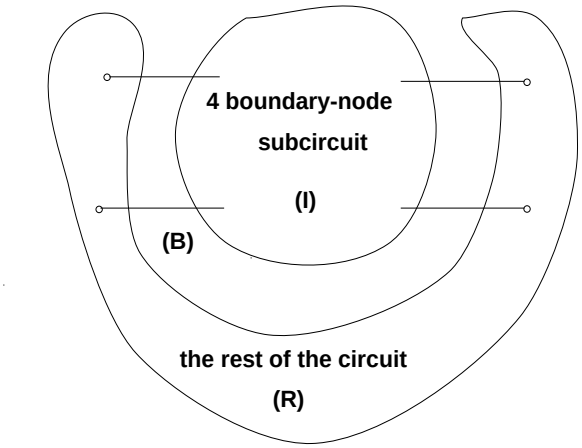


Figure 28: A partitioned circuit

variables (also called *tearing variables*) are those related to both the subcircuit and the rest of the circuit. Note that boundary variables include those variables required as the circuit inputs and outputs. Equations associated with only the *internal* variables are called the *internal* equations of a subcircuit. Their corresponding circuit matrix is called the *internal* circuit matrix. With this, the system-equation set (5.1) can be

rewritten in the following form:

$$\begin{bmatrix} \mathbf{A}^{II} & \mathbf{A}^{IB} & 0 \\ \mathbf{A}^{BI} & \mathbf{A}^{BB} & \mathbf{A}^{BR} \\ 0 & \mathbf{A}^{RB} & \mathbf{A}^{RR} \end{bmatrix} \begin{bmatrix} \mathbf{x}^{I} \\ \mathbf{x}^{B} \\ \mathbf{x}^{R} \end{bmatrix} = \begin{bmatrix} \mathbf{b}^{I} \\ \mathbf{b}^{B} \\ \mathbf{b}^{R} \end{bmatrix}. \tag{5.2}$$

The gray matrix, $\mathbf{A}^{II}$, is the *internal* matrix associated with internal variable vector $\mathbf{x}^I$. The basic idea underlining most hierarchical analysis methods is to suppress the number of equations and the number of variables until a set of equations involving only the desired variables remains. The circuit interpretation of such a reduction may be viewed to represent a subcircuit by an equivalent circuit at its terminals. We call such a procedure *subcircuit suppression*.

Let us illustrate subcircuit suppression by suppressing all the variables in $\mathbf{x}^I$. To this end, we multiply the following nonsingular matrix to both sides of the equation set (5.2), and obtain

$$\begin{bmatrix} \mathbf{I}^{II} & 0 & 0 \\ -\mathbf{A}^{BI}(\mathbf{A}^{II})^{-1} & \mathbf{I}^{BB} & 0 \\ 0 & 0 & \mathbf{I}^{RR} \end{bmatrix} \begin{bmatrix} \mathbf{A}^{II} & \mathbf{A}^{IB} & 0 \\ \mathbf{A}^{BI} & \mathbf{A}^{BB} & \mathbf{A}^{BR} \\ 0 & \mathbf{A}^{RB} & \mathbf{A}^{RR} \end{bmatrix} =$$

$$\begin{bmatrix} \mathbf{A}^{II} & \mathbf{A}^{IB} & 0 \\ 0 & \mathbf{A}^{BB} - \mathbf{A}^{BI}(\mathbf{A}^{II})^{-1}\mathbf{A}^{IB} & \mathbf{A}^{BR} \\ 0 & \mathbf{A}^{RB} & \mathbf{A}^{RR} \end{bmatrix} \tag{5.3}$$

for the left-hand side, and

$$\begin{bmatrix} \mathbf{I}^{II} & 0 & 0 \\ -\mathbf{A}^{BI}(\mathbf{A}^{II})^{-1} & \mathbf{I}^{BB} & 0 \\ 0 & 0 & \mathbf{I}^{RR} \end{bmatrix} \begin{bmatrix} \mathbf{b}^{I} \\ \mathbf{b}^{B} \\ \mathbf{b}^{R} \end{bmatrix} = \begin{bmatrix} \mathbf{b}^{I} \\ \mathbf{b}^{B} - \mathbf{A}^{BI}(\mathbf{A}^{II})^{-1}\mathbf{b}^{I} \\ \mathbf{b}^{R} \end{bmatrix} \tag{5.4}$$

for the right-hand side, where $\mathbf{I}^{II}$, $\mathbf{I}^{BB}$ and $\mathbf{I}^{RR}$ are identity matrices having the same sizes of $\mathbf{A}^{II}$, $\mathbf{A}^{BB}$ and $\mathbf{A}^{RR}$, respectively. Note that using the transformed matrix in the right-hand side of (5.3), we can solve for $\mathbf{x}^B$ and $\mathbf{x}^R$ without the knowledge of $\mathbf{x}^I$. Hence the transformation essentially simplifies the equation set (5.2) by suppressing

all the variables in $\mathbf{x}^I$. The suppressed equation set can be written as follows:

$$\begin{bmatrix} \mathbf{A}^{BB*} & \mathbf{A}^{BR} \\ \mathbf{A}^{RB} & \mathbf{A}^{RR} \end{bmatrix} \begin{bmatrix} \mathbf{x}^B \\ \mathbf{x}^R \end{bmatrix} = \begin{bmatrix} \mathbf{b}^{B*} \\ \mathbf{b}^R \end{bmatrix}, \tag{5.5}$$

where

$$\mathbf{A}^{BB*} = \mathbf{A}^{BB} - \mathbf{A}^{BI}(\mathbf{A}^{II})^{-1}\mathbf{A}^{IB}, \tag{5.6}$$

and

$$\mathbf{b}^{B*} = \mathbf{b}^B - \mathbf{A}^{BI}(\mathbf{A}^{II})^{-1}\mathbf{b}^I. \tag{5.7}$$

Subcircuit suppression can be performed for all the subcircuits by visiting the circuit hierarchy in a bottom-up fashion. Hierarchical *numerical* analysis performs subcircuit suppression numerically by partial LU decomposition [87]. Hierarchical *symbolic* analysis introduces intermediate variables to represent subcircuit suppression (5.6) and (5.7) by using a *sequence of expressions* [39, 70]. In the network formulation approach [39], the internal variables are suppressed one at a time, so $\mathbf{A}^{II}$ becomes a scalar – a_{ii} and Equation (5.6) becomes

$$\mathbf{A}^{BB*} = \mathbf{A}^{BB} - \frac{1}{a_{ii}}\mathbf{A}^{Bi}\mathbf{A}^{iB}, \tag{5.8}$$

where $\mathbf{A}^{Bi}$ is the column of $\mathbf{A}^{BI}$ and $\mathbf{A}^{iB}$ is the row of and $\mathbf{A}^{IB}$, and a_{ii} is the intersection of the row and the column.

5.3 Hierarchical Analysis Using Determinants and Cofactors

In this section, we show how hierarchical symbolic circuit analysis can be represented using determinants and cofactors. We first introduce some notations. Let $\mathbf{A}$ be an $n \times n$ matrix. It may be denoted as $[a_{u,v}], u, v = 1, ..., n$. Similarly, a vector $\mathbf{b}$ is denoted as $[b_u], u = 1, ..., n$. The *determinant* of matrix $\mathbf{A}$ is denoted by $det(\mathbf{A})$. According to linear algebra, the inverse of non-singular matrix $\mathbf{A}$ can be written as

$$\mathbf{A}^{-1} = \frac{1}{det(\mathbf{A})}[\Delta_{u,v}]^T, \tag{5.9}$$

where matrix $[\Delta_{u,v}]^T$ is the transpose of matrix $[\Delta_{u,v}]$, and

$$\Delta_{u,v} = (-1)^{u+v} det(\mathbf{A}_{a_{u,v}}). \tag{5.10}$$

Here $[\Delta_{u,v}]^T$ is called the *adjoint* matrix of $\mathbf{A}$, $\Delta_{u,v}$ is the first-order *cofactor* of $det(\mathbf{A})$ with respect to $a_{u,v}$, and matrix $\mathbf{A}_{a_{u,v}}$ is the $(n-1) \times (n-1)$-matrix obtained from matrix $\mathbf{A}$ by deleting row u and column v. Matrix $\mathbf{A}_{a_{u,v}}$ is sometimes written as $\mathbf{A}_{u,v}$ in the sequel. Note that each entry in the adjoint matrix is a first-order cofactor of the original matrix, and the adjoint matrix itself is a *full* (dense) matrix.

Now we consider subcircuit suppression. Substituting (5.9) to (5.6) and (5.7), we have

$$\mathbf{A}^{BB*} = \mathbf{A}^{BB} - \frac{1}{det(\mathbf{A}^{II})} \mathbf{A}^{BI} [\Delta^{II}_{u,v}]^T \mathbf{A}^{IB}, \tag{5.11}$$

and

$$\mathbf{b}^{B*} = \mathbf{b}^{B} - \frac{1}{det(\mathbf{A}^{II})} \mathbf{A}^{BI} [\Delta^{II}_{u,v}]^T \mathbf{b}^{I}. \tag{5.12}$$

Suppose that the number of internal variables is m, and the number of boundary variables is l. Equations (5.11) and (5.12) can be written in the following expanded forms:

$$a^{BB*}_{u,v} = a^{BB}_{u,v} - \frac{1}{det(\mathbf{A}^{II})} \sum_{k_1,k_2=1}^{m} a^{BI}_{u,k_1} \Delta^{II}_{k_2,k_1} a^{IB}_{k_2,v}, u, v = 1, ..., l, \tag{5.13}$$

and

$$b^{B*}_{u} = b^{B}_{u} - \frac{1}{det(\mathbf{A}^{II})} \sum_{k_1,k_2=1}^{m} a^{BI}_{u,k_1} \Delta^{II}_{k_2,k_1} b^{I}_{k_2}, u, = 1, ..., l. \tag{5.14}$$

Note that we need first-order cofactors $\Delta^{II}_{k_2,k_1}$ only when both a_{u,k_1} and $a_{k_2,v}$ are nonzeros, and for practical circuits l usually is much smaller than m, provided a good circuit partition is given, and $\mathbf{A}^{BI}$ and $\mathbf{A}^{IB}$ are generally very *sparse*. This implies that only *a very few* of first-order cofactors of $\mathbf{A}^{II}$ are needed for subcircuit suppression.

At the top level of the circuit hierarchy tree, we can simply use Cramer's rule to obtain the desired transfer function. Suppose that the reduced equation set for the

root circuit is $\mathbf{A}'\mathbf{x}' = \mathbf{b}'$, then the voltage gain from node i to node t can be expressed as follows:

$$H(s)_{it} = \frac{V_t(s)}{V_i(s)} = \frac{(-1)^{i+t} det(\mathbf{A}'_{it})}{(-1)^{i+i} det(\mathbf{A}'_{ii})}, \tag{5.15}$$

where $\mathbf{A}'_{uv}$ is the submatrix obtained from $\mathbf{A}^{'}$ by deleting row u and column v.

The key idea of the proposed method for hierarchical symbolic analysis is to represent all the determinants and cofactors in (5.13) (5.14), and (5.15) by determinant decision diagrams. DDDs allow the exploiting of the sharing among the determinants of subcircuit matrices and their first-order cofactors in a systematic manner. The exploration thereby leads to a very compact representation of transfer functions and renders DDDs extremely suitable for hierarchical symbolic analysis.

5.4 An Illustration Example

In this section, we illustrate the proposed hierarchical analysis method through a real analog circuit. We consider a Cauer parameter low-pass filter with 0.02-dB ripple in the passband and minimum 50dB suppress in stopband as shown in Figure 29. It has four topologically identical FDNR subcircuits, named $X1$ to $X4$. Figure 30 gives the detailed structure of the FDNR subcircuit. A FDNR subcircuit contains two operational amplifiers (Opamp), and each of them is implemented by a linear model of $\mu A741$ shown in Figure 31.

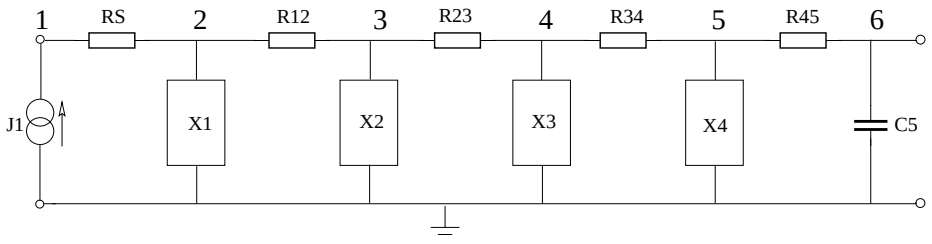


Figure 29: An active low-pass filter

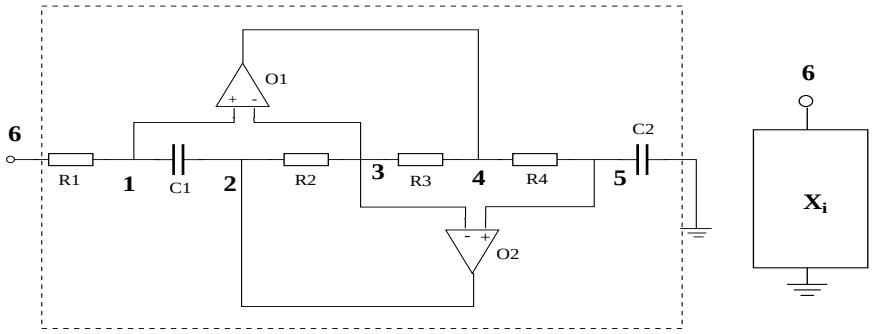


Figure 30: An FDNR subcircuit.

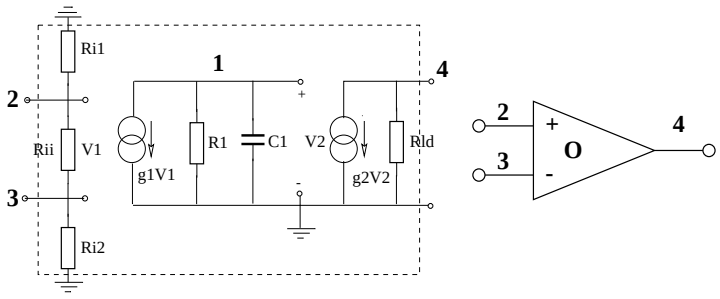


Figure 31: A linear model of $\mu A741$ Opamp

We first consider the linear model of $\mu 741$ Opamp in Figure 31. There are 4 nodes in this circuit, and thus 4 variables, $v_i, i = 1, ..., 4$, in the circuit equations. Except for the v_1, all the other variables are boundary variables. Its circuit matrix, $\mathbf{T}^o$, under the modified nodal analysis formulation, can be written as follows:

$$\mathbf{T}^o = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{array} \overbrace{\begin{bmatrix} sC_1 + \frac{1}{R_1} & g_1 & -g_1 & 0 \\ 0 & \frac{1}{R_{ii}} + \frac{1}{R_{i1}} & -\frac{1}{R_{ii}} & 0 \\ 0 & -\frac{1}{R_{ii}} & \frac{1}{R_{ii}} + \frac{1}{R_{i2}} & 0 \\ g_2 & 0 & 0 & -\frac{1}{R_{ld}} \end{bmatrix}}^{(\mathbf{T}^o)^{II}} . \tag{5.16}$$

The gray variable is the internal variable to be suppressed and the gray matrix, $(\mathbf{T}^o)^{II}$,

in the $\mathbf{T}^o$ is the internal circuit matrix of the linear model circuit, i.e., $(\mathbf{T}^o)^{II} = \left[sC_1 + \frac{1}{R_1}\right]$.

To suppress the linear model circuit, we need to suppress variable v_1. According to (5.13), the suppressed circuit matrix associated with only the boundary variables is given by:

$$(\mathbf{T}^o)^{BB*} = \begin{matrix} v_2 \\ v_3 \\ v_4 \end{matrix} \begin{bmatrix} \frac{1}{R_{ii}} + \frac{1}{R_{i1}} & -\frac{1}{R_{ii}} & 0 \\ -\frac{1}{R_{ii}} & \frac{1}{R_{ii}} + \frac{1}{R_{i2}} & 0 \\ \underbrace{(t_{42}^o)^{BB*}}_{\text{fill-ins}} & \underbrace{(t_{43}^o)^{BB*}}_{\text{fill-ins}} & -\frac{1}{R_{ld}} \end{bmatrix}, \tag{5.17}$$

where

$$(t_{42}^o)^{BB*} = -\frac{1}{det(\mathbf{T}^{io})} g_2 (\Delta_{11}^o)^{BB*} g_1 = -\frac{g_2 g_1}{sC_1 + \frac{1}{R_1}}, \tag{5.18}$$

$$(t_{43}^o)^{BB*} = -\frac{1}{det(\mathbf{T}^{io})} g_2 (\Delta_{11}^o)^{BB*} (-g_1) = \frac{g_2 g_1}{sC_1 + \frac{1}{R_1}}, \tag{5.19}$$

where $(\Delta_{11}^o)^{BB*}$ is the first order cofactor of $(\mathbf{T}^o)^{II}$. Note that

$$(\Delta_{11}^o)^{BB*} = (-1)^{(1+1)} det((\mathbf{T}_{11}^o)^{II}) = 1, \tag{5.20}$$

where $(\mathbf{T}_{11}^o)^{II} = 1$. The suppression of the linear model of μ741 Opamp circuit only calls for the DDD representation of two determinants: $det((\mathbf{T}^o)^{BB*})$ and $det((\mathbf{T}_{11}^o)^{BB*})$. Note that $det((\mathbf{T}_{11}^o)^{BB*})$ is constant 1. As a result, only one DDD vertex is needed; the resulting DDD is shown in Figure 32 with $a = sC_1 + \frac{1}{R_1}$.

After the suppression of the linear model of μ741 Opamp circuit, we are ready to consider its parent circuit, the FDNR circuit. Its circuit matrix can be constructed by combining the matrix in (5.17) for the linear Opamp circuit with the contributions from resistors in the FDNR circuit. The resulting circuit matrix for FDNR subcircuit

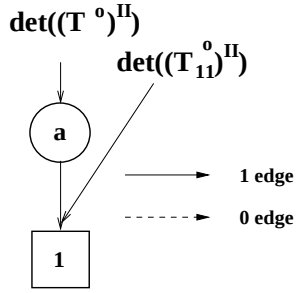


Figure 32: The DDD for $det((\mathbf{T}^o)^{II})$ and $det((\mathbf{T}^o_{11})^{II})$

can be written as follows:

$$\mathbf{T}^x = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{array} \overbrace{\begin{bmatrix} p^{l2}_{11} & -sC_1 & \frac{1}{-R_{ii}} & 0 & 0 & -\frac{1}{R_1} \\ -sC_1 & p^{l2}_{22} & -\frac{1}{R_2} + (t^o_{43})^{BB*} & 0 & (t^o_{42})^{BB*} & 0 \\ -\frac{1}{R_{ii}} & -\frac{1}{R_2} & p^{l2}_{33} & -\frac{1}{R_3} & -\frac{1}{R_{ii}} & 0 \\ (t^o_{42})^{BB*} & 0 & -\frac{1}{R_3} + (t^o_{43})^{BB*} & p^{l2}_{44} & -\frac{1}{R_4} & 0 \\ 0 & 0 & -\frac{1}{R_{ii}} & -\frac{1}{R_4} & p^{l2}_{55} & 0 \\ -\frac{1}{R_1} & 0 & 0 & 0 & 0 & p^{l2}_{66} \end{bmatrix}}^{(\mathbf{T}^x)^{II}} , \tag{5.21}$$

where

$$\begin{aligned} p^{l2}_{11} &= \frac{1}{R_1} + sC_1 + \frac{1}{R_{ii}} + \frac{1}{R_{i1}}, \\ p^{l2}_{22} &= sC_1 + \frac{1}{R_2} + \frac{1}{R_{ld}}, \\ p^{l2}_{33} &= \frac{1}{R_2} + \frac{1}{R_3} + \frac{2}{R_{ii}} + \frac{2}{R_{i2}}, \\ p^{l2}_{44} &= \frac{1}{R_3} + \frac{1}{R_4} + \frac{1}{R_{ld}}, \\ p^{l2}_{55} &= \frac{1}{R_4} + sC_2 + \frac{1}{R_{ii}} + \frac{1}{R_{i1}}, \\ p^{l2}_{66} &= \frac{1}{R_1}. \end{aligned}$$

Again, the gray variables are internal variables to be suppressed, and the gray matrix,

$(\mathbf{T}^x)^{II}$, in $\mathbf{T}^x$ is the internal circuit matrix of the FDNR subcircuit. The suppressed circuit matrix of $\mathbf{T}^x$, which is associated with only the boundary variable v_6, becomes a 1×1 matrix as follows:

$$(\mathbf{T}^x)^{BB*} = \left[\ p_{66}^{l2} + (t_{66}^x)^{BB*}\ \right], \tag{5.22}$$

where

$$(t_{66}^x)^{BB*} = -\frac{1}{det((\mathbf{T}^x)^{II})}(-\frac{1}{R_1})(\Delta_{11}^x)^{II}(-\frac{1}{R_1}). \tag{5.23}$$

Here $(\Delta_{11}^x)^{II}$ is the first order cofactor of $(\mathbf{T}^x)^{II}$ with respect to the element in row 1 and column 1 of $(\mathbf{T}^x)^{II}$, and it can be written as:

$$(\Delta_{11}^x)^{II} = (-1)^{(1+1)} det((\mathbf{T}_{11}^x)^{II}) = \begin{vmatrix} p_{22}^{l2} & -\frac{1}{R_2} + (t_{43}^o)^{BB*} & 0 & (t_{42}^o)^{BB*} \\ -\frac{1}{R_2} & p_{33}^{l2} & -\frac{1}{R_3} & -\frac{1}{R_{ii}} \\ 0 & -\frac{1}{R_3} + (t_{43}^o)^{BB*} & p_{44}^{l2} & -\frac{1}{R_4} \\ 0 & -\frac{1}{R_{ii}} & -\frac{1}{R_4} & p_{55}^{l2} \end{vmatrix}.$$

We now show how the two required determinants, $det((\mathbf{T}^x)^{II})$ and $det((\mathbf{T}_{11}^x)^{II})$, can be represented by DDDs. To simplify our presentation, we label each matrix entry in $(\mathbf{T}^x)^{II}$ with a distinct symbol as shown below:

$$det((\mathbf{T}^x)^{II}) = \begin{vmatrix} a & b & c & 0 & 0 \\ d & e & f & 0 & g \\ h & i & j & k & l \\ m & 0 & n & o & p \\ 0 & 0 & q & r & s \end{vmatrix},$$

where, the gray area is matrix $(\mathbf{T}_{11}^x)^{II}$. Determinant $det((\mathbf{T}^x)^{II})$ has 31 product terms, and $det((\mathbf{T}_{11}^x)^{II})$ has 10 product terms. Detailed analysis shows that they can be represented by two DDDs with 37 and 19 vertices, respectively. Exploiting the sharing of product terms among these two DDDs, the two determinants can actually be represented by a shared DDD with only 39 vertices as shown in Figure 33. This

result contrasts with 247 (= 37×5+19×4) DDD vertices if each symbol is represented by a DDD vertex without exploiting the sharing.

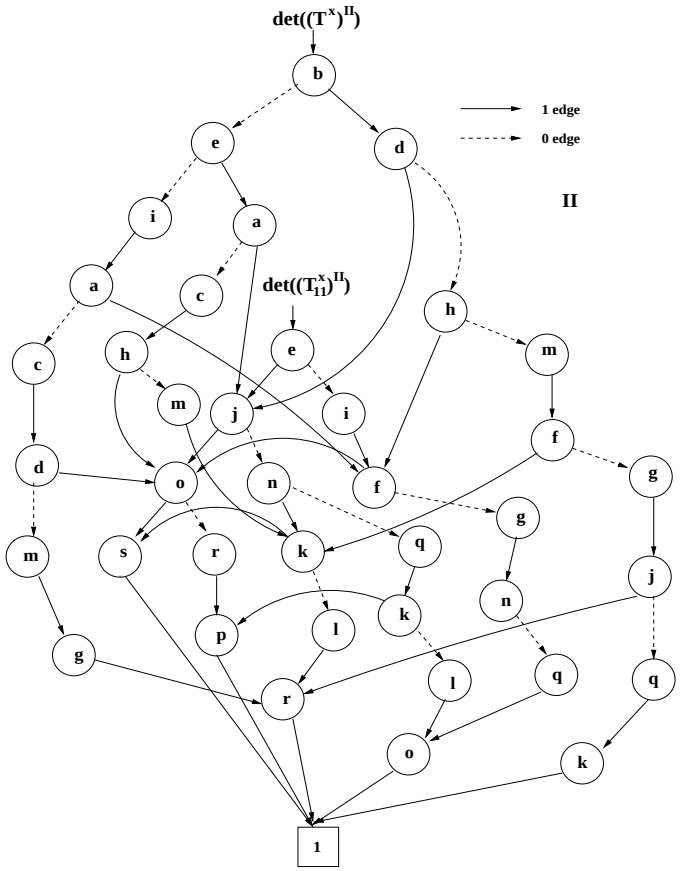


Figure 33: The DDD for $det((\mathbf{T}^x)^{II})$ and $det((\mathbf{T}^x_{11})^{II})$

Finally, the circuit matrix of the root circuit, the low-pass filter, is constructed after the suppression of its subcircuits, the FDNR circuits. The resulting circuit

matrix is given below:

$$\mathbf{T} = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{array} \begin{array}{c} \begin{array}{cccccc} \quad & \quad & \quad & \quad & \quad & \quad \end{array} \\ \begin{bmatrix} p_{11}^{l1} & -\frac{1}{R_s} & 0 & 0 & 0 & 0 \\ -\frac{1}{R_s} & p_{22}^{l1} & -\frac{1}{R_{12}} & 0 & 0 & 0 \\ 0 & -\frac{1}{R_{12}} & p_{33}^{l1} & -\frac{1}{R_{23}} & 0 & 0 \\ 0 & 0 & -\frac{1}{R_{23}} & p_{44}^{l1} & -\frac{1}{R_{34}} & 0 \\ 0 & 0 & 0 & -\frac{1}{R_{34}} & p_{55}^{l1} & -\frac{1}{R_{45}} \\ 0 & 0 & 0 & 0 & -\frac{1}{R_{45}} & p_{66}^{l1} \end{bmatrix} \end{array}, \tag{5.24}$$

where

$$\begin{aligned} p_{11}^{l1} &= \frac{1}{R_s}, \\ p_{22}^{l1} &= \frac{1}{R_s} + \frac{1}{R_{12}} + \frac{1}{R_1} + (t_{66}^x)_1^{BB*}, \\ p_{33}^{l1} &= \frac{1}{R_{12}} + \frac{1}{R_{23}} + \frac{1}{R_1} + (t_{66}^x)_2^{BB*}, \\ p_{44}^{l1} &= \frac{1}{R_{23}} + \frac{1}{R_{34}} + \frac{1}{R_1} + (t_{66}^x)_3^{BB*}, \\ p_{55}^{l1} &= \frac{1}{R_{34}} + \frac{1}{R_{45}} + \frac{1}{R_1} + (t_{66}^x)_4^{BB*}, \\ p_{66}^{l1} &= \frac{1}{R_{45}} + sC_5. \end{aligned}$$

Note that each FDNR subcircuit may have a difference set of device parameter values. Thus $(t_{66}^x)_i^{BB*}, i = 1, ..., 4$, are used to represent $(t_{66}^x)^{BB*}$ for four FDNR instances. Since all the FDNR subcircuits have the exact same topology, only one symbolic expression of $(t_{66}^x)^{BB*}$, and thus one DDD (Figure 8) is needed.

The required transfer function can now be derived and represented by DDDs. According to Cramer's rule, the voltage gain from V_1 to V_6 in Figure 29 can be expressed as:

$$G(s)_{16} = \frac{V_6}{V_1} = \frac{(-1)^{1+6} det(\mathbf{T}_{16})}{(-1)^{1+1} det(\mathbf{T}_{11})}.$$

To illustrate the DDD representation of the transfer function, we label each matrix

entry in $\mathbf{T}$ with a distinct symbol and rewrite $\mathbf{T}$ as follows:

$$det(\mathbf{T}) = \begin{vmatrix} a & b & 0 & 0 & 0 & 0 \\ c & d & e & 0 & 0 & 0 \\ 0 & f & g & h & 0 & 0 \\ 0 & 0 & i & j & k & 0 \\ 0 & 0 & 0 & l & m & n \\ 0 & 0 & 0 & 0 & o & p \end{vmatrix},$$

where the boxed matrix is $\mathbf{T}_{16}$ and the gray matrix is $\mathbf{T}_{11}$. Note that $\mathbf{T}_{11}$ is a 5×5 band matrix. The DDD representation of $det(\mathbf{T}_{11})$ and $det(\mathbf{T}_{16})$ is shown in Figure 34, where 13 DDD vertices are used to represent $det(\mathbf{T}_{11})$, and 5 DDD vertices (each for a matrix entry) are needed to represent $det(\mathbf{T}_{16})$. Counting the DDDs for subcircuit suppression (Figures 7 and 8), a total of 58 $(= 1 + 29 + 18)$ DDD vertices are used for entire hierarchical symbolic analysis of the low-pass filter circuit.

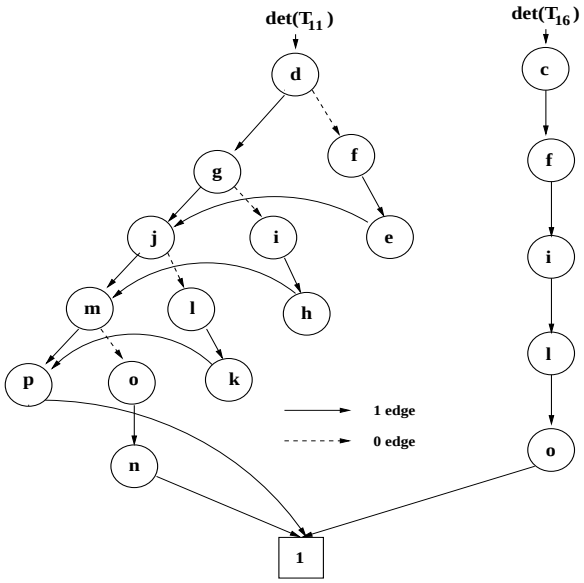


Figure 34: The DDD for $det(\mathbf{T}_{11})$ and $det(\mathbf{T}_{16})$

To conclude this section, we have the following observations:

- Suppression of a subcircuit may create fill-ins in its parent circuit matrix. For instance, $(t^o_{42})^{BB*}$ appearing at row 2 and column 5, as well as row 4 and column 1, in matrix $(\mathbf{T}^x)^{II}$ in (22) are fill-ins. In order to obtain a compact symbolic expression, the number of fill-ins should be minimized. Such a requirement essentially calls for minimizing the total number of the *boundary* variables of subcircuits. Finding the optimal partitions for DDD-based hierarchical analysis is addressed in [84].
- The new method is applicable to arbitrary tree hierarchies of a circuit, and is thus superior to the topological method [70], in which symbolic analysis must be performed from the flatten circuits, and to the network formulation approach [39], in which only the binary tree hierarchy is explored.
- Given a good partition of an analog circuit, subcircuit suppression only requires a few first-order cofactors of the subcircuit-matrix determinants. In our example, only one cofactor, $(t^x_{66})^{BB*}$, is required in the entire analysis process.

5.5 Hierarchical Symbolic Analysis Procedure

The proposed symbolic analysis method is performed by the depth-first traversal of the circuit hierarchy tree shown in Figure 27. The complete algorithm can be summarized as follows:

1. Partition the circuit or use the predefined subcircuit structure.
2. Build the circuit matrix for each leaf subcircuit by modified nodal analysis. Suppress each leaf subcircuit by (5.13) and (5.14). Represent the determinant of the internal subcircuit matrix and the first-order cofactors required in subcircuit suppression by a shared DDD.
3. Build the circuit matrix of a middle circuit after the suppression of all its children subcircuits. The entries of the middle circuit matrix consist of the contribution from children circuits in addition to those from the circuit devices in

the middle circuit. Suppress the middle circuit by (5.13) and (5.14). Represent the determinant of the internal subcircuit matrix and the first-order cofactors required in subcircuit suppression by a shared DDD. Recursively build and suppress the circuit matrices for all the middle circuits until the root circuit is reached.

4. Construct the desired symbolic transfer function at the root circuit by using Cramer's rule with all the required symbolic determinants and cofactors represented by DDDs.

5.6 Experimental Results

The proposed method has been implemented in symbolic analyzer SCAD3. The results from three examples are presented. The first example is the active low-pass filter circuit shown in Figure 29. We have tested our program on two different implementations of Opamps used in the subcircuit of the low-pass filter circuit: a linear model of 741 Opamp circuit shown in Figure 31 and a miller-compensated two-stage Opamp circuit shown in Figure 35. For the miller-compensated Opamp circuit, all the MOS transistors are replaced by their corresponding small-signal models at their DC operating points computed by SPICE. The AC analysis is performed by depth-first traversals of all the DDD vertices used to represent all symbolic expressions at each frequency point.

The second example is a band-pass filter circuit shown in Figure 36. This example was also used to illustrate hierarchical analysis in [39, 70]. The circuit can be partitioned into a circuit hierarchy as shown in Figure 37 with four topologically identical subcircuits $X1$ to $X4$ shown in Figure 38. For each Opamp in band-pass filter circuit, we also test two implementations of the Opamps in Figure 31 and Figure 35, respectively.

The third example is the bipolar $\mu A741$ circuit with 26 transistor and 11 resistors

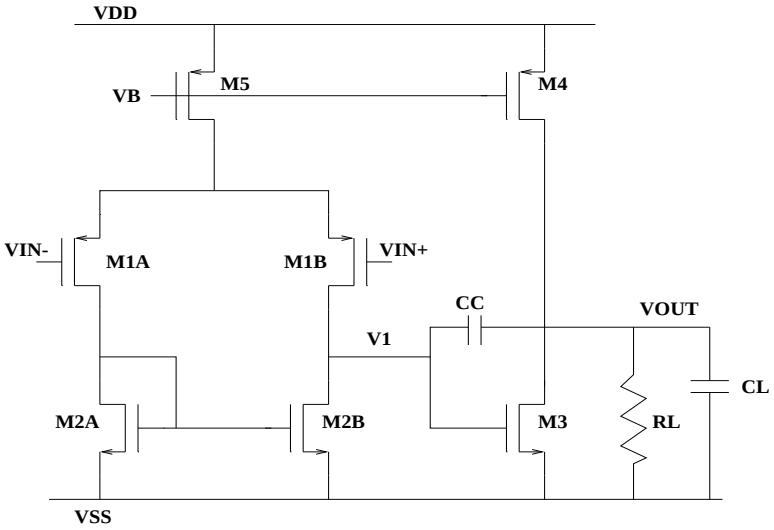


Figure 35: Miller-compensated two-stage Opamp

shown in Figure 14. All the BJT transistors are replaced by their corresponding small-signal models at their DC operating points.

We conducted a number of experiments on a SUN SPARCstation 5 with 32M memory. We first compared our program with SPICE on repetitive numerical evaluations. For each circuit, 1000 frequency points were computed. The results are summarized in Table 7, where $\#term$ is the actual number of distinct product terms generated, $|DDD|$ is the number of DDD vertices used to represent all the symbolic expressions. Columns 4, 5, and 6 list, respectively, the CPU time in seconds used by the proposed DDD-based method, SPICE, and the speedup of the proposed method over SPICE for each test circuit. From Table 7, we can see that the proposed DDD-based method outperforms SPICE for all the test cases. Further, the speedup increases with the size of the circuit.

We then compared our method with SCAPP—a best-known hierarchical symbolic analyzer [39]. We constructed the test circuits by cascading, respectively, the first 1, 2, 3 and 4 subcircuit blocks (Figure 37). The Opamp subcircuit was imple-

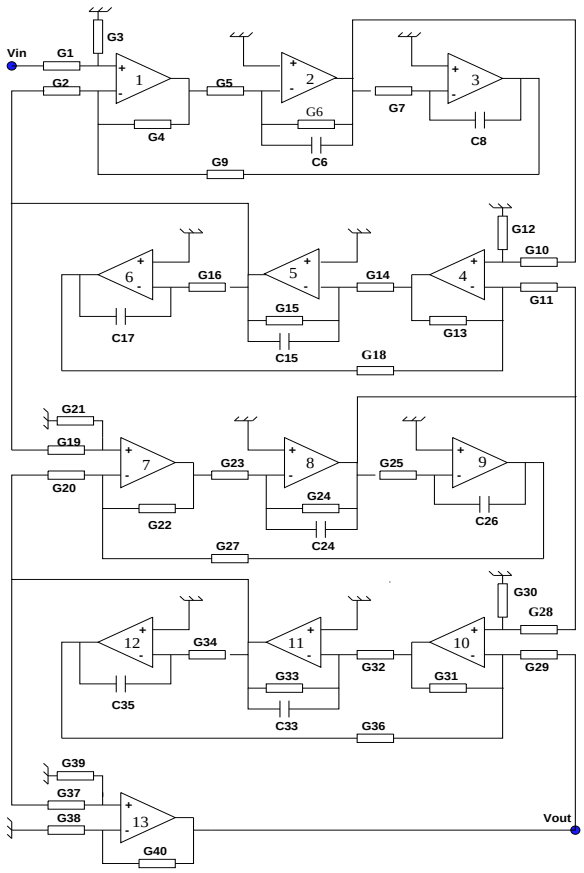


Figure 36: An active RC band-pass filter

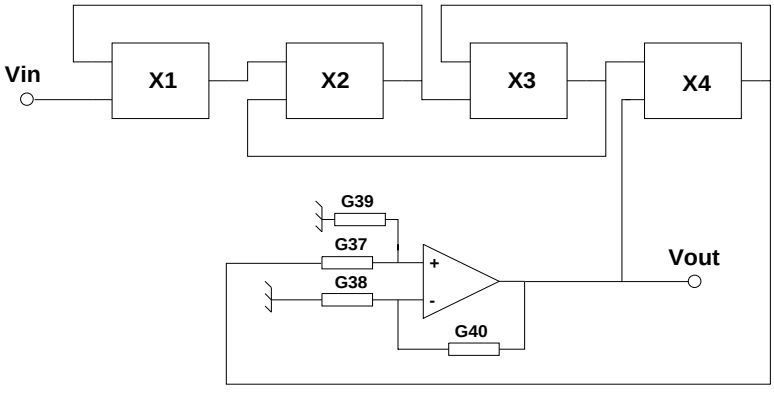


Figure 37: The partitioned RC band-pass filter

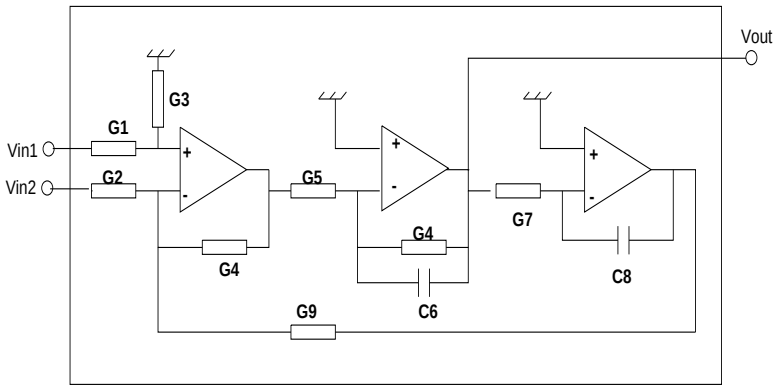


Figure 38: The subcircuit in the band-pass filter

Table 7: Comparison against SPICE in numerical evaluation

circuit	#term	|DDD|	DDD	Spice	Speedup
LP(linear)	52	58	1.07	6.58	6.15
LP(miller)	124	110	1.56	14.82	9.50
BP(linear)	562	228	3.07	9.70	3.16
BP(miller)	622	243	3.72	17.76	4.77

Note: LP is for low-pass filter and BP is for band-pass filter.

linear or *miller* denotes the corresponding Opamp model used.

mented by the miller-compensated Opamp circuit. The results are summarized in Table 8. Columns 1 and 2 list, respectively, the number of subcircuits cascaded for each test case and the total number of node voltage and branch current variables in the flatten circuits. Columns 3 and 4 describe the number of DDD vertices and the number of product terms represented. Columns 5 and 6 give the numbers of addi-

tions and multiplications used in the expressions generated by SCAPP. Note that the number of additions and multiplications used by the proposed DDD-based method is bounded by the number of DDD vertices. From Table 8, we can observe that the

Table 8: Comparison against SCAPP

#subckt	#var	DDD-based		SCAPP	
		#vertices	#terms	#add	#mul
1	23	146	500	556	274
2	39	369	2.60×10^4	1339	854
3	55	568	1.60×10^6	1772	1074
4	71	767	1.01×10^8	2479	1639

DDD-based representation is much more compact than the sequence-of-expressions representation used in SCAPP. Note that the number of DDD vertices is about 2-4 times less than the number of expressions in SCAPP, and the storage of each expression from SCAPP takes much more space than that for one DDD vertex. We also see that the DDD size grows almost linearly in the circuit size, despite the fact that the number of product terms grow exponentially.

Table 9 shows the statistics of using the DDD-based symbolic method and SCAPP for repetitive numerical evaluation. For SCAPP, we compiled the generated symbolic expressions and then executed the code for simulation. Columns 2 and 3 reports the CPU time required to construct the DDDs and then the CPU time taken for the simulation based on the constructed DDDs for each circuit. Columns 4, 5, and 6 report, respectively, the CPU time for SCAPP to generate the sequence-of-expressions (*analy*), the compilation time (*comp*), and the actual simulation time (*sim*). The last two columns give the matrix setup time and simulation time used

by SPICE. We can see from Table 9 that the proposed DDD-based method is more

Table 9: Comparison against SCAPP and SPICE in CPU time

#subckt	DDD-based		SCAPP			SPICE	
	const.	sim.	analy.	comp.	sim.	setup	sim.
1	0.37	2.09	0.81	13.1	2.60	1.10	5.34
2	1.01	4.75	2.09	33.3	7.49	2.70	8.98
3	2.42	6.91	3.69	44.2	10.37	3.12	15.58
4	12.75	9.19	5.54	64.7	12.06	3.42	22.10

efficient than both SCAPP and SPICE. Although it may take less time for SCAPP to obtain the sequence of expressions, it takes a much longer time to compile the generated expressions themselves. We also note that our program is interpreted, while SCAPP is compiled. Therefore we expect that the compiled and optimized version of our DDD-based method could offer even more improvement over SCAPP.

To test our program on analog circuits with closely coupled structures, we partition the $\mu A741$ Opamp circuit into a two-level binary tree hierarchy shown in Figure 39. The partitioned $\mu A741$ is shown in Figure 40. Table 10 shows the statistics for the two-way, three-level partitioning of $\mu A741$ circuits. Row 2 to 3 list the DDD sizes and the numbers of internal nets in the leaf subcircuit $I1$, $I2$, $II1$ and $II2$, respectively. Row 4 to 6 list, respectively, the DDD size, the numbers of internal nets and the number of cut nets in the two middle circuits I and II respectively. Among these cut nets, two nets are internal to I and three nets are internal to II as shown in 5th row. Row 7 to 8 show the corresponding DDD size for the root circuit and the total DDD size. Row 9 shows the best result from SCAPP with automated partitioning, where $\#mul$ and $\#add$ are the numbers of multiplications and additions,

Table 10: Statistics for three-level, two-way partitioning on $\mu A741$

leaf subcircuits	I1	I2	II1	II2
$\|DDD\|$	10	36	23	6
#internal nets	3	4	4	3

middle subcircuits	I	II
$\|DDD\|$	4	18
#internal nets	2	3
#cut nets	5	6

root $\|DDD\|$	20
total $\|DDD\|$	117
SCAPP	#mul:182, #add: 357

respectively.

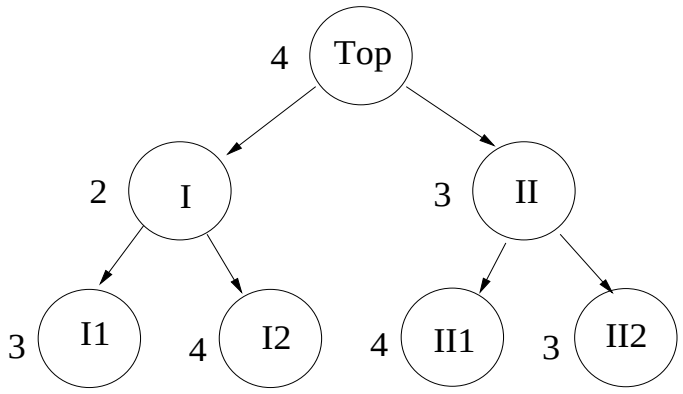


Figure 39: A three-level, two-way partition of $\mu A741$

5.7 Summary

A new method for hierarchical symbolic analysis of analog integrated circuits has been presented and implemented. The method takes the advantage of both the circuit hierarchy and determinant decision diagrams for symbolic determinant repre-

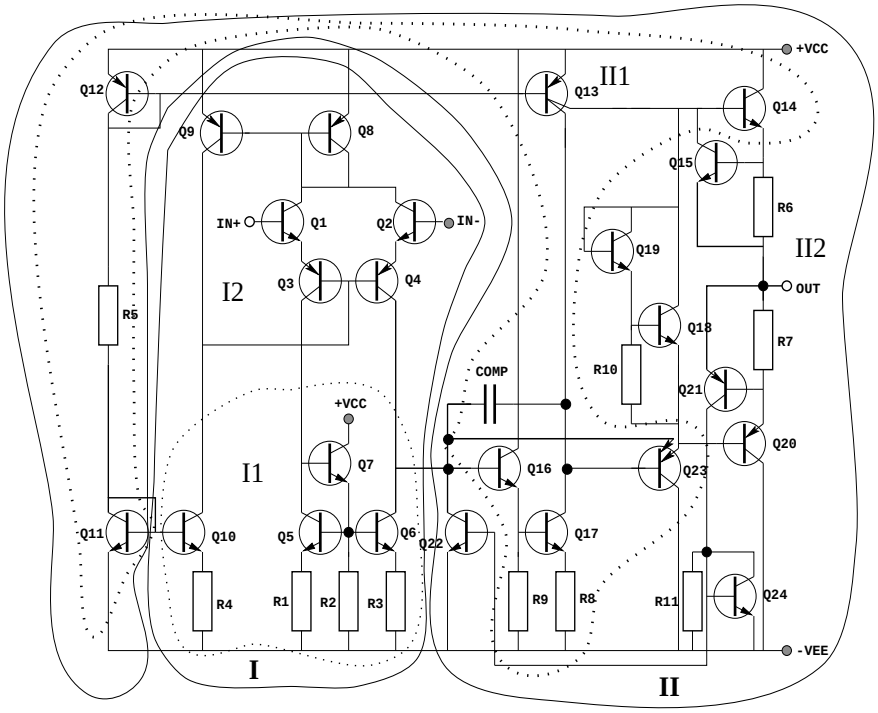


Figure 40: A three-level, two-way partitioned $\mu A741$

sentations. DDD representation enables systematically exploiting the sharing among symbolic expressions and thus results in very compact nested-form symbolic expressions. Experimental results have shown that the proposed method compares very favorably with the best-known symbolic analyzer SCAPP and numerical simulator SPICE for small-signal AC analysis.

CHAPTER VI BALANCED PARTITIONING OF ANALOG INTEGRATED CIRCUITS FOR HIERARCHICAL SYMBOLIC ANALYSIS

6.1 Introduction

In this chapter, we consider the problem of partitioning a circuit so that DDD-based hierarchical decomposition uses the minimal number of DDD vertices to represent the symbolic expressions.

The circuit partitioning problem, in general, is formulated as a graph optimization problem in which an optimal partition of nodes is sought, so as to minimize a certain measure on the interconnection between different groups of nodes. The problem of computing an optimal bisection of a graph is known to be NP-complete [2, 33]. We therefore have to resort to heuristics for practical solutions. Some attempts have been made at finding an optimal partition of analog circuits in the context of node-tearing nodal analysis [67], network formulation approach to symbolic analysis [36] and parallel circuit simulation on multiprocessors [95].

The partitioning algorithm in [67] is based on a clustering-based technique (called *contour tableau*) to find the clusters (subcircuits) of closely coupled circuit elements, so that the size of every cluster is less than a user-specified constant, and the interconnection among clusters is minimized. The algorithm has linear time complexity, but it may yield an unbalanced partition, and thus becomes less attractive in applications where a balanced partition is desired. The partitioning algorithm proposed in [36] partitions a circuit into a binary tree hierarchy by the permutations on the rows of the circuit matrix based on some interconnection information. The

method fails to consider balance constraints and explores only the binary-tree circuit hierarchy. The method in [95] seeks a balanced bisection of a circuit by applying the Kernighan-Lin algorithm [44], which has quadratic time complexity. A balance partition is achieved by incorporating the balance constraints into the cost function. It should be pointed out that thus far no effective partitioning results has been reported for symbolic analysis of such less regular-structured circuits as $\mu A741$ Opamps [31].

Chapter V described the DDD-based hierarchical symbolic analysis and demonstrated its power in reducing the complexity of the exact symbolic analysis. The effectiveness of the hierarchical decomposition, however, depends crucially on how a circuit is partitioned.

In this chapter, we formulate the partitioning problem so that the size of resultant DDD from hierarchical decomposition is minimized. We adopt a multi-way partitioning strategy with balance constraints. It consists of two phases: (1) initial partitioning phase and (2) iterative refinement partitioning phase. The resulting algorithm has time complexity $O(|E|(k + log_2|E| + k^2 log_2 k))$, where k is the number of parts a circuit to be divided and $|E|$ is the total number of nodes in a circuit.

To obtain a partition of tree hierarchy, the algorithm is simply used recursively until the constraint on the maximum number of internal variables of leaf circuits is attained.

The rest of the chapter is organized as follows: Section 6.2 formulates the partitioning problem for DDD-based hierarchical symbolic analysis. Section 6.3 presents a new multi-way multi-level partitioning heuristic. Section 6.4 describes some experimental results. Section 6.5 gives a summary.

6.2 Problem Formulation

An analog circuit can be modeled as a hypergraph $H(V, E)$ with vertex set $V = \{v_1, v_2, ..., v_m\}$ and hyperedge set $E = \{e_1, e_2, ..., e_n\}$. Each vertex corresponds

to a circuit device and each hyperedge represents a *node* or *net* in the circuit. A hyperedge e_j is a nonempty subset of two or more vertices, i.e. $e_j \in V$ and $|e_j|$ is the *size* of net e_j and $|e_j| \geq 2$.

The k-way hypergraph partitioning problem is to assign $v_i, i = 1, ..., m$ into k disjoint subsets $V_1, V_2, ..., V_k$. If $k > 2$, it is a *multi-way* partitioning problem. If subcircuit $V_j, j = 1, ..., k$ are further decomposed into smaller subcircuits, it is *multi-level* or *tree* partitioning.

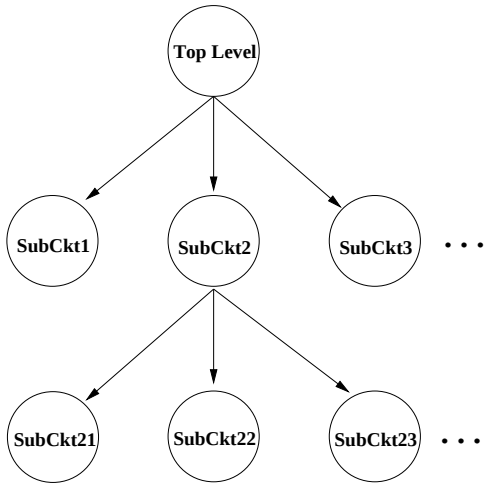


Figure 41: Model of a circuit hierarchy

A general circuit hierarchy is shown in Figure 41. A subcircuit containing no subcircuits is a *leaf* subcircuit; otherwise it is a *middle* subcircuit. We further define the *span* of a net e_j, denoted as $span(e_j)$, as the number of the subcircuits e_j connects. If $span(e_j) > 1$, e_j is called *cut* net, otherwise it is an *internal* net. In nodal analysis, a node voltage variable corresponds to a node (net) in an analog circuit, therefore a hyperedge in the corresponding hypergraph. Similarly, a variable is an *internal* variable if its corresponding net is an internal net; otherwise it is a *boundary* variable (its corresponding net is a cut net).

We first consider the effect of a circuit hierarchy on the way symbolic expressions are derived and represented by DDDs. From the expression (5.13), we observe that for a leaf circuit, the determinant of its *internal* circuit matrix, A^{II} and a number of its first-order cofactors, $\Delta^{II}_{k_2,k_1}$, are needed to be represented by DDDs. In order to reduce the number of those first-order cofactors, the number of *boundary* variables, $a^{BI}_{x,y}$ and $a^{IB}_{x,y}$, of the subcircuit should be minimized. Therefore, the total number of cut nets in all the subcircuits should be minimized. The partitioning objective for our application can be given by:

$$min \sum_{e_j \in E} span(e_j). \tag{6.1}$$

Min-span partitioning will cause very dense interconnection within each leaf subcircuit; thus the corresponding subcircuit matrices are dense. For a dense or full matrix, its DDD representation is still exponential. Figure 42 shows the number of vertices of a DDD for the determinant of a full matrix and the number of product terms in the determinant versus the size of the full matrix under the variable ordering given by the heuristic described in Chapter III.

To simplify the problem formulation, we make the following two assumptions. First, we assume that the size of a DDD representing an $l \times l$ determinant, $det(T)$, of matrix T is an exponential function $g(l)$ in l, $g(l) = l^{\alpha}$, where α is a positive constant. Second, we assume that the size of a DDD for $det(T)$ and the required first-order cofactors of $det(T)$ is $O(l^{\alpha})$. The second assumption can be justified by the fact that the required first-order cofactors due to subcircuit suppression usually are few given a small number of boundary variables. A few of the required first-order cofactors can be represented by DDDs whose size is compared to that for $det(T)$, which is k^{α}, because of the sharing among the required cofactors and $det(T)$.

Under the two assumption, the size of DDDs for hierarchical symbolic analysis

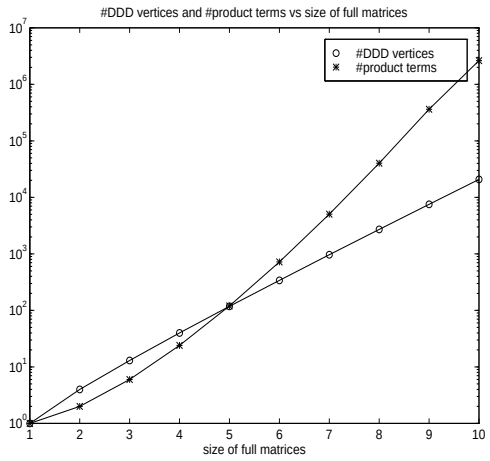


Figure 42: #DDD vertices and #product terms vs size of full matrices

of a circuit with k subcircuits will be given by

$$O(|DDD|) = O(I(V_1)^\alpha + I(V_2)^\alpha + \cdots + I(V_k)^\alpha + \xi(S_{cut})), \tag{6.2}$$

where $I(V_i)$ is the number of internal variables in subcircuit V_i, i.e.,

$$I(V_i) = \{e_j | e_j \subseteq V_i \text{ and } span(e_j) = 1\}, \tag{6.3}$$

and $\xi(S_{cut})$ is the size of DDDs due to the root circuit, and S_{cut} is the number of boundary variables. Note that we have

$$I(V_1) + I(V_2) + \cdots + I(V_k) + S_{cut} = |E|. \tag{6.4}$$

In order to minimize $O(|DDD|)$ one must clearly have

$$I(V_1) = I(V_2) = \cdots = I(V_k). \tag{6.5}$$

Equation (6.5) essentially calls for the balance on the internal variables of subcircuits. We solve the problem by introducing the following balance constraints:

$$I_{max} - I_{min} \leq \tau, \tag{6.6}$$

$$I_{min} = min\{I(V_i)\}, i = 1, ..., k,$$

$$I_{max} = max\{I(V_i)\}, i = 1, ..., k,$$

where τ is a measure of the offset from its balanced size (referred to as a *deviation factor*). The partitioning problem can be summarized as follows: Given a hypergraph $H(V, E)$, find a k-way partition of V subject to balance constraint (6.6), so that objective (6.1) is minimized.

For very large analog circuits, two-level partitioning may be still inadequate to reduce the overall DDD size. To keep the DDD size for each subcircuit manageable, multi-level partitioning has to be employed such that the minimum number of internal variables of leaf subcircuits is bounded. The upper bound on the number of internal nets of leaf circuit j can be expressed as

$$I(V_j) \leq \beta|E|, \tag{6.7}$$

where β is a positive constant and $\beta \in [0, 1]$. For convenience, subcircuit and subset are used interchangeably throughout the rest of this chapter.

6.3 Multi-Way, Multi-Level Balanced Partitioning Algorithm

Our approach to k-way constrained hypergraph partitioning consists of initial partition construction and iterative refinement.

6.3.1 Initial Partition Construction

Iterative refinement algorithms such as Kernighan-Lin's algorithm likely trap to local minima [44, 28]. A reasonable initial partition therefore is crucial to obtain good results. In addition, we need to find a feasible initial solution due to the balance constraints.

A vertex v becomes *locked* after it is moved; otherwise it is *free*. A net is designated *Free* when all its vertices are free; otherwise, the net designated is *Loose* if

it still has free vertices and all the locked vertices are in one subset, or it is *Locked* if locked vertices on the net are in more than one subsets or all the vertices are locked. A *Locked* net, however, may still have free vertices.

We use a seed-net growing strategy. The k clusters are growing in such a way that each step the number of internal nets are maximized. We *move* a net (move all its free vertices) each time. The moving scheme first selects a *seed* (the first net) for each subset and then moves all the nets with the consideration of their interconnection with the *Locked* nets in all the subsets until all the vertices are locked, which marks the end of the moving process. The algorithm is described in Figure 43.

InitPart $(H(V, E), k)$

1. Set all the nets *Free*, and sort increasingly all the nets according to their size and the resulting sorted net list is $\mathbf{L}[i]$, $i = 1, ..., |E|$.
2. **for** $s = 1$ to k **do** /* seed net for each cluster*/
   Find the first available *Free* net i (i.e. i is the smallest such that $\mathbf{L}[i]$.status = *Free*) and move net i into cluster s.
3. **for** $i = 1$ to $|E|$ **do**
   if ($\mathbf{L}[i]$.status = *Free*)
   Move net i into cluster s, subject to constraint (6.7), such that the increase in the number of internal nets in s is a maximum among all the clusters.
4. **for** $i = 1$ to $|E|$ **do**
   if ($\mathbf{L}[i]$.status = *Loose*)
   Move net i into the cluster in which the net has the locked vertices subject to constraint (6.7).
5. **for** $i = 1$ to $|E|$ **do**
   if (net i has free vertices)
   Randomly move all the free vertices in net i such that the net becomes a cut net.

Figure 43: Initial partition construction

When a *Free* net is moved into a subset s, the number of *internal* nets of s is increased by at least one. This is also true for the *Loose* net when it is moved into

the subset in which it has locked vertices. Therefore, the move operations in steps (3) and (4) are restricted by the balance constraint (6.7) to guarantee a feasible solution. Once a net is moved and it becomes *Locked*, which will affect the status of its adjacent nets. We therefore have to adjust the status of those effected nets. In addition, I_{max} and I_{min} in the balance constraint (6.7) are also adjusted after a move operation. In step (3), the selection of subset for a *Free* net is based on its interconnection with subsets. Such an selection scheme, which emphasizes the clustering of the closely coupled vertices, typically yields better results.

If we assume the number of vertices per net and the number of connections per vertex are limited by a constant, n_{max}, which is also significantly less than $|E|$ in a hypergraph $H(V, E)$, we have following proposition.

Proposition 2 *Given a hypergraph $H(V, E)$, the time complexity of* InitPart *algorithm for k-way partitioning is $O(k|E| + |E|log_2(|E|))$.*

Proof.: The sorting procedure in step (2) requires $O(|E|log_2(|E|))$ to finish. In step (3), we visit each *Free* net just once. For each net, we need to compute its status, which takes $O(n_{max}) = O(1)$ to complete. If it is a *Free* net, we calculate the changes of the number of internal nets for each subset when the net is moved into it. Such a process will take $O(n_{max}^3)$ to finish, as we must visit all the nets on all the vertices connected by the moved net and compute their status. Hence the time complexity for this step is $O(kn_{max}^3|E|) = O(k|E|)$. In step (4), we calculate status of each *Loose net* and the change in terms of the number of internal nets in one subset, so the time complexity for this step is $O(n_{max}^3|E|) = O(|E|)$. The last step can be finished in $O(n_{max}|E|) = O(|E|)$ as we scan once all the vertices for each net that still has free vertices. By counting all the steps, the total time complexity of InitPart is $O(k|E| + |E|log_2(|E|))$. □

6.3.2 Iterative Refinement Algorithm

An iterative refinement procedure is employed to further improve the quality of an initial partition. The algorithm extends the iterative refinement heuristic by Fiduccia and Mattheyses (FM) [28], with an improved gain and a balance-relaxed vertex moving scheme described in this section.

The FM's algorithm begins with two initial subsets and proceeds in a series of *passes*. In each pass, it keeps moving vertices between two subsets until each vertex has been moved exactly once. After each pass, the best solution observed during the pass becomes the initial solution for a new pass. During each pass, the moved vertices are locked from further exchanging. The pass terminates when a pass does not improve upon the most recent solution.

6.3.2.1 Computation of Gain and Potential Gain

Similarly to FM's algorithm, we define the gain of moving a vertex into or out of a subset as the decrease in the span of a partition. Unless otherwise specified, we assume that a vertex v is moved from set A to set B, A and B are two subsets among k subsets and $v \in A$.

We further define an *incident number* of a net e with respect to a set of vertices A, denoted by $\alpha_A(e)$, as

$$\alpha_A(e) = |\{v | v \in A \text{ and } v \in e\}|, \tag{6.8}$$

where v denotes a vertex. A *binding force* of a net with respect to a set A, denoted by $\beta(e)$, is defined as

$$\beta_A(e) = \begin{cases} \alpha_{A_F}(e) & \text{if } \alpha_{A_L}(e) = 0 \\ \infty & \text{if } \alpha_{A_L}(e) > 0 \end{cases}, \tag{6.9}$$

where A_F and A_L denote the subsets that contains all free and locked vertices in A, respectively.

In order to efficiently calculate the $span(e_j)$ of a net e_j, we divide the move operation of v into two steps:

1. The vertex v is selected from A and put into $\overline{A}$ ($\overline{A}$ is complement of A, $\overline{A} = V - A$); the gain of this move operation, denoted by $G^A_{get}(v)$, is

$$\begin{aligned} G^A_{get}(v) &= |\{e \in E_v | \beta_A(e) = 1 \text{ and } \beta_{\overline{A}}(e) > 0\}| \\ &- |\{e \in E_v | \beta_A(e) > 0 \text{ and } \beta_{\overline{A}}(e) = 0\}|, \end{aligned} \tag{6.10}$$

   where E_v denotes the set of nets that vertex v connects.

2. The vertex v is moved from the complement of B ($\overline{B}$) to B. [1] The gain, denoted by $G^B_{put}(v)$, is

$$\begin{aligned} G^B_{put}(v) &= |\{e \in E_v | \beta_{\overline{B}}(e) = 1 \text{ and } \beta_B(e) > 0\}| \\ &- |\{e \in E_v | \beta_{\overline{B}}(e) > 0 \text{ and } \beta_B(e) = 0\}|. \end{aligned} \tag{6.11}$$

Then the total gain of the move operation of v from A to B is given by

$$G_{span}(v) = G^A_{get}(v) + G^B_{put}(v). \tag{6.12}$$

We note that if we ignore these terms, which only consider the change in the span in (6.10) and (6.11), we obtain the same gain as that given in [75], which is the number of cut nets *removed* [2] due to the move operation:

$$\begin{aligned} G_{cut}(v) &= |\{e \in E_v | \beta_{\overline{B}}(e) = 1 \text{ and } \beta_B > 0\}| \\ &- |\{e \in E_v | \beta_A(e) > 0 \text{ and } \beta_{\overline{A}}(e) = 0\}|. \end{aligned} \tag{6.13}$$

To satisfy the balance constraint (6.7), we need to check the following two constraints at each moving step:

$$I^*_{max} - \tau \leq I(A) - I_A(c) \text{ for set A}, \tag{6.14}$$

[1] Note that $\overline{A} \neq \overline{B}$, but their differences have no impact on the gain calculations.

[2] A cut net is removed when it becomes an internal net.

$$I^*_{min} + \tau \geq I(B) + S_B(v) \text{ for set B}, \tag{6.15}$$

where $I_A(v)$ is the number of all the internal net of A in E_v, i.e.,

$$I_A(v) = |\{e \in E_v | \beta_A(e) > 0 \text{ and } \beta_{\overline{A}} = 0\}|, \tag{6.16}$$

and $S_B(v)$ is the number of all the cut nets in E_v, which become internal nets of B after the move, i.e.,

$$S_B(v) = |\{e \in E_v | \beta_{\overline{B}}(e) = 1 \text{ and } \beta_B > 0\}|, \tag{6.17}$$

and

$$I^*_{max} = max\{I(V_i) \text{ for } V_i \neq A, I(A) - I_A(v)\}, \tag{6.18}$$

$$I^*_{min} = min\{I(V_i) \text{ for } V_i \neq B, I(A) + S_B(v)\}. \tag{6.19}$$

The differences between I_{max} and I^*_{max}, between I_{min} and I^*_{min} reflect the fact the A and B may be the sets that have the maximum or minimum number of internal nets.

Because each vertex v has $k-1$ moving directions associated with $k-1$ gain values for each direction. We select the direction with the highest gain each time. So a vertex is sorted according its highest gain. Once we select a vertex, we also know its destination.

One way to improve the partitioning quality is to break the tie situation where two vertices have the same gain [19, 48, 74]. We solve this problem by introducing a penalty function devised for multi-way partitioning into the vertex gain as a *potential* gain. Let $G_P(e)$ denote the potential gain of net e imposed on all its connected vertices. The potential gain also consists of two parts which correspond to the two aforementioned steps in a move operation:

$$G_P(e) = G^A_{Pget}(e) + G^B_{Pput}(e), \tag{6.20}$$

where

$$G^A_{Pget}(e) = \begin{cases} 0 & \text{if } \beta_A(e) = |e| \text{ or } \beta_A(e) = 0 \\ -p & \text{if } \beta_A(e) = \infty \\ -p \times w(e, A) & \text{if } 0 < \beta_A(e) < |e| \end{cases}, \tag{6.21}$$

and

$$G^B_{Pput}(e) = \begin{cases} 0 & \text{if } \beta_B(e) = |e| \text{ or } \beta_B(e) = 0 \\ p & \text{if } \beta_B(e) = \infty \\ p \times w(e, B) & \text{if } 0 < \beta_B(e) < |e| \end{cases}, \tag{6.22}$$

and $w(e, X) = \frac{\alpha_X(e)}{n_{max}}$ if $|e| \leq n_{max}$; and $w(e, X) = \frac{\alpha_X(e)}{|e|}$ otherwise. The constant p is a constant, and n_{max} is a user-specified upper bound on the number of vertices that a net connects. Both $G^A_{Pget}(e)$ and $G^B_{Pput}(e)$ favor the vertices on the cut nets, which likely become internal nets of B if the vertices are moved. The total gain for the whole move operation is expressed as

$$G_{AB}(v) = G_{span}(v) + \sum_{e \in E_v} (G^A_{Pget}(e) + G^B_{Pput}(e)). \tag{6.23}$$

Now we illustrate the gain calculations by means of a three-way partitioning example shown in Figure 44. Let $n_{max} = 5$ and $p = 3$, and all the vertices are free. Without considering the potential gain, moving vertices a, f and d to some subsets will reduce the span of the partition by one (real gain equals 1). The potential gain for moving vertex a from A to B is calculated as follows: $G^A_{Pget}(e_1) = 0$ and $G^B_{Pput}(e_1) = 0$, $G^A_{Pget}(e_2) = -3/5$ and $G^B_{Pput}(e_2) = 6/5$, $G^A_{Pget}(e_3) = -3/5$ and $G^B_{Pput}(e_3) = 3/5$. Hence, the total gain for this move is $G_{AB}(a) = 1 + 3/5 (= 0 + 0 - 3/5 + 6/5 - 3/5 + 3/5) = 8/5$. On the other hand, the potential gain of moving f to either A or B and of moving d to either A and C is zero. Therefore, moving a from A to B has the highest gain. Intuitively, the advantage of moving a over the other two moves is that *cut* net e_2, which is otherwise difficult to be removed, is first removed when a is

moved first.

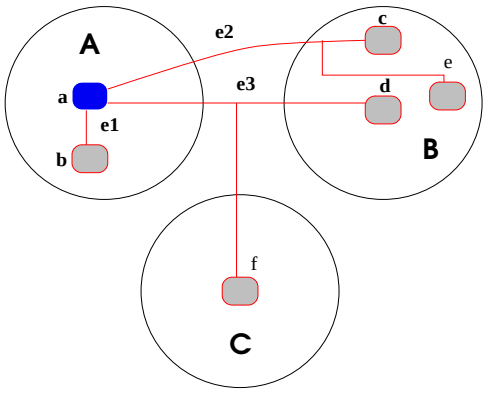


Figure 44: An illustration of gain calculation in multi-way partitioning

6.3.3 Balance Constraint Relaxation

Another problem with FM's algorithm is that moving a vertex is only feasible if the move operation dose not violate the balance constraints. Such a moving scheme will severely confine the solution space, especially when the constraints are strict [16, 82]. This problem can be alleviated by temporarily relaxing the balance constraints and allowing a sequence of move operations of vertices, called a *macro-step*, to be carried out as long as the balance constraints are restored after this macro-step.

Each macro-step begins with moving a vertex v from A to B without considering the balance constraint (6.15). But the move is still subject to the constraint (6.14). The reason is that we should not limit the increase of the internal nets in subset B, while we limit the decrease of the internal nets in subset A. This constraint-relaxation scheme is consistent with the min-span partitioning objective. We observe that after the move operation, $I[i_B]$ always increases. In case the move operation causes the violation of the balance constraint in B, we just move some vertices out of B to restore the balance.

More specifically, let $F = \{v_1, v_2, ..., v_m\}$ be a set of free vertices. Let $G_{max}[i]$ and $I[i]$ are the maximum gain of all the free vertices in subset i and the number of internal nets in subset i, respectively. The new balance-relaxed, multi-way partitioning algorithm, BalRelaxMP, is described in Figure 45.

BalRelaxMP($H(V, E)$)
while($|F| \neq 0$) **do** /* begin macro-step */
1. Sort $G_{max}[i], i = 1, ..., k$.
   for j = 1 to k **do**
   Try to select a free vertex v with the highest gain from subset j subject to constraint (6.14), and move v to its destination, subset t.
   if the free vertex v can be founded
   goto step 2.
   Current pass stops.
2. **while** (constraint (6.15) is violated) **do**
   move a free vertex v_x with the highest gain out of subset t subject to constraints (6.15) and (6.14).
3. Lock all the moved vertices and add them into current macro-step; record the current span size.

Figure 45: Balance-relaxed multi-way partitioning

A similar constraint-relaxation scheme has been shown to be effective on the digital circuit partitioning with the balanced area constraints [82].

6.3.3.1 Implementation and Time Complexity

In our implementation, the bucket data structure introduced in [28] is used. We modify this date structure to account for multi-way partitioning. Because the total gain of a vertex is still bounded after the introduction of the potential gains (6.21) and (6.22), the bucket sorting scheme is still applicable to our application. With this, resorting all the affected free vertices caused by moving a vertex will take a constant time.

Under the same assumption that the number of vertices per net and the number of connections per vertex are limited by a constant, n_{max}, we have the following proposition:

Proposition 3 *Given a hypergraph $H(V, E)$, the time complexity of* BalRelaxMP *for k-way partitioning is $O(|E|k^2 log_2(k))$.*

Proof.: Let $n(i)$ denote the the number of vertices on net e_i, then the total number of gain calculations due to the vertices on net i during a pass is

$$n(i) + (n(i) - 1) + \ldots + 1 = O(n^2(i)). \tag{6.24}$$

Hence, the total number of gain calculations in a pass is given by

$$O(\sum_{i=1}^{|E|} n^2(i)), \tag{6.25}$$

which also equals

$$\sum_{i=1}^{|E|} n^2(i) \leq \sum_{i=1}^{|E|} n_{max}^2 = |E| n_{max}^2. \tag{6.26}$$

As a result, the time complexity of the algorithm is $O(|E|n_{max}^2) = O(|E|)$. Within a vertex gain calculation, $k - 1$ gain values for each moving direction must be re-calculated, as each vertex has $k - 1$ move directions. Such a process takes $O(k)$ to complete. In BalRelaxMP algorithm, each macro-step begins by sorting k subsets. In the worst case, every macro-step corresponds to just one vertex move. The sorting process has the time complexity of $O(klog_2 k)$. Therefore, the total time complexity is $O(|E|k^2 log_2 k)$. □

Considering both initial partition construction and iterative refinement, the time complexity of the entire partitioning process in one pass is

$$O(|E|(k + log_2(|E|) + k^2 log_2(k))). \tag{6.27}$$

6.3.4 Multi-Way Tree Partitioning

Two-level partitioning can be easily extended to tree partitioning by recursively applying the multi-way algorithm, until the constraints (6.7) on the number of the internal nets of all the leaf circuits are satisfied.

In our implementation, multi-level partitioning is treated as a special sequence of multi-way partitioning processes where each multi-way partitioning is performed such that vertices are moved only from the subsets at a particular hierarchical level, and all the other vertices are locked during that pass. In this way, the gain calculations (6.23), balance constraints (6.14) and (6.15) will still remain valid.

6.4 Experimental Results

The proposed balanced multi-level, multi-way partitioning algorithm was implemented and linked to our symbolic analyzer SCAD3 [79, 83]. We report the results from two examples. All the results were obtained from a SUN SPARCstation 5 with 32M memory.

6.4.1 $\mu A741$ Operational Amplifier

The first example is the bipolar $\mu A741$ circuit with 26 transistors and 11 resistors shown in Figure 14. We first performed two-level, multi-way partitioning of $\mu A741$. The total number of nets for $\mu A741$ is 23. For small-signal AC analysis, the power and ground nets are the reference node in the nodal formulation method. They will not appear in the circuit matrix, and are ignored by partitioning. All the nets corresponding to the circuit inputs and outputs are always cut nets. We selected the *deviation factor*—τ to be 4.

Figures 46, 47 and 48 show, respectively, the results of two-way, three-way and four-way balanced partitioning of $\mu A741$, where each partitioned subcircuit is marked by an index (I up to IV). Table 11 summarizes the statistics of these partitioning. Columns 1, 2 and 3 list, respectively, the number of subcircuits, the span and the

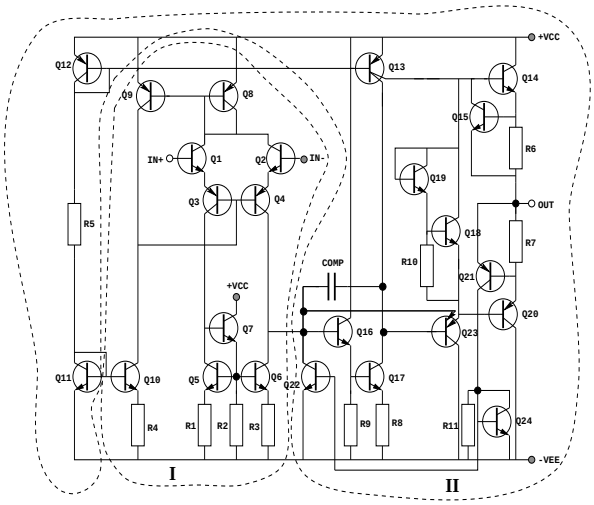


Figure 46: A two-way partitioned $\mu A741$

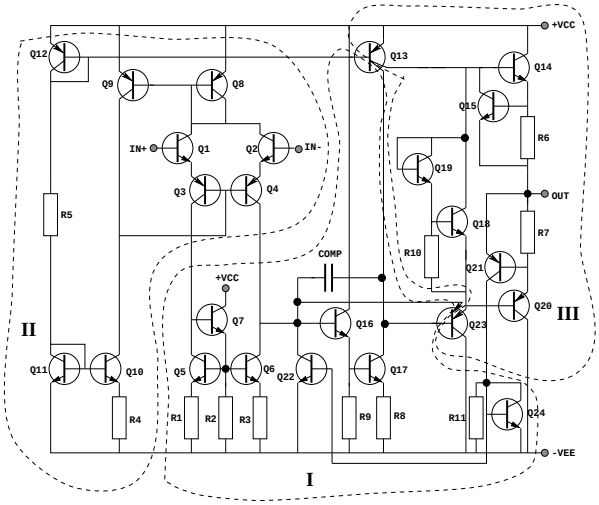


Figure 47: A three-way partitioned $\mu A741$

number of cut nets. Columns 4 to 7 list, respectively, the number of internal nets in each subcircuit and their corresponding DDD size. Column 8 gives the number of DDD vertices in the root circuit, $|topD|$, where the last column describes the total number of DDD vertices, $|totalD|$. We make the following observations:

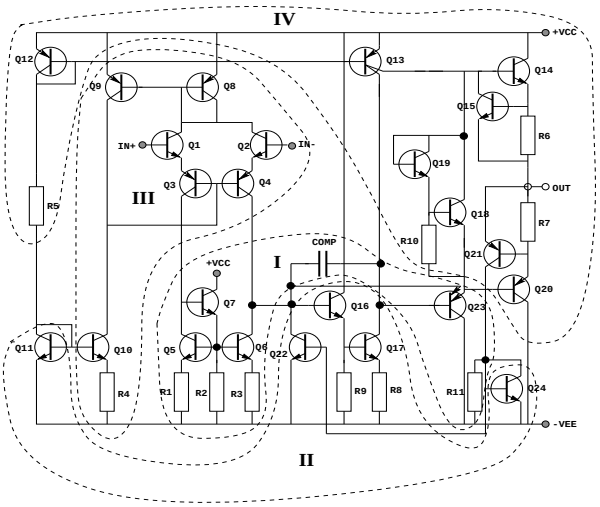


Figure 48: A four-way partitioned $\mu A741$

Table 11: Statistics of two-level, multi-way partitioning on $\mu A741$

k	span	#cut	#internal nets/$\|DDD\|$				$\|topD\|$	$\|totalD\|$
			I	II	III	IV		
2	8	4	9/260	10/557	-	-	20	837
3	13	7	5/29	6/79	5/55	-	80	241
4	18	8	3/5	2/5	5/53	5/57	114	237

- The total number of DDD vertices used for representing all the subcircuits decreases from 817 in the balanced two-way partitioning to 125 in the balanced four-way partitioning. Meanwhile, the number of DDD vertices used for the root circuit increases from 20 to 114. The total number of DDD vertices decreases as k increases from 2 to 4. If we further increase k, more nets will be cut and the size of the DDD for the root circuit will increase rapidly, and dominate the overall DDD size.
- As we have reported in [79], without partitioning and hierarchical symbolic

analysis, the total number of DDD vertices to represent the $\mu A741$ is 7431, where the number of product terms in fully-expanded classical symbolic analysis is 374884. Thus, hierarchical symbolic analysis with automatic balanced partitioning leads to a very compact behavioral model (237 vs 7431)! Since the number of multiplications is linear in the DDD size, hierarchical symbolic analysis with automated partitioning speeds up canonical symbolic analysis (without partitioning) by a factor of 31, where canonical symbolic analysis already speeds up fully-expanded symbolic analysis by several orders of magnitude (7431 multiplications vs 374884 product terms).

We further perform a three-level, two-way partitioning of $\mu A741$ based on the hierarchy tree shown in Figure 39. The third-level partitioning is based on the two-level, two-way partitioning and the resulting tree hierarchy is shown in Figure 46. The resulting partitioning is shown in Figure 40.

Table 12 summarizes the partitioning statistics. We can see that hierarchical symbolic analysis with automated three-level, two-way partitioning reduces the number of DDD vertices from 7431 to 117! Since at most one multiplication is needed for one vertex, the total number of multiplications is bounded by 117. This is compared with the best-known hierarchical symbolic analyzer—SCAPP [39]. Row *SCAPP* lists the best result from SCAPP with automated partitioning, where $\#mul$ and $\#add$ are number of multiplications and additions, respectively.

6.4.2 A Band-Pass Filter

The second example is a band-pass filter, shown in Figure 36, where each Opamp subcircuit is a miller-compensated, two-stage MOS circuit, shown in Figure 35.

An important feature of the proposed partitioning algorithm is its capability in performing higher-level partitioning based on the existing subcircuit definitions. For example, we can view each Opamp circuit as a three-terminal block in the band-pass

Table 12: Statistics for three-level, two-way partitioning on $\mu A741$

leaf subcircuits	I1	I2	II1	II2
$\|DDD\|$	10	36	23	6
#internal nets	3	4	4	3

middle subcircuits	I	II
$\|DDD\|$	4	18
#internal nets	2	3
#cut nets	5	6

root $\|DDD\|$	20
total $\|DDD\|$	117
SCAPP	#mul:182, #add: 357

filter circuit shown in Figure 36 and perform partitioning at the whole circuit level. The resulting partitioning is shown in Figure 49. This contrasts with the partitioning algorithm in SCAPP, in which the automatic partitioning has to be carried out on the flatten circuits.

Table 13 summarizes the partitioning statistics along with a comparison with the best results from SCAPP in automated partitioning. Column *Op* shows the number of DDD vertices used to represent the Opamp subcircuit. Columns 3 to 6 describe the DDD size for each middle subcircuit. The last two columns give the DDD size at the root and the total number of DDD vertices. Rows 2 and 3 describe, respectively, the DDD size and number of internal nets for each subcircuit. Row $|DDD|$ *(w/o)* shows the DDD size without partitioning. Row *SCAPP* shows the best results from SCAPP with automated partitioning. Note that, for this example, the difference in the DDD size with and without partitioning is quite marginal. This is because the cascade structure in the band-pass circuit has already led to a small DDD representation [79].

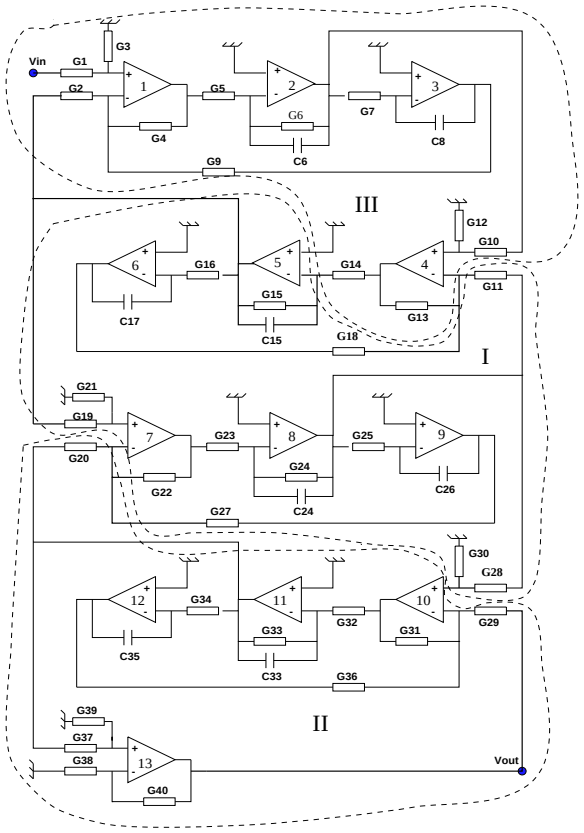


Figure 49: A three-way partitioning of the band-pass filter

Table 13: Statistics of three-way, tree partitioning of the band-pass filter

subcircuits	Op	I	II	III	#top	#total
$\|DDD\|$	17	168	140	166	173	664
#internal net	3	8	8	9	7	-
$\|DDD\|$ (w/o)	767					
SCAPP	#mul:1639, #add:2479					

6.4.3 Effectiveness of InitPart

We then studied the impact of the proposed initial partition construction algorithm (InitPart) on the final quality of a partition. For comparison, we implemented

another initial partition construction algorithm, RandInitPart, which randomly moves each net into a subset as long as no violations of balance constraints occur. The results from these two algorithms were then fed into the iterative refinement algorithm.

The experiment was conducted by performing four-way partitioning on both the $\mu A741$ circuit and the band-pass filter for 20 runs with *deviation factor* $\tau = 4$. The total number of nets in $\mu A741$ and band-pass circuit are 26 and 33, respectively (we counted power and ground nets in both circuits). Table 14 shows the statistics about the experimental results.

Table 14: Comparison between two initial partition construction algorithms

	InitPart			RandInitPart		
span	avg	std	min	avg	std	min
$\mu A741$	26.5	3.58	21	28.3	3.76	23
band-pass	27.2	5.7	18	32.4	8.9	21
#cut nets	avg	std	min	avg	std	min
$\mu A741$	9.5	1.2	7	12.8	1.9	9
band-pass	10.2	1.5	8	16.5	5.1	9

Rows 3 to 4 are the average (*avg*), the standard deviations (*std*) and the minimum (*min*) of the span of the resulting partitions by the InitPart algorithm (in columns 2, 3 and 4) and by the RandInitPart algorithm (in columns 5, 6 and 7) for circuits $\mu A741$ and band-pass filter, respectively. Rows 6 and 7 show the results in terms of cut nets for both circuits.

We make the following observations. First, the InitPart algorithm outperforms the RandInitPart algorithm in both circuits. Second, the InitPart algo-

rithm leads to more stable results in the 20 runs on both circuits. This is reflected in the standard deviations of both span and numbers of cut nets in the resulting partitions. Hence, given the improved average results and reduced volatility, the InitPart algorithm can considerably reduce CPU time, since we can obtain similar results with less number of trials in practice.

6.4.4 Comparison with Contour Tableau Method

Finally we compared our method with the contour tableau method [67] on real analog circuits. In the contour tableau method, the partitioning procedure is directed performed on a graph $\Omega(N, B)$ representing an analog circuit, where N is a set of nodes in the circuit and B is a set of edges representing circuit elements connecting nodes in the analog circuits. A clustering procedure is performed by selecting one node at a time. The cluster is identified such that a minimum number of adjacent nodes outside the cluster is observed before the user-specified upper bound on the number of nodes, n_{up}, in the cluster is reached. This method is efficient at finding closely coupled circuit elements [67]. Unfortunately it may yield very unbalanced partitions for some circuit structures, as shown in the following example.

In the band-pass filter circuit shown in Figure 36, we selected several upper bounds , n_{up}, on the clusters. The results are shown in Table 15. Column 2 is the number of clusters (subcircuits) generated. Columns 3 and 4 are the span and the number of cut nets in resulting partitions. Column 5 is the number of DDD vertices used for the partitioned circuits. Columns 6 to 11 are the number of nodes (also the number of internal nets) in each cluster. We note that in all the resulting partitions, the differences between the maximum number of internal nets and the minimum number of internal nets are quite remarkable. The partitions with $n_{up} = 10$ are drawn in Figure 50. The reason is that after a cluster has been identified and removed from the graph along with the cut nets, the remaining circuit will become

Table 15: Partitioning results on band-pass filter
by the contour tableau method

n_{up}	k	span	#cut	$\|DDD\|$	#nodes in each cluster					
					I	II	III	IV	V	VI
5	5	22	11	735	5	1	5	5	5	-
7	6	26	9	927	7	1	2	5	7	1
10	5	17	5	684	10	2	10	2	3	-
11	5	17	5	684	10	2	10	2	3	-
13	5	17	5	684	10	2	10	2	3	-
15	3	11	5	837	10	2	15	-	-	-
DDD	5	23	9	570	3	6	5	3	6	-

a set of separated graphs (the remaining graph is not a single connected graph). If some of the separated graphs are very small, the algorithm will soon identify them as clusters, thus resulting in the unbalanced partitions. In this example, after cluster I is identified and removed, cluster II becomes a separated graph. This is also true for cluster IV after cluster III is identified. We note that this inherent problem can't be solved by simply increasing n_{max} as shown in Table 15. A more balanced partition obtained by the new partitioning algorithm is shown in the last row of the table, which indeed results in smaller DDD size.

6.5 Summary

We have addressed the problem of partitioning an analog circuit in such a way that hierarchical decomposition will use the minimum number of DDD vertices. An efficient balanced multi-level, multi-way partitioning algorithm has been presented for its solution. The novelty is the introduction of a new potential gain for each vertex and a balance-relaxed vertex moving scheme. This idea has also been recently applied

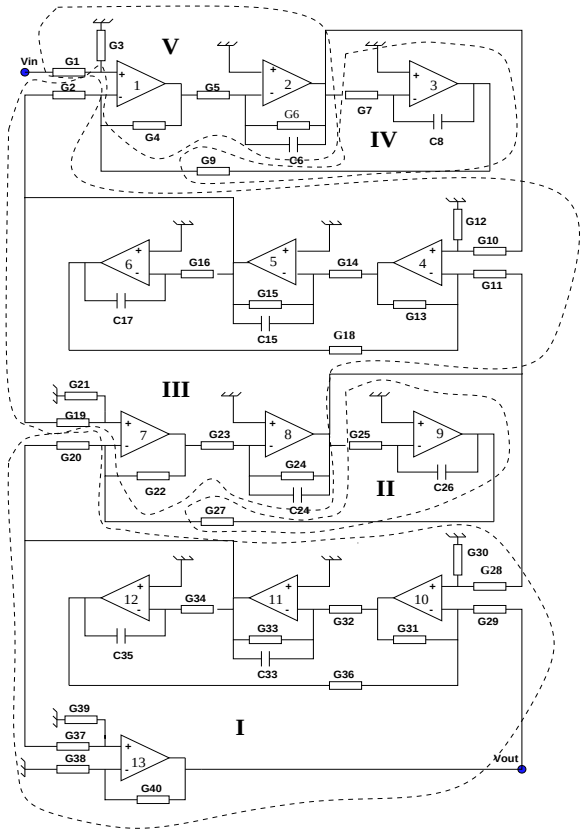


Figure 50: A five-way partition of band-pass filter by contour tableau method

to the circuit layout problem and shown to be superior to other recent work [82]. Our symbolic analyzer in conjunction with the proposed partitioning algorithm has shown its advantage over the best analog symbolic analysis program SCAPP.

CHAPTER VII INTERPRETABLE SYMBOLIC SMALL-SIGNAL CHARACTERIZATION OF ANALOG INTEGRATED CIRCUITS

7.1 Introduction

Deriving interpretable symbolic small-signal characteristics of analog integrated circuits by approximation can build the circuit behavioral models and gain intuitive insights into the circuit behavior. Approximation can also avoid the burden of generating all the product terms and thus overcome the long-standing circuit-size problem of symbolic analysis. The various approximation schemes that have been developed can be classified into three categories: (1) approximations-before-generation methods [40, 97] (2) approximations-during-generation methods [25, 97, 90] and (3) approximations-after-generation method [23, 30, 68, 69, 92]. We have briefly reviewed these methods in Chapter II; more detailed summaries of approximation techniques can be found in [26, 31].

In this chapter, we propose a new strategy for symbolic approximations based on determinant decision diagrams and their graph manipulations [79, 80]. The proposed strategy consists of several approximation steps to derive the interpretable expressions. The key idea of the proposed method lies in the exploration of the remarkable capability of DDDs and s-expanded DDDs in representing a huge number of symbolic product terms. As it is shown in the experimental results section, 6.40×10^{19} product terms can be captured by only 297117 DDD vertices! From this highly compact yet exact representation, we show that approximations can be performed efficiently by using simple yet efficient graph operations with the linear time complexity in terms of DDD size (number of DDD vertices).

Since we begin with exact symbolic expressions, the new approximation approach thereby integrates both the reliability in the approximations-after-generation methods, and the capability for handling large analog circuits in approximations-before/after-generation methods. In addition to the interpretable symbolic expressions, the new algorithm is also able to produce very accurate expressions in a highly compact fashion for analog circuit behavioral modeling and for repetitive numerical evaluation. This contrasts with existing approximations-before/during-generation methods, where increased accuracy will lead to the exponential growth of product terms. Finally, we provide a general framework for simplifying not only transfer functions like input/output impedances, current gains, voltage gains, but also such characteristics as poles, zeros, sensitivities.

This chapter is organized as follows: Section 7.2 reviews the framework of DDD-based symbolic analysis, which is the starting point of our approximation approach. Section 7.3 describes the first step of approximation: discarding insignificant terms. In Section 7.4 we show how coefficients of higher powers of s are suppressed. Section 7.5 presents the method of eliminating the canceling terms, which significantly improves the efficiency of the dominant-term generation procedure given in Section 7.6. Error control scheme used in our approximation method is provided in Section 7.7. Experimental results are described in Section 7.8. Section 7.9 summarizes the results in this chapter.

7.2 A Framework for Interpretable Small-Signal Characterization

A linear(ized) analog circuit can be described by a set of linear equations, in frequency domain, in the following general form by using the modified nodal analysis (MNA) approach [86]:

$$\mathbf{Tx} = \mathbf{w}, \tag{7.1}$$

where the *circuit unknown vector* $\mathbf{x}$ may be composed of node voltages and branch currents, and the *circuit matrix* $\mathbf{T}$ is a large sparse symbolic matrix.

We consider small-signal characterization as finding interpretable transfer functions, sensitivity expressions, symbolic pole and zero expressions. According to Cramer's rule, the kth component x_k of the unknown vector $\mathbf{x}$ is obtained as follows:

$$x_k = \frac{\sum_{i=1}^{n} \; w_i \; (-1)^{i+k} \; det(\mathbf{T}_{t_{i,k}})}{det(\mathbf{T})}. \tag{7.2}$$

Note that the numerator and the denominator are only composed of the determinant and cofactors of the circuit matrix, which can be represented effectively by using DDDs. With this, interpretable small-signal characterization can be performed by DDD-graph manipulation:

1. A network function is defined as the ratio of an output unknown from $\mathbf{x}$ over an input from $\mathbf{w}$. This can be represented as the ratio of two complex DDDs, or two s-expanded DDDs.
2. Under the assumption that poles are far away from each other, the root splitting method can be applied to find symbolic expressions for poles and zeros [27]. In this case, a pole or zero can be represented as the ratio of two consecutive coefficient DDDs.
3. Sensitivity of a DDD with respect to a circuit parameter amounts to finding a DDD cofactor. Sensitivities of general small-signal characteristics with respect to circuit parameters can be represented as expressions of DDDs and their cofactors.

After we obtain the exact DDD representations of small-signal characteristics, the key task is how to simplify a DDD or a ratio of two DDDs.

It turns out that DDD simplification can be easily performed by means of efficient DDD-graph manipulations. Moreover, since we begin with an exact and compact

DDD representation of a symbolic matrix determinant, all the error-controlling mechanisms, such as monitoring response magnitudes/phases, poles/zeros, can be effectively implemented in our framework. This is only feasible previously in approximations-after-generation procedures, which work only for small analog circuits [30, 92].

Our DDD-based approximation algorithm uses the following procedures: (1) *Discarding insignificant terms*; (2) *Coefficient suppression*; (3) *Removing canceling terms*; (4) *Generation of dominant-terms*.

7.3 Discarding Insignificant Terms

Discarding insignificant terms involves removing those terms insignificant to the characteristics of interest. It consists of two methods: *device elimination* and *node contraction*.

Consider a transfer function written as the following form:

$$f(p) = \frac{N}{D} = \frac{pN_p + N_{\overline{p}}}{pD_p + D_{\overline{p}}}, \tag{7.3}$$

where p is a circuit element in the admittance form, N_p (D_p) is the sum of all the product terms in N (D) containing p from which p is removed, and $N_{\overline{p}}$ ($D_{\overline{p}}$) is the sum of product terms in N (D) not containing p.

There are two scenarios where p can be eliminated from both N and D. First, if both $D_{\overline{p}}$ and $N_{\overline{p}}$ are compared to N and D, then f can be simplified by $f' = \frac{N_{\overline{p}}}{D_{\overline{p}}}$. This step removes those devices that are not significant in the small-signal characteristics of interest. So it is called *device elimination*.

Second, if both pN_p and pD_p dominate N and D, then f can be simplified by $f' = \frac{N_p}{D_p}$. This is called *node contraction*. With this, the number of elements in each product term is decreased by one. A detailed explanation of these two methods is as follows.

7.3.1 Device Elimination

In modified nodal analysis (MNA) formulation, a circuit device parameter p in the form of an admittance can appear in four rectangular positions in a circuit matrix T and its cofactors. If we denote the four appearances of p as p_k, $k = 1, ..., 4$. Then the value change, ε_p, in $det(T)$ by removing p can be expressed as:

$$\varepsilon_p = \sum_{k=1}^{4} p_k \cdot (-1)^{(r_k(p)+c_k(p))} \cdot |T_{r_k(p),c_k(p)}|, \tag{7.4}$$

where $r_k(p)$ and $r_k(p)$ denote the row and the column indices of p in its four rectangular positions in T. Since a circuit device may not appear in all the four positions, the summation in (7.4) is taken over all the positions the device is present in T.

In our implementation, the error due to removing a circuit device p is computed by DDD evaluations. First, we compute the numerical response of a network function at each sampled frequency point by DDD evaluations. Second, we *numerically* remove p from all the matrix entries containing p, and compute the network response again at each sampled frequency point. If the differences in terms of magnitude and phase between the first result and the second one at each frequency point are bounded by user-specified error margins, device p is *permanently* removed; otherwise we put p back by updating matrix entries containing p. Note that the evaluation of the exact DDDs (first step) is needed only once for the entire device elimination process.

If a matrix entry becomes empty due to device elimination, all DDD vertices representing this matrix entry can be removed from the corresponding DDDs by DDD REMAINDER operation in Table 1.

7.3.2 Node Contraction

Bias circuitry typically consists of a large portion of devices in an analog integrated circuit. Usually devices in the bias circuitry do not have direct impacts on small-signal characteristics. *Node contraction* exploits this fact by factorizing insignif-

icant circuit device p (most of them are in bias circuitry) from both the numerator and the denominator in a network function.

In a practical analog circuit, circuit devices that are topological connected usually have the same impacts on circuit behavior. This is especially true for integrated circuits in which the small signal models of some nonlinear devices are connected together. So unlike [97], which considers each individual device p, we consider a group of devices connected to a particular circuit node. This idea has been proven to be more effective. Therefore in our algorithm, we try to factorize a group of devices that have a common node in the circuit. Such a simplification in effect removes all the devices connected to a circuit node. In terms of complex DDDs, it is equal to factorizing a complex DDD node corresponding to a diagonal entry in the system matrix and removing all other appearances of the eliminated devices in the system matrix. To this end, DDD operation Cofactor(D,p) is employed to build the new DDD representing all the product terms containing p in D and from which p is removed. Hence the number of elements in each product terms is also decreased by one. The simplified transfer function are then evaluated for each frequency point, and the results are checked against exact value to decide if the entry under question can be factorized out or not.

Both Cofactor and Remainder operations have a linear time complexity in size of DDDs. So the time complexity of both *device elimination* and *node contraction* operations are $O(kn|DDD|)$, where k is the number of frequency points, n is number of circuit devices, and $|DDD|$ is the size of DDD in both the numerator and the denominator of the network function before the two approximation procedures.

Experimental results in Table 17 show that the two approximation procedures can substantially reduce the complexity of a circuit and thus its circuit matrix. The effectiveness of these two procedures can be measured by the number of product terms

before and after the two operations shown, respectively, in row 5 and row 13 of the table.

7.4 Coefficient Suppression

After the first two approximation procedures, the simplified complex DDDs are transferred into s-expanded DDDs by one depth-first search on the complex DDDs (see Chapter IV or [80]). The resulting network function takes the following s-expanded form:

$$H(s) = \frac{a_m s^m + ...a_1 s + a_0}{b_n s^n + ...b_1 s + b_0}, \tag{7.5}$$

where each coefficient a_i or b_j in the numerator and denominator is represented by a coefficient DDD. The numerical value of each coefficient can be readily computed by EVALUATE operation on each coefficient DDD once.

For a practical analog circuit, we note that the numerical contributions of the terms $x_i s^i$ in numerators or denominators are not same and depend on frequency. Commonly, for a limited frequency range, terms with high-order powers of s can be ignored without affecting the transfer functions under consideration. For instance, in the $\mu A741$ Opamp circuit, the highest power of s in the denominator of the voltage gain is 22. Its coefficient has the numerical value in the magnitude of 10^{-211}. If the frequency range is up to the unity-gain frequency (about 1MHz), the numerical value of denominator polynomial is about 10^{-60}, while the contribution from the term $a_{22}s^{22}$ in the denominator at 1MHz is in the magnitude of 10^{-79}, which is substantially smaller than 10^{-60} and therefore should be ignored.

In particular, we begin from the lowest frequency point. At each frequency point, we select all the terms $x_i s^i$ in a polynomial $P(s)$ (the numerator or the denominator of a transfer function), such that $|x_i s^i| > \xi |P(s)|$, where ξ is a real number and $0 < \xi < 1$. We then compare the simplified transfer function against the unsimplified one at the same frequency point. If the errors are bounded by user-specified

error margins in terms of magnitude and phase, we proceed with the next frequency point; otherwise we reduce ξ to select more terms from the polynomial until the errors caused by the simplified transfer function are bounded by the error margins. Note that after the first two approximation procedures, some higher-order terms still can be suppressed given a frequency range and error margins as shown in Table 17.

7.5 Elimination of Canceling Terms

Term cancellation in the framework of determinant expansion comes from the MNA formulation used and device matching in analog circuits. For instance, consider the s-expanded DDD in Figure 19. Since $g = k = \frac{1}{R_3}$ and $i = j = -\frac{1}{R_3}$, term agk cancels term $-aji$ in the coefficient DDD of s^0.

Symbolic canceling terms will greatly degrade the efficiency of our dominant-term generation algorithm (described in Section 7.6). In practice, a majority of expanded terms may cancel each other due to the MNA formulation. Removing canceling terms has been known as a difficult problem in determinant-based symbolic analysis methods [30, 66].

We consider several matrix patterns shown in Figure 51. For example, case

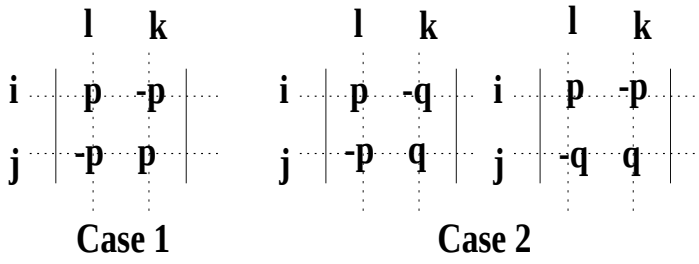


Figure 51: The matrix patterns causing term cancellation

1 may come from the rectangular appearance of a floating resistor in the nodal admittance formulation. Case 2 may come from a floating resistor in parallel with

controlled and controlling terminals in a controlled sources (like voltage-controlled voltage source).

Let L_1, L_2, L_3, L_4 denote, respectively, unique DDD symbols at four rectangular positions (i,l), (i,k), (j,l), (j,k). Then we show the following result:

Proposition 4 *For each product term containing L_1 and L_4, there exist a corresponding canceling product term containing L_2 and L_3.*

Proof. First, we give the definition of a determinant $det(\mathbf{A})$ as follows:

$$det(\mathbf{A}) = \begin{vmatrix} a_{1,1} & a_{1,2} & ... & a_{1,n} \\ a_{2,1} & a_{2,2} & ... & a_{2,n} \\ ... & ... & ... & ... \\ a_{n,1} & a_{n,2} & ... & a_{n,n} \end{vmatrix} = \sum_{j_1 \neq j_2 \neq ... \neq j_n} (-1)^p \cdot a_{1,j_1} \cdot a_{2,j_2} ..... a_{n,j_n}, \qquad (7.6)$$

where $(j_1, j_2, ..., j_n)$ is a permutation of column indices of $\mathbf{A}$, and p is the number of permutations needed to make the sequence $(j_1, j_2, ..., j_n)$ monotonically increasing. Note that a valid product term consists of a combination of the nonzero matrix entries such that all the row indices and column indices in $\mathbf{A}$ appear just once in those nonzero entries. We also see that both L_1, L_4 and L_2, L_3 contain the same row indices i, j and column indices l, k. As a result, if there is a valid product term g which contains L_1 and L_4, there must exist a product term g' which contains L_2, L_3 and all the other matrix entries in g except L_1 and L_4 according to the conditions for a valid product terms. So the products of all matrix entries in g and g' are same as $L_1 \cdot L_4 = L_2 \cdot L_3$. Meanwhile, g and g' have the opposite signs, as we can obtain the column permutation of g' by one permutation on that of g. This completes the proof. □

Based on Lemma 4, the algorithm for eliminating all canceling terms due to case 1 and case 2 is described in Figure 52. From steps 1 to 6, the algorithm finds all the terms containing L_1 and L_4 and also containing L_2 and L_3 according to Lemma 4. After the two COFACTOR operations, all the matrix entries L_1, L_2, L_3, L_4 have been

```
DECANCEL(D, L1, L2, L3, L4)
1   Dc = COFACTOR(D, L1)
2       if ( Dc = Empty )
3           return Empty
4   Dc = COFACTOR(Dc, L4)
5       if ( Dc = Empty )
6           return Empty
7   S1 = MULTIPLY(Dc, L1)
8   S1 = MULTIPLY(Dc, L4)
9   S2 = MULTIPLY(Dc, L2)
10  S2 = MULTIPLY(Dc, L3)
11  D = SUBTRACT(D, S1)
12  D = SUBTRACT(D, S2)
13  return D
```

Figure 52: Algorithm for symbolic canceling term elimination

removed from all the resulting product terms. So the algorithm rebuilds all these canceling terms in steps 7 to 10 by using MULTIPLY operations. Finally, all the these canceling terms are subtracted from the original DDDs by SUBTRACT operations. As an illustrative example, the cancellation-free DDD is shown in Figure 53 with 13 paths. Matrix patterns involving more than two rows (columns) can also cause term cancellation. These term-cancellation situations can easily be detected in the dominant-term generation procedure (see Section 7.6) and the canceling terms can be removed according to the same principle (but with 3 or more COFACTOR and 6 or more MULTIPLY operations). Our experimental results indicate that most canceling terms can be removed effectively by just considering cases 1 and 2.

We note that operation $D_{\overline{P}}$ =SUBTRACT(D, P) may increase the size of the D although D contains more product terms than $D_{\overline{P}}$. The reason is that some sharing in D may be destroyed by removing all the terms in P. For instance, in

the experimental results shown in Table 17, after the decancellation procedure, the DDD size for Opamp μ741 and Cascode actually increases (row 18 and row 20). The change in the DDD size, however, is not very significant in general (no more than twice in both circuit examples). So we can still safely assume that the size of a DDD, after SUBTRACT operations, $|DDD|$, is still $O(|DDD|)$. Since the number of devices a node connects typically is limited by a constant for practical analog circuits, each device will produce a limited number of canceling matrix patterns (also bounded by the constant). Hence the number of canceling matrix patterns due to case 1 and case 2 is also $O(n)$, n is the number of devices in a circuit. So the time complexity of removing all the canceling terms due to case 1 and case 2 is $O(nl|DDD|)$ for a network function, where l is the total number of coefficients in both the numerator and the denominator of the network function.

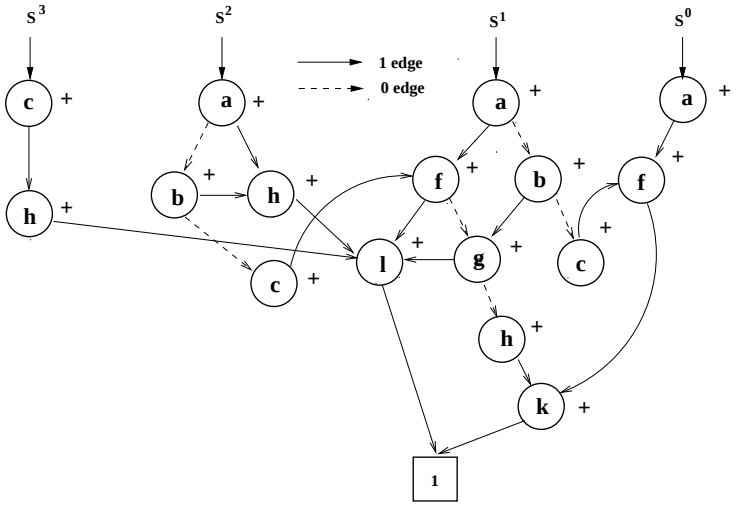


Figure 53: A cancellation-free s-expanded DDD

7.6 Generation of Dominant-Terms

Many small-signal characteristics are dominated by a small number of product terms called *significant* or *dominant-terms*, which are not negligible compared to the magnitude of the numerator or the denominator containing them. In our framework, the extraction of significant product terms can be transformed into the problem of finding k shortest paths in a DDD.

We need to introduce the notion of path weight in a DDD.

Definition 1 *The cost of a path in a DDD is defined to be the total cost of the edges along the path where each 0-edge costs 0 and each 1-edge costs $-log|a_i|$, and $|a_i|$ denotes the numerical value of the DDD vertex a_i that originates the corresponding 1-edge.*

We then show the following result:

Proposition 5 *The most significant product term in a symbolic determinant D corresponds to the minimum cost (shortest) path in the corresponding DDD between the root and the 1-terminal.*

Proof. Each 1-path in a DDD is defined as a path from the root of the DDD to 1-terminal. It designates a product term consisting of all the symbols represented by the DDD vertices that originate 1-edge. So given a 1-path in a DDD and its corresponding product term p, $-log(|p|)$ is exactly the path weight defined in Definition 1 for this 1-path. If p is the most significant product term, i.e., $|p| \geq |p_i|$, where p_i is any other product except p in the DDD, we obviously have $-log(|p|) \leq -log(|p_i|)$. So the 1-path representing p is the shortest path from the root of the DDD to 1-terminal. □

By definition, a determinant decision diagram is also a directed acyclic graph (DAG) with multiple sources and two sinks. For a DAG, the shortest path can be efficiently found by two depth-first searches in time $O(|V| + |E|)$ [14], $|V|$ is the number of vertices and $|E|$ is the number of edges in the DAG. In terms of DDDs,

the time complexity is $O(|V|)$, where $|V|$ is the size of DDD, since $|E|$ equals $2|V|$ in DDDs. A nice property of DDD is that after we find the shortest path from a DDD, we can subtract it from the DDD using a SUBTRACT operation (see Table 1 in Chapter III or [79]), and then we can find the next shortest path in the resulting DDD. If we still assume that DDD size will be $O(|DDD|)$ before and after all the SUBTRACT operations, we can find the k shortest paths in time $O(k|DDD|)$.

This procedure can be performed on the s-expanded DDD, after the decancellation procedure. We note that this approach also handles numerical cancellation due to device parameter match. Since numerical canceling terms are extracted one after another, they can be eliminated by examining two consecutive terms.

7.7 Error Control Scheme

There are various mechanisms to control how error is introduced to approximating expressions [30, 69, 90, 97]. The selection of an appropriate error control scheme, however, depends on the types of circuits and the applications of the simplified expressions. In the proposed framework, we monitor both magnitude and phase of the simplified expressions at each step. Our method is yet flexible enough to incorporate other error control schemes. Except for the decancellation step, all the other approximation steps will introduce new error into the simplified expressions. The total error is an accumulation of all the errors introduced by each approximation step. Here we discuss in detail the error control scheme for dominant-term generation.

To make the simplified expression valid for a range of frequencies while keeping the number of the generated product terms as small as possible, we use an error control scheme similar to that in coefficient suppression step in Section 7.4 and also used in [97]. At each frequency point $s_i (s_i = 2\pi f_i)$, we select all the product terms

from all the coefficient DDDs such that the selected product terms satisfy

$$\alpha|P(s_i)| < |t_k \cdot s_i^k|, \;\; k = 0, ..., m, \tag{7.7}$$

where $|P(s_i)|$ is the magnitude of $P(s_i)$, and α is an error control threshold with $0 < \alpha < 1$ and t_k is a product term generated from the coefficient of s^k, and m is the highest power of s. We begin the dominant-term generation procedure with initial α, and dynamically adjust the value of α until the generated terms from different coefficient DDDs are accurate enough in terms of magnitudes and phases. It is worth pointing out that such an error control is more amenable to our DDD-based dominant-term generation algorithm than to the spanning tree enumeration algorithm [97]. A simple shortest-path algorithm is used in our method on all the coefficient DDDs to generate the most significant product terms. In contrast, the same dominant-terms in the coefficient of particular power of s have to be found by solving a complicated color-constrained spanning-tree-enumeration problem [97], which is a NP-compete problem.

7.8 Experimental Results

The proposed approximation method has been implemented in our symbolic analyzer SCAD3. Here we describe results for three integrated circuit examples *TwoStage*, *Cascode*, $\mu A741$. For each circuit, DC analysis was carried out using SPICE, and our program read in small-signal element values from the SPICE output. The algorithms described in Chapter III and IV were used to construct complex DDDs and s-expanded DDDs.

7.8.1 TwoStage Operational Amplifier

The first example is a simplified two-stage CMOS operation amplifier from [27] in Figure 54. The transfer function derived for *TwoStage* is open-loop voltage gain from input node V_{in2} (positive input) to output node V_{out} with node V_{in1} short-

circuited to ground. The simplified expression given by our program is:

$$G(s) = \frac{V_{out}}{V_{in2}} = \frac{g_{m2}g_{m6} + s^1(g_{m2}CC)}{(r_{o2} + r_{o4})(r_{o6} + r_{o7}) + s^1(g_{m6}CC) + s^2CC(c_{db6} + CL)}.$$

This expression is valid beyond the unity-frequency point of the Opamp circuit as

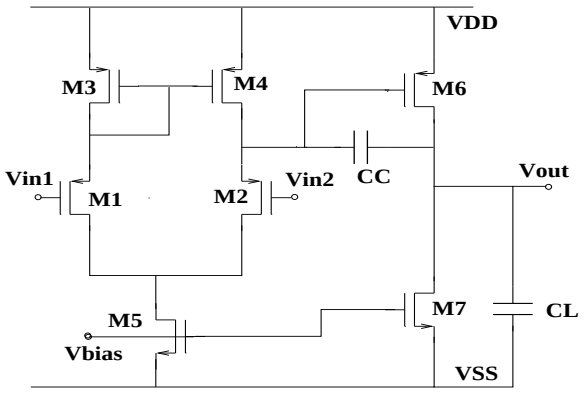


Figure 54: A simplified two-stage CMOS Opamp

shown in Figures 55 and 56. The statistics in each simplification step are summarized in Table 17.

Exact symbolic analysis shows that *two-stage* actually has three zeros and three poles in its voltage gain transfer function as shown in Table 16.

Table 16: Zeros and poles for two-stage Opamp

poles	$-1.68 * 10^7$	$-8.80 * 10^5$	-373.74
zeros	$3.18 * 10^9$	$-1.68 * 10^7$	$1.11 * 10^7$

Since all three poles are far away from each other, the pole splitting method can be used to find the three poles and zeros. We note, however, that if we apply the pole splitting method directly on the simplified expression of the transfer function as

done in many previous approaches, we can't obtain the third pole. This is because the third pole and second zero cancel each other, and the third pole information is lost during the simplification. On the other hand, by using the pole splitting method directly on the exact expression of the transfer function represented as s-expanded DDDs, we can obtain the simplified expression for the third pole as shown below:

$$-\frac{g_{m3}}{c_{gd1}+c_{db1}+c_{gs3}+c_{db3}+c_{gs4}} = -1.67 * 10^7.$$

This is in agree with the exact third pole described in Table 16.

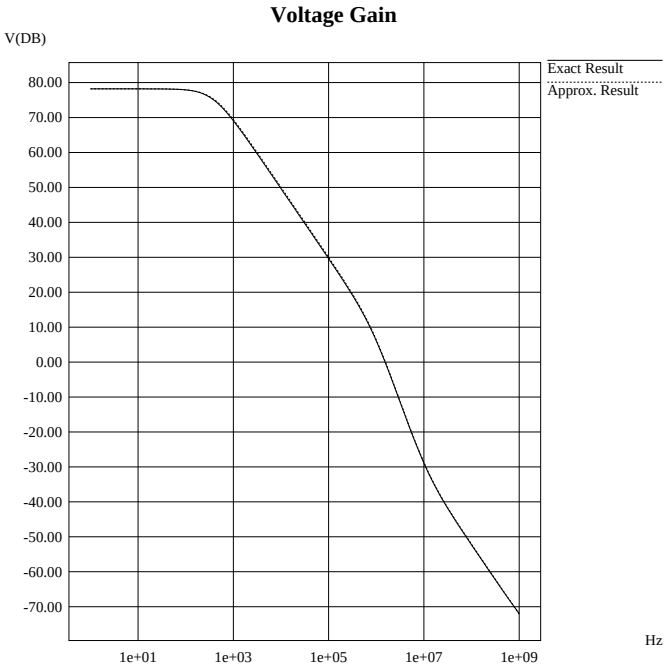


Figure 55: Voltage gains of two-stage Opamp in exact and simplified expressions by the proposed algorithm

7.8.2 Uncompensated CMOS Cascode Operational Amplifier

The second example is an uncompensated CMOS Cascode Opamp shown in Figure 15 with 22 transistors. The simplified expression that is valid from DC point to unity-gain frequency for both voltage gain and phase contains total 83 product

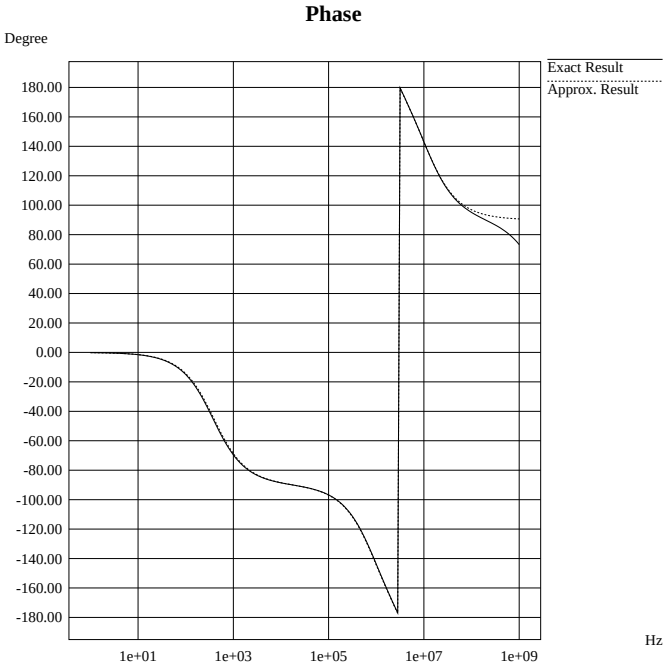


Figure 56: Phases of two-stage Opamp in exact and simplified expressions by the proposed algorithm

terms. Figures 57 and 58 plot the approximate results against exact results. The numerical results from the approximate expression from **Rainier** [97], containing a total of 784 product terms, are also compared against ours. The statistics of the symbolic expressions are shown in Table 17. The whole simplification process takes 11.3 seconds in the Sun Ultra-I Workstation.

Figure 59 shows the distributions of number of product terms with respect to different powers of s in the exact symbolic expression, the expression after device elimination and node contraction, and the expression after decancellation process in the denominator of transfer function of Cascode Opamp.

7.8.3 $\mu A741$ Operational Amplifier

The third circuit is the bipolar operation amplifier $\mu A741$ containing 26 transistors and 11 resistors (Figure 14). The numerical results from the simplified expression

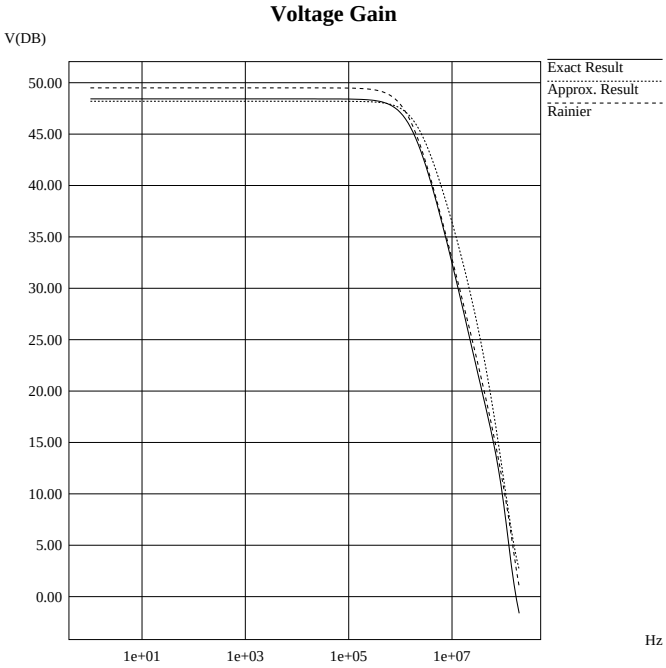


Figure 57: Accuracy comparison in voltage gain for CMOS Cascode Opamp

that contains a total of 32 terms and is valid up to unity-gain frequency are shown in Figures 60 and 61. Again the numerical results from the simplified expression that contains 89 terms, by **Rainier** , are also compared against ours. The whole simplification process takes 54.7 seconds in the same workstation.

Figure 62 shows the distributions of number of product terms with respect to different powers of s in the exact symbolic expression, the expression after the first two simplification steps and the expression after decancellation process in the denominator of transfer function of $\mu A741$ Opamp.

7.8.4 Discussions

In this subsection, we give a detailed elaboration on Table 17. It is organized according to the order by which the approximation steps were carried out in our algorithm. Row 3 to row 9 first summarize the statistics about the circuits, the

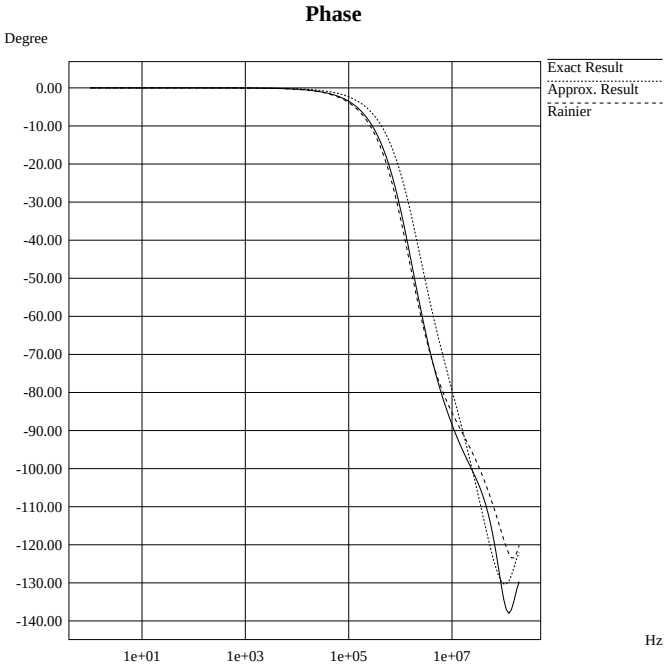


Figure 58: Accuracy comparison in phase for CMOS Cascode Opamp

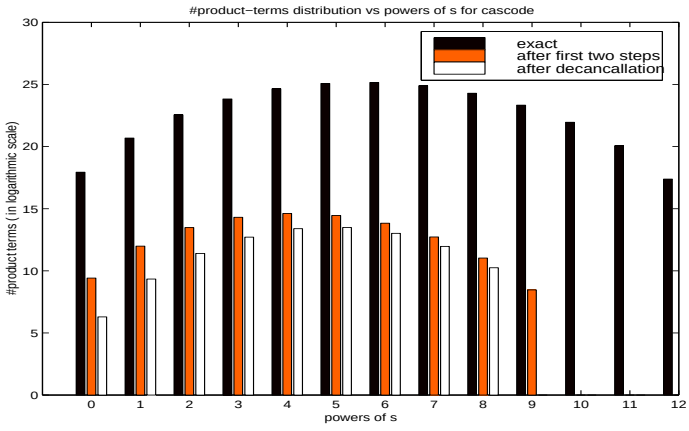


Figure 59: #terms distribution vs powers of s for Cascode

complex DDD and s-expanded DDD in exact symbolic transfer functions. We can observe that DDD is highly compact, and the number of vertices is many orders of

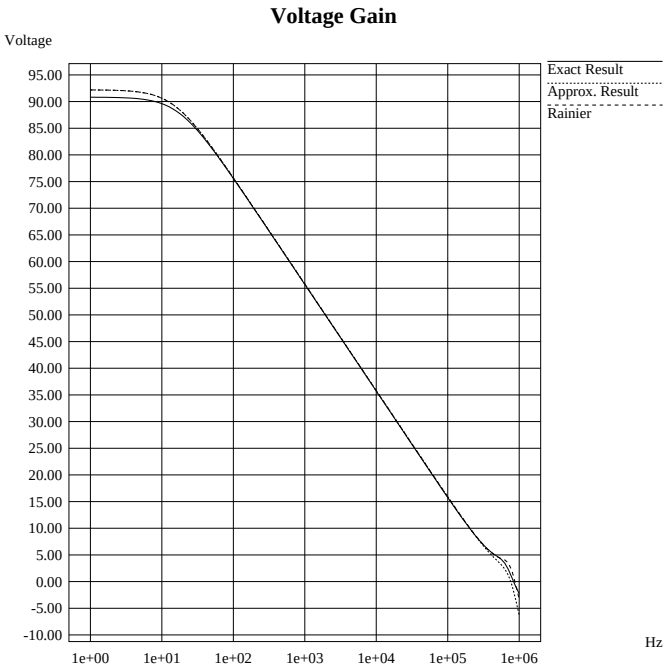


Figure 60: Accuracy comparison in voltage gain for $\mu A741$

magnitude less than the number of product terms. For example, the denominator of the s-expanded DDD for $\mu 741$ Opamp has $6.40 * 10^{19}$ product terms, but the entire DDD to represent both the denominator and numerator contain only 297117 vertices (this is for the complete and exact transfer function)!

We then performed device removal and node contraction based on the complex DDD representation. The results are summarized in row 10 to row 13. We observe that this step removes significantly the number of devices (mainly the bias circuitry not in the signal path.) and also reduces significantly the size of complex DDDs.

Next, we constructed s-expanded DDDs from simplified complex DDDs. Note that even after device elimination and node contraction, the number of product terms by the matrix determinant method is still in the range of millions. We then perform the following three simplifications:

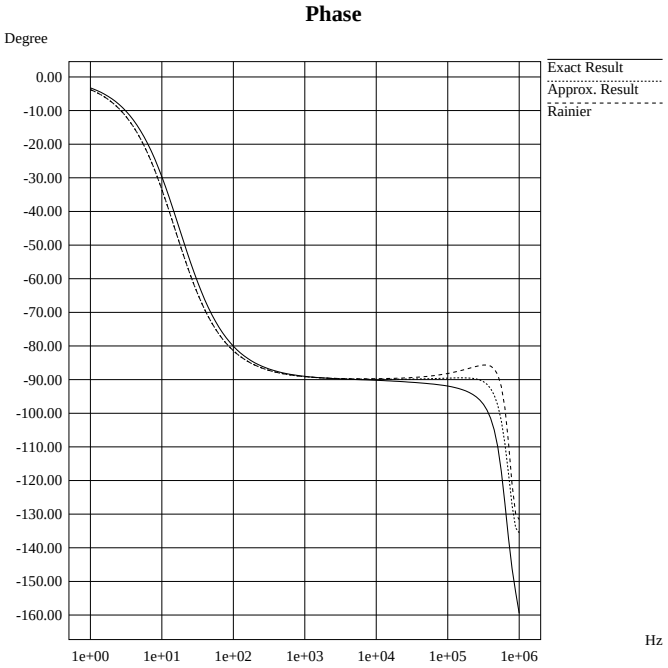


Figure 61: Accuracy comparison in phase for $\mu A741$

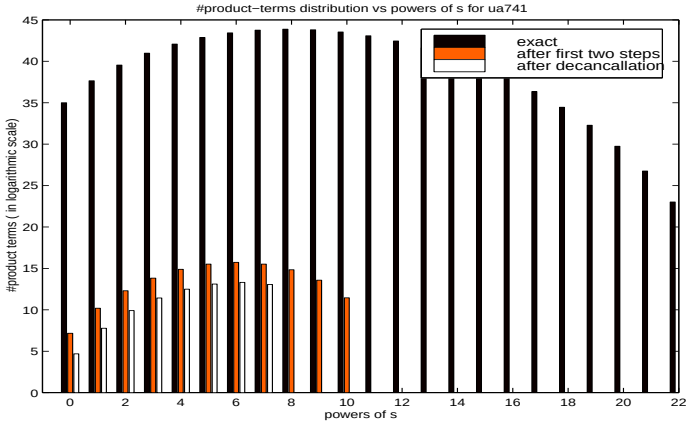


Figure 62: #terms distribution vs powers of s for $\mu A741$

First, we suppressed those insignificant high order terms and the results are shown in row 16 to row 18. This reduces the size of each DDD by about 10 precept. Second, we performed decancellation. The results are shown in row 19 to row 22. We

see that over 80 percent terms are canceling terms. Finally, we extracted the significant terms from the resulting s-expanded DDD. For *TwoStage*, *Cascode* and $\mu A741$, the number of product terms in the final simplified transfer functions (including both the numerator and denominator) are respectively 6, 83, and 71. On the other hand, the expressions from **Rainier** for *Cascode* and $\mu A741$ contains, respectively, 784 and 89 product terms. The results from **Rainier** for *Cascode* and $\mu A741$ are also plotted for comparison in Figures 57, 58, 60, and 61, respectively.

7.9 Summary

A new approach is proposed to automatically generate interpretable expressions for small-signal characteristics of large analog circuits. Unlike simplification-during-generation approaches, the proposed approach works on a complete yet exact representation of transfer functions. It can monitor the errors in the magnitude, phase, pole and zero of the simplified expressions, and thus provides a reliable and robust simplification scheme. The proposed approach has a time and space complexity that is many orders-of-magnitude less than the simplification-after-generation approaches. It provides, for the first time, a framework for deriving not only interpretable network functions, but also interpretable expressions for other small-signal characteristics such as poles, zeros and sensitivities. The proposed approximation approach has demonstrated a superior performance over other state-of-the-art tools.

Table 17: Statistics for simplified symbolic expressions

#row	Circuit	Twostage		Cascode		$\mu A741$	
2		Num	Den	Num	Den	Num	Den
3	#lumped devices	14		84		111	
4	#nodes in the circuit	5		14		24	
5	#terms in exact complex DDDs	1	3	9437	28218	381526	$3.82 * 10^6$
6	#vertices in exact complex DDDs	12		1406		9572	
7	#terms in exact s-expanded DDDs	16	74	$2.57 * 10^{10}$	$3.60 * 10^{11}$	$8.98 * 10^{16}$	$6.40 * 10^{19}$
8	#vertices in exact s-expanded DDDs	52		18109		297117	
8	Highest power of s in exact expressions	3	3	12	12	22	22
10	#terms after device elimination in complex DDDs	1	2	800	1752	816	9338
11	#devices eliminated	6		34		68	
12	#nodes contracted in complex DDDs	1		3		8	
13	#terms after contraction in complex DDDs	1	2	33	107	84	516
14	#terms after expansion in s-expanded DDDs	2	11	$4.28 * 10^5$	$8.04 * 10^6$	$2.06 * 10^5$	$2.56 * 10^7$
15	Highest power of s after expansion	1	2	7	9	9	10
16	Highest power of s after suppression	1	2	7	8	5	7
17	#terms after suppression in s-expanded DDDs	2	11	$4.28 * 10^5$	$8.04 * 10^6$	$1.52 * 10^5$	$2.19 * 10^7$
18	#vertices after suppression in s-expanded DDDs	18		1873		2641	
19	#terms after decancellation in s-expanded DDDs	2	9	$2.00 * 10^5$	$2.43 * 10^6$	23162	$1.95 * 10^6$
20	#vertices after decancellation in s-expanded DDDs	17		2500		4303	
21	#canceling terms eliminated due to case 1	2		$3.25 * 10^6$		$1.68 * 10^7$	
22	#canceling terms eliminated due to case 2	0		$2.58 * 10^6$		$3.31 * 10^6$	
23	#terms in final expression	2	4	21	62	12	59
24	Highest power of s in final expression	1	2	4	6	3	4
25	#lumped devices in each product term	2		9		14	

CHAPTER VIII SYMBOLIC CIRCUIT-NOISE ANALYSIS AND MODELING

8.1 Introduction

Noise behavior is an important characteristic of analog circuits, as it usually determines the fundamental performance limits of an analog circuit or system. Numerical noise analysis of analog circuits in the DC steady state can be carried out efficiently using the adjoint method [65]. For system level noise simulation, the adjoint method, however, still can't offer adequate efficiency. Hierarchical noise analysis becomes an attractive alternative. The noise model in terms of noise spectral density is first constructed for each analog block, and then noise analysis at the system level is carried out using the generated noise models for each block. Algorithms to generate approximate noise models have been reported by using numerical order-reduction techniques [21] and by using symbolic approximations [30].

In this chapter, we present a new approach to deriving the exact noise models by utilizing DDDs and s-expanded DDDs introduced in the previous chapters. Symbolic noise analysis essentially amounts to computing a number of transfer functions due to different noise sources. Such an application scenario is extremely amenable to DDD-based symbolic analysis, since the sharing of symbolic expressions among different transfer functions can be exploited efficiently by determinant decision diagrams. Hence, the size of an analog circuit that can be handled by DDDs is much larger than that by other symbolic analyzers. In addition, the new method offers exact noise models of circuit blocks, which contrasts with the aforementioned existing methods that only generate approximate noise models.

The new noise modeling technique consists of the computation of symbolic transfer functions and the subsequent generation of the noise spectral density for analog circuits. Experimental results show that the cost of obtaining all the transfer functions required by a noise model almost equals that of computing a single transfer function.

The rest of the chapter is organized as follows: Section 8.2 reviews device noise models and symbolic noise analysis. Section 8.2.3 presents our new circuit-noise analysis and modeling technique. Section 8.3 presents some experimental results. Section 8.4 summarizes this chapter.

8.2 Noise Models and Symbolic Noise Analysis

8.2.1 Noise Models

Noise in integrated circuits is caused by some random physical phenomena which lead to small current and voltage fluctuations within circuit devices. Mathematically, noise is characterized in terms of mean and autocorrelation in time domain, or power spectral density in the frequency domain. The integration of the noise power spectral density over frequency gives the total noise power. The most important noise sources in integrated circuit devices are *thermal noise*, *shot noise* and *flicker noise* [32]

- *Thermal noise* is also called white noise due to its independence of frequency. It is caused by the thermal motion of electrons. The spectral density function of thermal noise, $i_{th}^2/\Delta f$, can be given by

$$i_{th}^2/\Delta f = \frac{4KT}{r}, \tag{8.1}$$

  where K is Boltzman's constant ($1.38 \times 10^{-23} JK^{-1}$), T is the temperature in Kelvins, and r is the resistance value.

- *Shot noise* is caused by the current through a PN junction consisting of discrete charge carriers randomly crossing a potential barrier. The noise spectral density

of shot noise, $i_{sh}^2/\Delta f$, can be given by

$$i_{sh}^2/\Delta f = 2qI_d, \tag{8.2}$$

where q is the electron charge and I_d the average junction current.

- *Flicker noise* (or 1/f noise) can be found in all active devices. The origins of flicker noise vary in different devices and are not well understood. The spectral density of flicker noise, $i_{fl}^2/\Delta f$, is given by

$$i_{fl}^2/\Delta f = K_{device}\frac{I^a}{f}, \tag{8.3}$$

where K_{device} is a constant for a particular device and a is a constant in the range from 0.5 to 2.

In analog integrated circuits, resistors, bipolar junction transistors (BJTs) and MOS transistors are all noisy devices. Each of these devices may include several noise sources due to different physical phenomena, and some of noise sources are lumped into one noise source for convenience. The widely used noise models for these devices are shown in Figure 63 [32, 61]. The noise source in a resistance r is given by

$$i_r^2 = \frac{4KT}{r}\Delta f. \tag{8.4}$$

The lumped noise sources in a BJT are given by

$$i_{rb}^2 = \frac{4KT}{r_b}\Delta f, \tag{8.5}$$

$$i_b^2 = 2qI_b\Delta f + K_{bjt}\frac{I^a}{f}\Delta f, \tag{8.6}$$

$$i_c^2 = 2qI_c\Delta f, \tag{8.7}$$

where r_b is due to the thermal noise of the base resistance and I_b and I_c are, respectively, the base current and collect current of the BJT, and K_{bjt} is a constant dependent on device properties of the BJT. The lumped noise source in an MOS

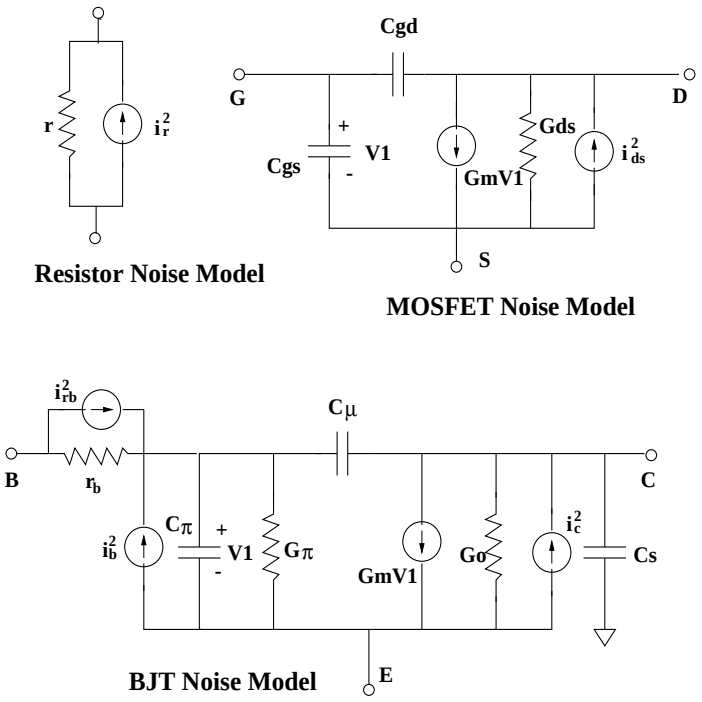


Figure 63: Noise models for common IC devices

transistor is given by

$$i_{ds}^2 = 4KT(2/3)g_m\Delta f + K_{mos}\frac{I^a}{f}\Delta f, \tag{8.8}$$

where the first term $4KT(2/3)g_m\Delta f$ is the thermal noise in the channel resistance and the second term $K_{mos}\frac{I^a}{f}\Delta f$ is the flicker noise in the transistor and K_{mos} is a constant dependent upon the device characteristics.

8.2.2 Symbolic Noise Analysis

Since all the noise sources are uncorrelated, their contributions at an output have to be calculated separately. Let $H_j(s)$ denote the transfer function from noise source i_j^2 to an output; then the noise voltage in the root mean square (rms) form at the output is given by

$$V_{out}(s) = \sqrt{\sum_{j=1}^{n} |H_j(s)|^2 i_j^2}. \tag{8.9}$$

$V_{out}^2(s)/\Delta f$ is called the noise spectral density. With this, computing the noise spectral density function of an output essentially amounts to the symbolic computation of a set of transfer functions with different inputs and the same output, and consequent squaring operations on each transfer function and summation of the results. For practical circuits, however, one transfer function is already very lengthy; a number of symbolic functions along with the squaring operations will lead to a huge number of product terms in the resulting s-expanded expressions [30, 31]. Consequently, noise analysis in the symbolic analyzer ISAAC [30] is limited to only very small analog circuits.

Note that the adjoint method [65] used in numerical noise analysis also requires the same number of symbolic transfer functions and thus shows no advantage over the straightforward calculation method.

8.2.3 New Circuit-Noise Modeling Method

We now introduce a new symbolic noise analysis and modeling method. It consists of two steps: (1) deriving the symbolic transfer function of each noise source, and (2) computing the noise spectral density function using s-expanded DDDs and algebraic operations.

8.2.4 Symbolic Transfer Function Computation

For a linear(ized) time-invariant analog circuit with a DC steady state, we can use the modified nodal analysis formulation to describe its circuit equations as follows:

$$\mathbf{Tx} = \mathbf{w}, \tag{8.10}$$

where $\mathbf{x}$ is a vector of the node voltage variables and branch current variables, $\mathbf{T}$ is the nodal admittance matrix and $\mathbf{w}$ represents the independent noise sources. Because all the noise sources are modeled as current sources, we only need to compute the transresistance function defined as the ratio of voltage at an output port to the current

at an input port. As an illustration, consider the 2-port network in Figure 64. The

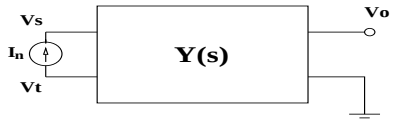


Figure 64: A network with a single noise source

excitation vector $\mathbf{w} = \{0, .., I_n, ..., -I_n, ..., 0\}^T$ has only two nonzero elements in the row s and t. According to Cramer's rule, the transfer function from input current source I_n to the output V_o can be expressed as

$$H(s) = \frac{V_o}{I_n} = \frac{(-1)^{(s+o)} det(T_{so}) - (-1)^{(t+o)} det(T_{to})}{det(T)}, \tag{8.11}$$

where T_{so} is the matrix obtained by deleting row s and column o in T. We note also that all the transfer functions have the same denominator, $det(T)$, but different numerators consisting of different first-order cofactors of $det(T)$. So computing all the trans-resistance functions from various inputs to an output is equal to representing $det(T)$ and a number of its first-order cofactors. DDDs are extremely efficient to represent the expressions in a determinant and its cofactors due to the sharing of subexpressions among them and the exploitation of such sharing by DDDs.

We illustrate this multi-function computation process by means of a simple RC filter circuit shown in Figure 65. The noise sources of noisy resistors in the circuit are also marked in the figure. The circuit matrix of the RC filter, by ignoring all the noise sources, can be expressed by

$$\begin{array}{c} v_3 \\ v_2 \\ v_1 \end{array} \begin{bmatrix} \frac{1}{R_3} + sC_3 + \frac{1}{R_2} & -\frac{1}{R_2} & 0 \\ -\frac{1}{R_2} & \frac{1}{R_2} + sC_2 + \frac{1}{R_1} & -\frac{1}{R_1} \\ 0 & -\frac{1}{R_1} & \frac{1}{R_1} + sC_1 \end{bmatrix}.$$

If we view each entry as one symbol, the resulting system determinant T is shown

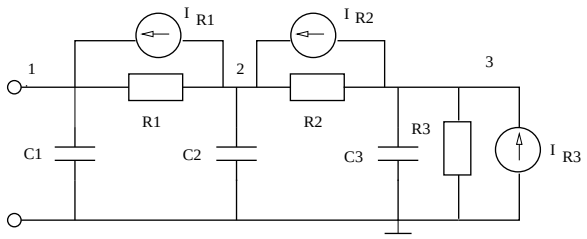


Figure 65: A RC circuit example

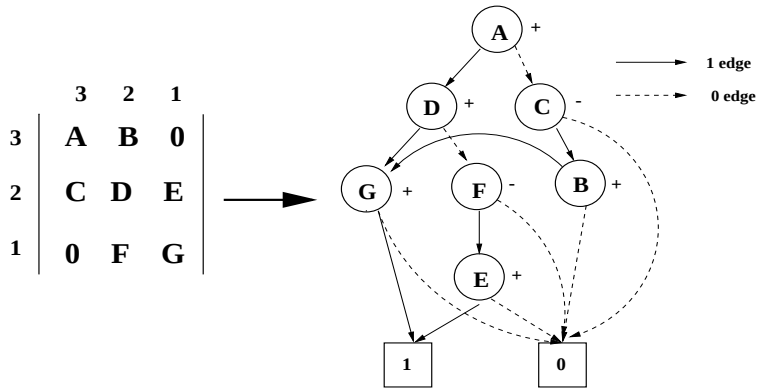


Figure 66: A matrix determinant and its DDD

in the left-hand side of Figure 66. For each noise source, we have a transfer function:

$$
\begin{aligned}
H_{R_1}(s) &= \frac{V_3}{I_{R_1}} = \frac{(-1)^{(1+3)}det(T_{13}) - (-1)^{(2+3)}det(T_{23})}{det(T)}, \\
H_{R_2}(s) &= \frac{V_2}{I_{R_2}} = \frac{(-1)^{(2+3)}det(T_{23}) - (-1)^{(3+3)}det(T_{33})}{det(T)}, \\
H_{R_3}(s) &= \frac{V_3}{I_{R_3}} = \frac{(-1)^{(3+3)}det(T_{33})}{det(T)}.
\end{aligned}
$$

Note that, in addition to the system determinant $det(T)$, we also need three first order minors of $det(T)$: $det(T_{13}) = BE$, $det(T_{23}) = BG$ and $det(T_{33}) = DG - FE$. In fact, minor $det(T_{33})$ and $det(T_{23})$ already exist in $det(T)$ as shown in Figure 67. To represent $det(T_{13})$, we need two extra DDD vertices. So we end up with only 9

vertices to represent all the transfer functions required for the output noise spectral density for this RC filter circuit.

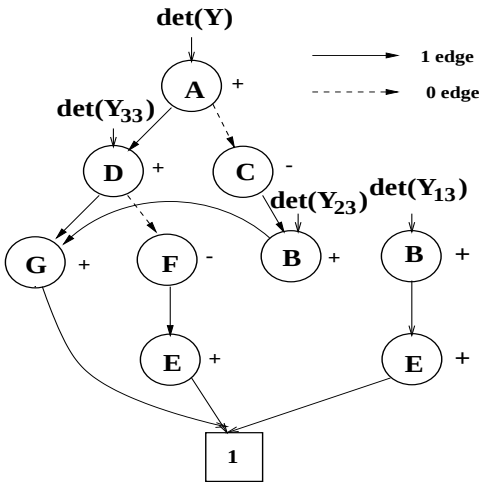


Figure 67: DDDs of representing noise transfer functions

8.2.5 Generations of Noise Spectrum Density Functions

To improve the efficiency of noise analysis in system-level noise simulation, hierarchical noise simulation is adopted, where the noise model in terms of noise spectral density function in frequency f, $V_j^2(f)/\Delta f$, is first constructed for each analog block j. The noise spectral density in turn is used as the noise source for the circuit block at higher-level noise simulation. Since we can obtain the exact symbolic expressions for each transfer function, it is possible to derive the exact expressions of noise spectral density functions by DDD-graph manipulations and algebraic operations.

After all the transfer functions required for a noise spectral density are computed and represented by DDDs, they are transformed into s-expanded forms using s-expanded DDDs, and each of them takes on the following form:

$$H(s) = \frac{\sum_{i=0}^{m} D_{a_i} s^i}{\sum_{j=0}^{n} D_{b_j} s^j}, \tag{8.12}$$

where D_{a_i} and D_{b_j} are coefficient DDDs.

To obtain the noise spectral density at output port o,

$$v_o^2(s)/\Delta f = (\sum_{j=1}^{n} |H_j(s)|^2 i_j^2)/\Delta f, \tag{8.13}$$

we need to take squaring operation on each transfer function first. To this end, we first compute the numerical values of the coefficients in each transfer function. This can be done using the DDD EVALUATE operation. With the numerical values of the coefficients, $|H_j(s)|^2$ can be easily computed by using simple algebraic operations. The result is a rational function in frequency variable f in the following form:

$$|H(f)|^2 = \frac{\sum_{i=0}^{m} p_{2i} f^{2i}}{\sum_{j=0}^{n} q_{2i} f^{2i}}, \tag{8.14}$$

where p_i and q_j are real numbers. We note that any noise source, i_j^2, in aforementioned circuit devices is either a constant or a reciprocal function in f, i.e., c/f, where c is a constant. Hence $|H_j(f)|^2 i_j^2$ will still be a rational function or can be scaled to a rational function.

After we obtain all the $|H_j(f)|^2 i_j^2$, the noise spectral density function is the sum of all $|H_j(f)|^2 i_j^2$. This can also be easily performed by simple algebraic additions, as the denominators in all the transfer functions, $H_j(f)$, are the same (determinant of the circuit matrix). Hence the noise spectral density function is still a rational function in f.

8.3 Experimental Results

The proposed noise modeling algorithm has been implemented in our symbolic analyzer SCAD3. Experimental results from a set of analog circuits are presented. We took the numerical values of current noise sources from SPICE and fed them into our program. The algebraic operations were performed using Maple-V [11]. Table 18 summarizes the experimental results obtained from SUN Ultra-I workstation with

Table 18: Statistics of noise analysis for analog circuits

Circuits	System $\|DDD\|$	Total $\|DDD\|$	Max f	Sim.(sec)	Spice (sec.)	Speedup
miller	24	45	8	0.14	6.17	44.1
butter	16	34	24	0.30	1.18	3.79
ladder7	22	99	0	0.07	2.67	38.1
tlctest	130	370	4	0.11	2.47	22.5
ccstest	109	298	6	0.09	4.09	45.4
vcstest	121	334	4	0.13	3.65	28.1
ladder21	64	735	6	0.17	8.03	47.2
Cascode	1406	3601	28	0.94	29.29	31.2
$\mu A741$	2357	18491	44	1.05	70.70	67.3
bigtst	642	9653	24	0.25	12.46	49.8
ladder100	301	15351	0	0.62	51.40	82.9
rctree1	211	2229	0	0.28	15.91	56.8
rctree2	302	5239	0	0.38	23.31	61.3

167 MHz clock rate. *System* $|DDD|$ is the number of complex DDD vertices used to represent the system determinant of the circuit. *Total* $|DDD|$ is the total number of complex DDD vertices used to represent all the determinants in all the required transfer functions in the noise spectral density function. *Max f* is the highest power of f in both numerator and denominator of the derived spectral density function which is the rational function in frequency variable f. *Sim. (sec.)* is the CPU time used to evaluate the obtained spectral density function for 10,000 frequency points. *SPICE (sec.)* is the CPU time used by SPICE3f4 to perform the same noise analysis over the same number of frequency points. *Speedup* shows the speedup of proposed method over SPICE in terms of CPU time.

We then performed the system level noise analysis on a state variable filter circuit shown in Fig. 68. The filter circuit consists of four identical operational amplifiers (Opamps), which in turn were implemented by the CMOS Cascode Opamp.

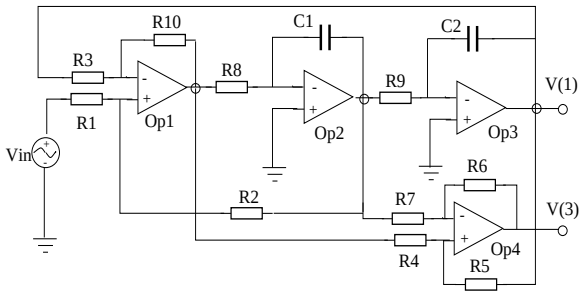


Figure 68: State variable filter circuit

The noise of an Opamp circuit can be modeled by three independent noise sources as shown in the left-hand side of Fig. 69. For an Opamp with MOSFET input stage,

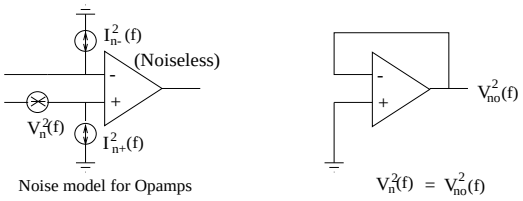


Figure 69: Noise model for operational amplifiers

the two current, $I_{n-}^2(f), I_{n+}^2(f)$, noise sources can be ignored at low frequencies. The voltage noise source, V_n^2, can be obtained by using the circuit configuration shown in the right-hand side of Fig. 69. All the current noise sources in circuit devices were taken from SPICE and fed into our program. The computed exact voltage noise spectral density function for Cascode Opamp is shown as in Equation (8.15).

$$v^2(f)/\Delta f = \frac{\begin{array}{lll} +1.49\times10^{-321}f^{27} & +1.72\times10^{-312}f^{26} & +2.66\times10^{-301}f^{25} \\ +3.11\times10^{-292}f^{24} & +5.83\times10^{-283}f^{23} & +1.10\times10^{-273}f^{22} \\ +6.17\times10^{-265}f^{21} & +1.61\times10^{-255}f^{20} & +3.66\times10^{-247}f^{19} \\ +1.26\times10^{-237}f^{18} & +1.12\times10^{-229}f^{17} & +5.40\times10^{-220}f^{16} \\ +1.45\times10^{-212}f^{15} & +1.13\times10^{-202}f^{14} & +7.39\times10^{-196}f^{13} \\ +9.87\times10^{-186}f^{12} & +8.22\times10^{-179}f^{11} & +3.57\times10^{-169}f^{10} \\ +1.73\times10^{-162}f^{9} & +3.81\times10^{-152}f^{8} & +8.48\times10^{-146}f^{7} \\ -8.68\times10^{-137}f^{6} & +2.27\times10^{-129}f^{5} & +1.47\times10^{-119}f^{4} \\ +5.17\times10^{-114}f^{3} & +2.41\times10^{-104}f^{2} & +2.61\times10^{-99}f^{1} \\ +4.52\times10^{-90} & & \end{array}}{\begin{array}{lll} f\cdot(5.73\times10^{-323}f^{28} & +1.38\times10^{-302}f^{26} & +3.85\times10^{-284}f^{24} \\ +3.78\times10^{-266}f^{22} & +1.42\times10^{-248}f^{20} & +2.52\times10^{-231}f^{18} \\ +4.32\times10^{-214}f^{16} & +4.91\times10^{-197}f^{14} & -1.90\times10^{-180}f^{12} \\ +2.43\times10^{-163}f^{10} & -1.43\times10^{-146}f^{8} & +5.90\times10^{-131}f^{6} \\ +7.18\times10^{-114}f^{4} & +2.13\times10^{-98}f^{2} & +1.21\times10^{-83}) \end{array}} \tag{8.15}$$

We then performed noise analysis on the state-variable filter circuit, where each Opamp circuit is treated as a noiseless circuit with an input voltage noise source (derived above) cascaded at the positive terminal. Each transfer function from a noise source to the output in the filter circuit was computed by the DDD-based hierarchical decomposition method. The resulting transfer functions were expanded into s-expanded forms. We then computed all the numerical coefficients of s^i in each transfer function. The resulting transfer functions are functions in only s or frequency f.

Figure 70 shows the noise spectral density of the state variable filter circuit computed by SPICE and by the proposed DDD-based method. It can be seen that the results are identical. We note that the noise spectral density calculation from the resulting symbolic expression took 0.03 second, while SPICE noise analysis took 1.69 second. We used a Linux platform with an Intel Pentium-II CPU at 450 MHz clock

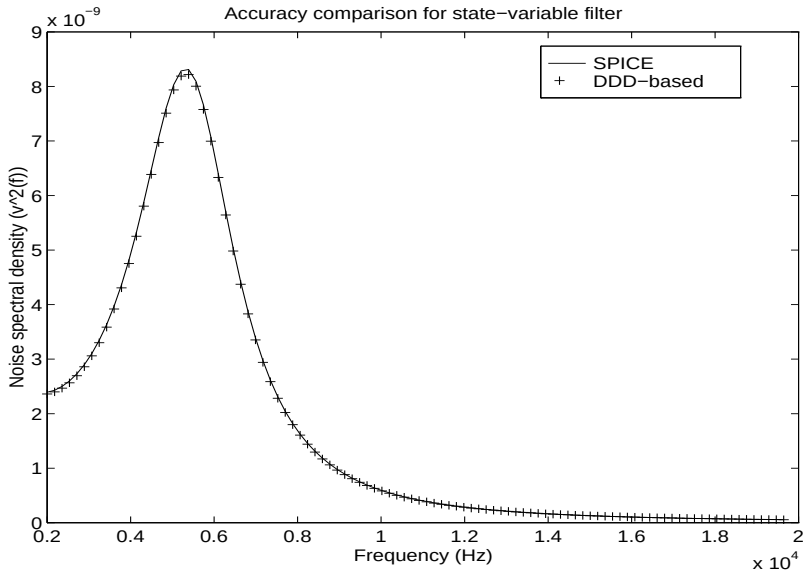


Figure 70: Noise spectral densities by SPICE and DDD

rate, and 1000 frequency points were computed.

8.4 Summary

An efficient symbolic noise analysis and modeling technique has been proposed. It consists of symbolic computations of transfer functions and numerical noise model generations by means of DDD-graph manipulations. In contrast to other noise modeling techniques, the new method is capable of generating exact noise models in an efficient manner. Experimental results have demonstrated the advantage of the proposed method over the simulation-based methods in system level noise simulation.

CHAPTER IX CONCLUSIONS AND FUTURE WORK

Symbolic analysis of analog integrated circuits has presented a challenge for CAD researchers for decades. In spite of many research efforts and increasing industrial demands from design community for the design automation of analog circuits, exact symbolic analysis is still limited to small-size circuits (15 nodes, 30 branches range). Symbolic analysis as an important technique for analog-circuit design automation is still far away from a wide industrial acceptance.

This thesis establishes a new theory and methodology for symbolic analysis based on a new graphical, canonical representation—Determinant Decision Diagrams (DDDs)—for symbolic determinants. The proposed method has significantly increased the generatability of symbolic expressions for large analog integrated circuits. With elegant and powerful DDD-graph manipulations, DDDs enable efficient implementations of behavioral modeling, hierarchical symbolic analysis, small-signal characterization, and noise modeling of analog circuits. In this chapter, the main results of the thesis are first summarized. Research work based on DDDs in several major aspects of design automation of analog circuits is then reviewed. Finally, some promising research topics are pointed out.

9.1 Summary of Research Contributions

9.1.1 DDDs and s-Expanded DDDs

In Chapter III, we formally proposed the concept of determinant decision diagrams for symbolic determinant representation. We showed that a DDD, a variant of binary decision diagrams (BDDs), is the first decision diagram proposed for analog

circuit design automation. Like a BDD, a DDD is ordered, reduced and canonical under a fixed variable ordering. Unlike a BDD, a DDD is a graph with signed vertices and fixed number of 1-edges in any 1-paths from a root vertex to the 1-terminal. Each vertex in the graph representing a multiple-valued variable instead of a Boolean variable.

Using the DDD concept, we developed an algorithm for exact symbolic analysis of analog circuits. Unlike previous approaches based on either the expanded forms or the nested form representations of symbolic expressions, DDD-based symbolic analysis exploits the inherent sparsity of circuit matrices and sharing in the symbolic expressions in a canonical and systematic manner. The power of DDDs is so remarkable that the proposed algorithm enables, for the first time, the exact analysis of such large integrated analog circuits as $\mu A741$ operational amplifier with 26 nodes and 89 branches. Evaluations and manipulations of symbolic determinants (such as sensitivity calculations) have the time complexity linear in number of DDD vertices.

We also studied the relevant vertex-ordering problem in DDDs and proposed an efficient vertex-ordering heuristic. We proved theoretically that the heuristic yields an optimum ordering for ladder-structured circuits, and we observed experimentally that for practical circuits, the numbers of DDD vertices are quite small—usually several orders-of-magnitude smaller than the number of product terms in the s-expanded form. Generating the complete sum-of-product symbolic expressions from the DDD representation offers orders-of-magnitude improvement in both CPU time and memory usages over best-known exact symbolic analyzers for large analog circuits.

In Chapter IV, we presented a new approach to derive s-expanded symbolic expressions by utilizing DDD-graph manipulations. We then introduced a multi-rooted DDD structure, named s-expanded DDD, to represent s-expanded symbolic expressions, with each rooted DDD representing a coefficient of a particular power

of s. We proved that s-expanded DDDs share the same properties of original DDDs (also called complex DDDs). An efficient algorithm was devised for constructing s-expanded DDDs from complex DDDs.

A multi-rooted DDD structure is proposed to represent s-expanded polynomial expressions. We proved that both the time complexity of the construction algorithm and the size of the resulting s-expanded DDD are bounded by $km|DDD|$, where $|DDD|$ is the size of the original DDD representing the determinant of a circuit matrix, m is the highest power of s in the s-expanded expression, and k is the maximum number of circuit parameters in an entry of the circuit matrix. For practical circuits, the time complexity is bounded by $m|DDD|$ since k typically is bounded by a constant.

The power of the s-expanded DDDs for representing symbolic product terms has been demonstrated in one of real analog circuit examples where 10^{19} product terms are represented with less than 3×10^5 DDD vertices. Hence DDDs and s-expanded DDDs have significantly increased the capabilities of exact symbolic analysis, and make possible the modeling and small-signal characterization of many analog circuit blocks typically encountered in real-world mixed-signal circuits.

Fully expanded symbolic expressions can serve as circuit behavioral models used in such applications as system-level simulation, statistical analysis and design centering. Experimentally, we showed that as behavioral models, s-expanded expressions achieve a speedup of about 50 over SPICE3f5 in frequency-domain simulation.

9.1.2 Hierarchical Decomposition and Approximations

Based on the concepts of DDDs and s-expanded DDDs, we further investigated (1) hierarchical symbolic analysis for large analog integrated circuits and (2) interpretable symbolic small-signal characterization of analog integrated circuits.

A new approach to hierarchical symbolic analysis has been presented and im-

plemented in Chapter V. DDD-based hierarchical symbolic analysis consists of performing symbolic suppression of each subcircuit to its terminals in terms of subcircuit matrix determinants and cofactors, and applying Cramer's rule to solve symbolically the set of equations at the top level of the circuit hierarchy. The method takes the advantage of both circuit hierarchy and determinant decision diagrams for symbolic determinant representations.

The related problem of partitioning a circuit so that hierarchical decomposition uses the minimum number of DDD vertices was treated in Chapter VI. Based on a simplified model of the DDD size for dense matrices, we formulated the partitioning problem as a hypergraph partitioning problem and adopted a multi-way tree partitioning strategy with balance constraints. The algorithm is an extension of the iterative refinement algorithm from Fiduccia and Mattheyses [28]. The novelties are the introduction of a new potential gain into the vertex-moving gain and the relaxation of the balance constraints. An efficient clustering-based initial partitioning algorithm was also developed to reduce the dependence of iterative refinement algorithms on initial partitions. Experimental results showed that analog integrated circuits with less structural regularities, such as operational amplifiers, have been successfully partitioned and hierarchically analyzed by our symbolic analyzer SCAD3. We also compared the proposed partitioning algorithm with the method based on the contour-tableau technique [67] and showed the contour tableau-based method may yield very unbalanced partitions. SCAD3 also compares very favorably with the partitioning-based symbolic analyzer SCAPP, and with numerical simulator SPICE for small-signal AC analysis.

In Chapter VII, we addressed the problem of deriving the interpretable symbolic small-signal characteristics of large analog circuits by DDD-graph manipulations. The algorithm consists of several steps to simplify the symbolic expressions. We showed

that two key algorithms of generating interpretable symbolic expressions—term decancellation and generation of dominant-terms—can be performed in linear time with respect to the number of DDD vertices representing the symbolic expressions under some mild assumptions. Unlike simplification-during-generation approaches, the proposed approach works on a complete and exact representation of transfer functions. It can monitor the errors in the magnitude, phase, poles and zeros of the simplified expressions, and thus provides a reliable and robust simplification scheme. The new approximation approach has the time and space complexity many-orders-of-magnitude less than the simplification-after-generation approaches. It provides, for the first time, a framework for deriving not only interpretable network functions, but also interpretable expressions for other small-signal characteristics such as poles, zeros and sensitivities. The new approximation approach exhibits a superior performance over approximation-based symbolic analyzer RAINIER [99] on real analog circuits.

9.1.3 Symbolic Circuit-Noise Analysis and Modeling

We presented a new symbolic noise analysis and modeling technique in Chapter VIII. It consists of symbolic computations of transfer functions and noise model generations by means of DDD-graph manipulations and algebraic operations. It contrasts with existing noise modeling techniques, which either give approximate noise models valid only for a limited range of frequencies by numerical order-reduction technique or else generate simplified symbolic noise models by traditional symbolic analysis methods limited to small circuits. Instead, the new method can generate exact noise models for large analog circuit blocks in an efficient way. It explores the sharing among different transfer functions in a systematic manner. And using the generated noise models, circuit noise analysis can be carried out faster than simulation-based methods, as demonstrated in the experimental results.

9.2 Future Research Topics

Determinant decision diagrams were proposed and studied in this thesis mainly for the symbolic analysis, small-signal characterization and modeling of analog integrated circuits. Its remarkable capability of representing huge numbers of product terms has been exploited and demonstrated. The power of this new technique, however, has not been fully explored, and many promising research topics and application areas still exist and need to be further investigated.

Analog circuit testing and diagnosis, of interest to researchers, have become flourishing research fields because of increasing industrial demands for mixed-signal ICs [4]. Analog circuit fault diagnosis is more complicated than digital circuit diagnosis because analog circuits are usually nonlinear; they have widely spread parameter values and complicated relations between input and output signals. As a result, building fault dictionary requires an extremely large number of simulations. The traditional method is often costly and time-consuming, but it can become very efficient when a symbolic approach is employed. Symbolic approach requires one analysis for circuit topology to generate the network transfer function and a parameter substitution to obtain the frequency response of system. Symbolic analysis has been used in simulation-before-test fault dictionary diagnosis [96]. DDD-based hierarchical decomposition can provide more compact symbolic expressions than existing hierarchical methods; thus a more efficient fault diagnosis method can be developed by utilizing DDD-based hierarchical decomposition.

Computing the testability of an analog network is also a potential application for DDDs. The widely used testability measure of an analog system can be obtained by constructing the Jacobian matrix associated with the circuit transfer functions [76]. To this end, sensitivity of the transfer functions with respect to each faulty circuit parameter has to be explicitly calculated. Numerical methods, however, are of a very

high computational complexity and are inevitably subject to round-off errors. Testability measurement by symbolic analysis SAPEC [54] was proposed in [9]. However, the symbolic sensitivity computation still suffers the inherent circuit-size problem. Sensitivity with respective to a circuit parameter, on the other hand, can be efficiently performed in s-expanded DDDs by the COFACTOR operation once the transfer function is presented by s-expanded DDDs. The resulting method can handle much larger circuits in a more efficient way.

RLC-circuit macromodeling is also a promising application for DDDs. The macromodeling of RLC circuits is to approximate a RLC circuit to a simplified circuit (called a macromodel) such that both circuits have the same behavior at the input and output ports in both time and frequency domains. This problem arises in the simulation and timing analysis of digital circuits containing millions of RLC components typically extracted from circuit layouts due to parasitic effects of interconnects in deep submicron region. Macromodels can significantly reduce the CPU time used for circuit simulation. Many numerical methods have been proposed to solve this problem by using asymptotic waveform evaluation (AWE) [64], Padé via Lanczos algorithm (PVL) [20], Arnoldi algorithm [77] and congruence transformations [49, 63].

The extracted RLC networks for interconnects are essentially ladder-structured circuits. For ladder-structured circuits, DDDs have optimal representations. We can thereby derive the exact circuit transfer functions for huge RLC networks in s-expanded forms and then perform the Padé approximation to obtain the reduced-order models. For very large RLC networks with multiple interconnects, hierarchical symbolic analysis can be used where each RLC ladder circuit corresponding to a interconnect is partitioned as a subcircuit. By utilizing the efficiency of DDD-graph manipulations, DDDs promise a new valuable solution to the important macromod-

eling problem.

A third challenging research area for DDDs is with the simulation and optimization of the Power/Ground (p/g) networks in the VLSI circuits. Power/Ground networks connect the power/ground supplies within the circuit modules to the p/g pads on a chip. Improperly designed p/g network may cause malfunctions of logic circuits or even failure of chips due to electro-migration and excessive voltage drops. If the topology of a p/g network is fixed and all the circuit modules are modeled as independent current sources, the area of p/g networks can be efficiently optimized by using a sequence of linear programs [84], which is scalable even for huge p/g networks. For a sized p/g network, time-domain simulation with the presence of switching activities at each node of the network is important to precisely predict its dynamic behavior. A symbolic approach to deriving the time-domain waveforms at each node in a p/g network was proposed in [78]. The method consists of modeling transition switches as the initial conditions of the network under consideration, and solving circuit equations symbolically in frequency domain and relating back the results to time-time domain by the moment-matching technique [64]. The symbolic expressions are in sequence-of-expression forms. Responses in a new time step are computed by using the solution of previous steps as the initial condition. The effectiveness of this method, however, is determined by how symbolic expressions are constructed and evaluated. We note that most p/g networks are modeled as mashes of segments of RC ladder circuits. If each RC ladder segment is partitioned as a subcircuit for which DDD has a optimal representation, the resulting symbolic expression in terms of DDD size will be significantly smaller than that generated by the sequence of expressions method, and the DDD-based method will achieve a further speedup over simulation-based methods.

REFERENCES

[1] S. B. Akers, "Binary decision diagrams," *IEEE Trans. Computer*, vol. 27, no. 6, pp. 509-516, 1976.

[2] A. V. Aho, J. E. Hopcroft and J. D. Ullman, *The Design and Analysis of Computer Algorithm* Reading, Addison-Wesley, 1974.

[3] P. E. Alderson and P. M. Lin, "Computer generation of symbolic network functions - a new theory and implementation," *IEEE Trans. Circuit Theory*, vol. CT-20, no. 1, pp. 48-56, 1973

[4] J. W. Bandler and A. E. Salama, "Fault Diagnosis of Analog Circuits," in *Proc. of IEEE*, vol. 73, no. 8, pp. 1279-1327, Aug. 1985

[5] R. E. Bryant, "Graph-based algorithms for Boolean function manipulation," *IEEE Trans. Computer*, pp. 677–691, 1986.

[6] R. E. Bryant, "Binary decision diagrams and beyond: Enabling technologies for formal verification," in *Proc. Int. Conf. on Computer-Aided Design (ICCAD'95)*, 1995.

[7] K. S. Brace, R. L. Rudell and R. E. Byrant "Efficient implementation of a BDD package," in *Proc. 27th IEEE/ACM Design Automation Conf.*, pp. 40-45, 1990.

[8] J. G. Broida and S. G. Williamson, *A Comprehensive Introduction to Linear Algebra* Addison-Wesley, 1989.

[9] R. Carmassi, M. Catelani, G. Iuculano, A. Liberatore, S. Manetti and M. Marini, "Analog network testability measurement: a symbolic formulation approach," *IEEE Trans. Instrumentation and Measurement*, vol. 40, pp. 930-935, Dec. 1991.

[10] S.-M. Chang, J.-F. MacKey, and G. M. Wierzba, "Matrix reduction and numerical approximation during computation techniques for symbolic analog circuit analysis," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 1153-1156, 1992.

[11] B. W. Char, *et al.*, *Maple V: Language Reference Manual* Springer-Verlag, 1991.

[12] W. K. Chen, "Topological analysis for active networks," *IEEE Trans. Circuit Theory*, vol. CT-12, pp. 85-91, Mar. 1965

[13] W. K. Chen, *Applied Graph Theory: Graphs and Electrical Networks* American Elsevier, New York, 1976.

[14] T. Cormen, C. E. Leiserson and R. L. Rivest, *Introduction to Algorithms* The

MIT Press, Cambridge, Mass., 1990.

[15] E. F. Cota, M. Lubaszewski and E. J. Domenico, "A new frequency-domain analog test generation tool," in *Proc. Int. Conf. on Very Large Scale Integration*, pp. 503-514, 1997

[16] J. Cong, L. Hagen and A. Kahng, "Net partitioning yields better module partitions," in *Proc. 29th IEEE/ACM Design Automation Conf.*, pp. 47-52, 1992.

[17] M. Degrauwe *et al*, "IDAC: An interactive design tool for analog CMOS circuits," *IEEE J. of Solid-State Circuits*, vol. SC-22, no. 6, pp. 1106-1116, Dec. 1987.

[18] M. Degrauwe *et al*, "The ADAM analog design automation system," in *Proc. IEEE Int. Symp. Circuits and System*, pp. 820-822, 1990.

[19] S. Dutt and W. Deng, "A probability-based approach to VLSI circuit partitioning," in *Proc. 33th IEEE/ACM Design Automation Conf.*, pp. 100-105, 1996.

[20] P. Feldmanna and R. W. Freund, "Efficient linear circuit analysis by Padé approximation via the process," *IEEE Trans. Computer-Aided Design*, vol. 14, pp. 639-649, May 1995.

[21] P. Feldmanna and R. W. Freund, "Circuit noise evaluation by Padé approximation based model-reduction techniques," in *Proc. IEEE Int. Conf. Computer Aided Design (ICCAD)*, pp.132-138, 1997.

[22] F. V. Fernández, A. Rodríguez-Vázquez and J. L. Huertas, "A tool for symbolic analysis of analog integrated circuits including pole/zero extraction," in in *Proc. European Conf. Circuit Theory and Design*, pp. 752-761, 1991,

[23] F. V. Fernández, J. D. Martín, A. Rodríguez-Vázquez and J. L. Huertas, "On simplification techniques for symbolic analysis of analog integrated circuits," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 1149-1152, 1992.

[24] F. V. Fernández, A. Rodríguez-Vázquez, J. Martin and J. Huertas, "Formula approximation or flat and hierarchical symbolic analysis," *Kluwer J. Analog Integrated Circuits and Signal Process*, vol. 3, no. 1 pp. 43-58, Jan. 1993

[25] F. V. Fernández, P. Wambacq, G. Gielen, A. Rodríguez-Vázquez and W. Sansen. "Symbolic analysis of large analog integrated circuits by approximation during expression generation," in *Proc. IEEE Int. Symp. Circuits and Systems.*, pp. 25-28, 1994.

[26] F. V. Fernández and A. Rodríguez-Vázquez, "Symbolic analysis tools—the state of the art," in in *Proc. IEEE Int. Symp. Circuits and System*, pp. 798-801, 1996.

[27] H. Floberg, *Symbolic Analysis in Analog Integrated Circuit Design* Kluwer Academic Publishers, Mass., 1997.

[28] C.M. Fiduccia and R.M. Mattheyses, "A linear time heuristic for improved net-

work partitions," in *Proc. 19th ACM/IEEE Design Automation Conf.*, pp.175-181. 1982.

[29] G. Gielen, H. Walscharts and W. Sansen, "Analog circuit design optimization based on symbolic simulation and simulated annealing," *IEEE J. Solid-State Circuits*, vol. 25, no. 3, pp. 707-713, June 1990.

[30] G. Gielen and W. Sansen, *Symbolic Analysis for Automated Design of Analog Integrated Circuits* Kluwer Academic Publishers, 1991.

[31] G. Gielen, P. Wambacq and W. Sansen, "Symbolic analysis methods and applications for analog circuits: A tutorial overview," *Proc. IEEE*, vol. 82, no. 2, pp. 287-304, Feb. 1994.

[32] P. R. Gray and R. G. Meyer, *Analysis and Design of Analog Integrated Circuits*, 3rd edition, New York, John Wiley & Sons, 1993.

[33] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-completeness* W.H. Freeman, San Francisco, 1979.

[34] R. Harjani, R. Rutenbar and L. Carley, "OASYS: A framework for analog circuit synthesis," *IEEE Trans. Computer-Aided Design*, vol. 8, no.12, pp. 1247-1266, Dec. 1989.

[35] M. M. Hassoun and P. M. Lin, "A new network approach to symbolic simulation of large-scale network," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 806-809, 1989.

[36] M. M. Hassoun and P. M. Lin, "An efficient partitioning algorithm for large-scale circuits," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 2405-2408, 1990.

[37] M. M. Hassoun, B. Alspaugh and S. Burns, "A state-variable approach to symbolic circuit simulation in the time domain," in *Proc. Int. Symp. Circuits and Systems*, pp. 1589-1592, 1992.

[38] M. M. Hassoun and K. McCarville, "Symbolic analysis of large-scale networks using a hierarchical signal flow-graph approach," *J. Analog VLSI Signal Process,* vol. 3, pp. 31-42, Jan. 1993.

[39] M. M. Hassoun and P. M. Lin, "A hierarchical network approach to symbolic analysis of large scale networks," *IEEE Trans. Circuits and Systems*, vol. 42, no. 4, pp. 201-211, April 1995.

[40] J.-J. Hsu and C. Sechen, "DC small signal symbolic analysis of large analog integrated circuits," in *Proc. IEEE Trans. Circuits and Systems*, vol. 41, no. 12, pp. 817-828, Dec. 1994.

[41] J.-J. Hsu and C. Sechen, "Fully symbolic analysis of large analog integrated circuits," in *Proc. IEEE Custom Integrated Circuit Conf.*, pp. 457-460, 1994.

[42] J.-J. Hsu and C. Sechen, "Accurate extraction of simplified symbolic pole/zero expression for large analog ICs," in *Proc. Int. Symp. Circuits and Systems*, pp. 2083-2087, 1995.

[43] G. D. Hachtel and F. Somenzi, *Logic Synthesis and Verification Algorithm* Kluwer Academic Publishers, 1996.

[44] B. W. Kernighan and S. Lin, "An efficient heuristic procedure for partitioning graphs," *Bell Syst. Tech. J.* vol. 49, pp.291-307, 1970.

[45] H. Koh, C. Squin and P. Gray, "OPASYS: A compiler for CMOS operational amplifiers," *IEEE Trans. Computer-Aided Design*, vol. 9, no. 2, pp. 113-125, Feb. 1990.

[46] A. Konczykowska and M. Bon, "Automated design software for switch-capacitor IC's with symbolic simulator SCYMBAL," in in *Proc. IEEE/ACM Design Automation Conf.*, pp. 363-369, 1988.

[47] A. Konczykowska and M. Bon, "Analog design optimization using symbolic approach," in *Proc. Int. Symp. Circuits and Systems*, pp. 786-789, 1991.

[48] B. Krishnamurthy, "An improved min-cut algorithm for partitioning VLSI networks," *IEEE Trans. Computers*, vol. C-33, pp. 438-446, May 1984.

[49] K. J. Kerns, I. L. Wemple and A.T. Yang, "Stable and efficient reduction of subtract model networks using congruence transforms," in *Proc. IEEE/ACM Computer-Aided Design.*, pp. 207-214, 1995.

[50] J. Lee and R. Rohrer, "AWEsymbolic: Complied analysis of linear(ized) circuits using asymptotic waveform evaluation," in *Proc. 29th ACM/IEEE Design Automation Conf.*, pp. 213-218, 1992.

[51] P. M. Lin, *Symbolic Network Analysis*, Elsevier Science Publishers B.V., 1991.

[52] P. M. Lin, "Sensitivity analysis of large linear networks using symbolic program," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 1145-1148, 1992.

[53] A. Liberatore, S. Manetti and M. C. Piccirilli, "Network symbolic analysis for automated fault diagnosis," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 1169-1171, 1992.

[54] A. Liberatore and S. Manetti "SAPEC—A personal computer program for the symbolic analysis of electronic circuits," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 897-900, 1988.

[55] W. Mayeda and S. Seshu, "Topological formulas for networks," *Engineering Experimentation Station Bulletin 446,* University of Illinois, Urbana, 1957.

[56] S. J. Mason, "Feedback theory – some properties of signal flow graphs," in *Proc. IRE*, vol. 45, pp. 829-838, 1953.

[57] S. J. Mason and H. J. Zimmermann, *Electronic Circuits, Signals, and Systems.* Wiley, New York, 1960.

[58] W. J. McCalla and D. O. Pederson, "Elements of computer-aided analysis," *IEEE Trans. Circuit Theory*, vol. 18, pp. 14-26, 1971.

[59] S. Minato, "Zero-suppressed BDDs for set manipulation in combinatorial problems," in in *Proc. 30th IEEE/ACM Design Automation Conf.*, Dallas, TX, pp. 272-277, June 1993.

[60] S. Minato, *Binary Decision Diagrams and Application for VLSI CAD* Kluwer Academic Publishers, Boston, 1996.

[61] L. W. Nagel, *SPICE2: A Computer Program to Simulate Semiconductor Circuits*, Ph.D. Dissertation, Department of Electrical and Computer Engineering, University of California, Berkeley CA, May 1975.

[62] B. Noble, J. W. Danniel, *Applied Linear Algebra* Prentice-Hall, 1988.

[63] A. Odabasioglu, M. Celik and L. T. Pileggi, "Prima: passive reduced-order interconnect macromodeling algorithm," *IEEE Trans. on Computer-Aided Design*, vol. 17, no. 8, pp. 645-654, Aug. 1998.

[64] L. T. Pillage and R. A. Rohrer, "Asymptotic waveform evaluation for timing analysis," *IEEE Trans. Computer-Aided Design*, vol. 9, pp. 352-366, Apr. 1990.

[65] R. Rohrer, L. Nagel, R. Mayer and L. Weber, "Computationally efficient electronic-circuit noise calculations," *IEEE Journal of Solid-State Circuits*, vol. SC-6, no. 4, pp. 204-213, Aug. 1971.

[66] P. Sannuti and N. N. Puri, "Symbolic network analysis—an algebraic formulation," *IEEE Trans. Circuits and Systems*, vol. 27, no. 8, pp. 679-687, Aug. 1980.

[67] A. Sangiovanni-Vincentelli, L. -K. Chen and L. O. Chua, "An efficient heuristic cluster algorithm for tearing large-scale networks," IEEE Trans. on Circuits and Systems, vol. CAS-24, no. 12, Dec. pp. 709-717, 1977.

[68] S. J. Seda, M. G. R. Degrauwe and W. Fichtner, "A symbolic analysis tool for analog circuit design automation," in *Proc. IEEE Int. Conf. Computer Aided Design (ICCAD)*, pp. 488-491, 1988.

[69] S. J. Seda, M. G. R. Degrauwe and W. Fichtner, "Lazy-expansion symbolic expression approximation in SYNAP," in *Proc. IEEE Int. Conf. Computer Aided Design (ICCAD)*, pp. 310-317, 1992.

[70] J. A. Starzky and A. Konczykowska, "Flowgraph analysis of large electronic networks," *IEEE Trans. Circuits and Systems,* vol. 33, no. 3, pp. 302-315, March 1986.

[71] J. A. Starzky and E. Silwa, "Tolerance in symbolic network analysis," in *Proc.*

IEEE Int. Symp. Circuits and Systems, pp. 810-813, 1989.

[72] K. Singhal and J. Vlach, "Generation of immittance functions in symbolic form for lumped distributed active networks," *IEEE Trans. Circuit Theory*, vol. CAS-21, pp. 39-45, 1974.

[73] K. Swings and W. Sansen, "DONALD: A workbench for interactive design space exploration and sizing of analog circuits," in *Proc. European Design Automation Conference.* pp. 475-478, 1990.

[74] C. Sechen and D. Chen, "An improved objective function for mincut circuit partitioning," in *Proc. IEEE Int. Conf. Computer Aided Design (ICCAD)*, pp. 501-505, 1994

[75] L. A. Sanchis, "Multiple-way network partitioning," *IEEE Trans. on Computers*, vol. C38, no. 1, pp. 62-81, 1989.

[76] R. Saeks, N. Sen, H. M. S. Chen, K .S. Lu, S. Sangani and R. A. De Carlo, "Fault analysis in electronic circuits and systems," Inst. for Electron. Sci. Texas Tech. Univ. Lubbock, Tech. Rep., Jan. 1978

[77] L. M. Silveira, M. Kamon and J. White, "Efficient reduced-order modeling of frequency-dependent coupling inductances associated with 3-D interconnect structures," in *Proc. IEEE/ACM Design Automation Conf.*, pp. 376-380, 1995.

[78] J. C. Shah, A. A. Younis, S. S. Sapatnekar and M. M. Hassoun, "An algorithm for simulating power/ground networks using Padé approximants and its symbolic implementation," *IEEE Trans. Circuit and System II*, vol. 45, no. 10, pp. 1372-1382, Oct., 1998.

[79] C.-J. Shi and X.-D. Tan, "Symbolic analysis of large analog circuits with determinant decision diagrams," in *Proc. IEEE Int. Conf. Computer Aided Design (ICCAD)*, pp. 366-373, Nov. 1997.

[80] C.-J. Shi and X.-D. Tan, "Efficient derivation of exact s-expanded symbolic expressions for behavioral modeling of analog circuits," in *Proc. IEEE Custom Integrated Circuits Conf. (CICC)*, pp. 463-466, May 1998.

[81] X.-D. Tan, C.-J. Shi, D. Lungeanu, J.-C. F. Lee, L.-P. Yuan, "Reliability-contrainted area optimization of VLSI power/ground networks via sequence of linear programming," in *Proc. IEEE/ACM Design Automation Conf.*, New Orleans, LA, June 21-25, 1999.

[82] X.-D. Tan, J.-R. Tong, P.-S. Tang and F. Lombardi, "An efficient multi-way algorithm for balanced partitioning of VLSI circuits," *IEEE Int. Conf. Computer Design (ICCD)*, pp. 608-613, 1997.

[83] X.-D. Tan and C.-J. Shi, "Hierarchical symbolic analysis of large analog circuits with determinant decision diagrams," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 318-321, June 1998.

[84] X.-D. Tan and C.-J. Shi, "Balanced multi-level, multi-way partitioning of large analog circuits for hierarchical symbolic analysis," in *Proc. IEEE Asia and South Pacific Design Automation Conference (ASP-DAC)*, Hong Kong, pp. 1-4, Jan. 1999.

[85] X.-D. Tan and C.-J. Shi, "Interpretable symbolic small-signal characterization of large analog circuits using determinant decision diagram," in *Proc. IEEE Design, Automation and Test in Europe (DATE)*, Munich, Germany, pp. 448-453, Mar. 1999.

[86] J. Vlach and K. Singhal, *Computer Methods for Circuit Analysis and Design*, Van Nostrand Reinhold, New York, 1994.

[87] M. Vlach, "LU decomposition algorithms for parallel and vector computation," in *Analog Methods for Computer-Aided Circuit Analysis and Diagnosis*, T. Ozawa (ed.), Marcel Dekker, New York, pp. 37-64, 1988.

[88] P. Wambacq, G. Gielen and W. Sansen, "Symbolic simulation of analog circuits in s- and z-domain," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 814-817, 1989.

[89] P. Wambacq, G. Gielen and W. Sansen, "Symbolic simulation of harmonic distortion in analog integrated circuits with weak nonlinearities," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 536-539, 1990.

[90] P. Wambacq, G. Gielen and W. Sansen, "A cancellation-free algorithm for the symbolic simulation of large analog circuits," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 1157-1160, May 1992.

[91] P. Wambacq, *Symbolic Analysis of Large and Weakly Nonlinear Analog Integrated Circuits*, Ph.D thesis, Katholieke Universiteit Leuven, 1996.

[92] P. Wambacq, G. Gielen and W. Sansen, "A new reliable approximation method for expanded symbolic network functions," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 584-587, 1996.

[93] G. Wierzba *et al.*, "SSPICE - a symbolic SPICE program for linear active circuits," in *Proc. Midwest Symp. on Circuits and Systems,* 1989.

[94] T. Wei, M. T. Wong and Y. S. Lee, "Fault diagnosis of large scale analog circuits based on symbolic method," in *Proc. 3td IEEE Int. Mixed Signal Testing Workshop*, pp. 3-7, 1997.

[95] D. C. Yeh and V. B. Rao, "Partitioning issue in circuit simulation on multiprocessors," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 300-303, 1988.

[96] Z. You, E. Sánchez-Sinencio and J. P. de Gyvez, "Analog system-level fault diagnosis based on a symbolic method in the frequency domain," *IEEE Trans. Instrumentation and Measurement*, vol. 44, no. 1, pp. 28-35, Feb. 1995.

[97] Q. Yu and C. Sechen, "A unified approach to the approximate symbolic analysis

of large analog integrated circuits," *IEEE Trans. Circuits and Systems*, vol. 43, no. 8, pp. 656-669, Aug. 1996.

[98] Q. Yu and C. Sechen, "Efficient approximation of symbolic network functions using matroid intersection algorithm," in *Proc. IEEE Int. Conf. Computer Aided Design*, pp. 2088-2091, 1995.

[99] Q. Yu and C. Sechen, "Approximate symbolic analysis of large analog integrated circuits," in *Proc. IEEE Int. Symp. Circuits and Systems*, pp. 664-671, 1994.

[100] Q. Yu and C. Sechen, "Efficient approximation of symbolic network function using matroid intersection algorithm," *IEEE Trans. Computer-Aided Design*, vol. 16, no. 1, pp. 1073-1081, Oct. 1997.